STUDY HANDBOOK · OCTOBER 2026 EXAM VERSION

Databricks Certified Data Engineer Professional

A complete, exam-aligned guide to all nine domains of the **new exam that goes live on October 9, 2026** — concepts, code examples, exam traps, 140+ practice questions with explanations, and a final mixed mock exam.

QUESTIONS	TIME LIMIT
60 scored · MCQ	**120 minutes**

FEE	LANGUAGE
USD 200 (50% with festival voucher)	**English only · valid 2 years**

Prepared for Mike · Senior Data Engineer · Aligned to the official Databricks exam guide published September 2026 (new exam outline effective Oct 9, 2026) and the "Professional Data Engineering" learning pathway of the Advanced Learning Festival (Sep 16 – Oct 14, 2026).

This is an independent study aid and is not affiliated with or endorsed by Databricks. All practice questions are original.

Contents

Domain weights are the official percentages for the new exam (from Oct 9, 2026). Sections 1, 5, 2 and 3 together make up 62% of the exam.

INTRO Chapter 0 — How the Exam Works & How to Use This Handbook 3

23% Domain 1 — Developing Code for Data Processing (Python & SQL) 5

12% Domain 2 — Data Ingestion & Acquisition (incl. Lakeflow Connect, Iceberg, OpenSharing, Clean Rooms, Federation) 17

12% Domain 3 — Data Manipulation (incl. VARIANT, AI functions, expectations) 26

10% Domain 4 — Monitoring and Alerting (incl. SDK, Lakehouse alerts) 33

15% Domain 5 — Cost & Performance Optimization (incl. Predictive Optimization, Delta cache) 40

8% Domain 6 — Ensuring Data Security and Compliance (incl. ABAC & governed tags) 47

5% Domain 7 — Data Governance 55

10% Domain 8 — Debugging and Deploying 60

5% Domain 9 — Data Modeling (incl. Iceberg layouts, Metric Views) 66

REF Chapter 10 — Cross-Domain Cheat Sheets (Delta internals, Streaming, Commands, Checklist) 72

MOCK Chapter 11 — Final Mixed Mock Exam (20 questions) + Answer Key 76

CHAPTER 0

How the Exam Works & How to Use This Handbook

WHICH VERSION WILL YOU TAKE?

Databricks published a new exam guide in September 2026 that covers **two versions**: the current exam (available through **Oct 8, 2026**) and the new exam (live in Webassessor from **Oct 9, 2026**). The Learning Festival voucher is distributed on Oct 19, 2026, so anyone using it sits the **new** exam. This handbook is written for the new outline. If you happen to test before Oct 9, the old outline (10 domains, 59 questions) overlaps ~85% with this book — the extra material here won't hurt you.

Exam facts at a glance (new exam, from Oct 9, 2026)

Format	60 scored multiple-choice questions, plus up to 10 unscored pilot items you cannot identify. Four options, one correct answer. No labs, no free text.
Duration	120 minutes — under 2 minutes per question. Extra time is already factored in for unscored items.
Language	English only (the previous version was also offered in Japanese, Portuguese and Korean).
Delivery	Online proctored (Kryterion/WebAssessor) or a test centre. Webcam, ID, clean desk. No notes, no documentation, no second monitor.
Passing score	Not officially published. Third-party sources cite ~70%. Aim for 85%+ on timed practice exams before booking.
Prerequisites	None enforced. Databricks recommends ~1 year of hands-on data engineering on the platform plus the four self-paced courses below.
Validity / renewal	2 years. Recertify by passing the then-current version of the exam.
Retake	Full fee again; a 14-day wait applies between attempts. Book only when practice scores are consistently high.

The nine exam domains (official blueprint, new exam)

#	Domain	Weight	What it really tests
1	Developing Code for Data Processing using Python and SQL	23%	Bundle project structure, dependency troubleshooting on serverless/pipelines, Pandas/Python/SQL UDFs incl. Unity Catalog functions, Lakeflow Declarative Pipelines + Auto Loader, Lakeflow Jobs (UI/API/CLI), streaming table vs materialized view, AUTO CDC (SCD 1/2), Structured Streaming vs SDP, control flow, serverless compute & performance mode, high-memory tasks, retries, testing, **Structured Streaming stateful semantics** (watermarks, output modes, foreachBatch, checkpoints)
2	Data Ingestion & Acquisition	12%	Formats incl. **Iceberg**; Kafka/Kinesis/Pub/Sub; CDC pipelines with Delta or Iceberg targets; **Lakeflow Connect CDC from SQL Server, MySQL, PostgreSQL**; **OpenSharing (D2D, D2O) and Clean Rooms**; Lakehouse Federation with UC permissions and connection credentials
3	Data Manipulation	12%	Window functions, joins, aggregations; **VARIANT** (parse_json, variant_get); **AI functions (ai_query)** in pipelines; expectations to quarantine/drop/fail
4	Monitoring and Alerting	10%	System tables (billing, compute, access, lakeflow); REST/CLI/**SDK**; pipeline event logs; **Lakehouse alerts** (metrics, data quality & anomalies, cost, warehouse health, audit, AI agent quality, **job branching**); Jobs UI/API
5	Cost & Performance Optimization	15%	UC managed tables, Predictive Optimization, Liquid Clustering; deletion vectors, CLUSTER BY AUTO; **Delta cache**; CDF for incremental downstream; query profile bottlenecks; Liquid vs partitioning/Z-ORDER by table size and query pattern
6	Ensuring Data Security and Compliance	8%	Least-privilege ACLs on UC securables and workspace objects; **ABAC policies with governed tags**; anonymization/pseudonymization; PII detection & masking pipelines; purging for GDPR erasure
7	Data Governance	5%	UC tags and comments for discoverability; permission inheritance model
8	Debugging and Deploying	10%	Spark UI, cluster logs, system tables, query profiles; job repair & parameter overrides; event logs; Declarative Automation Bundles (formerly DABs); Git folders CI/CD
9	Data Modeling	5%	Delta/**Iceberg** layouts (partitioning to grain, clustering to access patterns, compaction); dimensional models with materialized views and **Unity Catalog Metric Views**

WHAT CHANGED VERSUS THE PREVIOUS EXAM (NOV 2025 → OCT 2026)

- 10 domains → 9. "Data Sharing and Federation" is folded into **Ingestion & Acquisition** (now 12%); Cost & Performance rises to 15%; Security drops to 8%, Governance and Modeling to 5% each.
- New named topics: Lakeflow Connect database CDC connectors, Iceberg tables, Clean Rooms, VARIANT, AI functions (`ai_query`), Structured Streaming stateful semantics, Databricks SDK, Lakehouse alerts and the SQL alert job task, Delta cache, ABAC policies with governed tags, Metric Views, serverless performance mode.
- Renamed products the guide now uses: **Lakeflow Declarative Pipelines** / **Apache Spark™ Declarative Pipelines** (= DLT), **Declarative Automation Bundles** (= Databricks Asset Bundles, renamed March 2026), **AUTO CDC** (= APPLY CHANGES), **OpenSharing** (= Delta Sharing), **Git folders** (= Repos), **Data quality monitoring** (= Lakehouse Monitoring).

Recommended preparation (from the exam guide) and how this handbook maps to it

Official self-paced course (Databricks Academy)	Chapters in this handbook
Advanced Techniques with Apache Spark™ Declarative Pipelines	1, 2, 3, 10 (Delta & streaming cheat sheets)
Databricks Data Privacy	6, 7
Databricks Performance Optimization	4, 5, 9, 10
Automated Deployment with Declarative Automation Bundles	1, 8
Instructor-led: Advanced Data Engineering with Databricks	All

The guide also points to Databricks' **AI Prep Guide** (use a chatbot as a tutor primed with the exam guide) and stresses **hands-on practice in Databricks Free Edition**; every domain chapter here ends with concrete lab suggestions in its answer key and the study plan below.

A four-week plan that fits the Learning Festival window

Week	Academy course	Handbook	Hands-on (do this in a real workspace)
Week 1	Advanced Techniques with SDP	Ch. 1, 2, 3 + Ch. 10 cheat sheets	Build bronze→silver→gold with Auto Loader, expectations, a VARIANT column and an AUTO CDC SCD2 table; add a watermark + windowed aggregation in Structured Streaming and restart it from its checkpoint.
Week 2	Performance Optimization	Ch. 5, 9, 10	Create a managed table with `CLUSTER BY AUTO` ; compare Query Profile before/after OPTIMIZE; force a skewed join and read the Spark UI; create a metric view and query it with `MEASURE()` .
Week 3	Data Privacy	Ch. 6, 7, 2 (sharing/federation)	Create a governed tag, tag a column, write one ABAC column-mask policy on a schema; run a GDPR delete + REORG + VACUUM flow; create a share and a foreign catalog.
Week 4	Declarative Automation Bundles	Ch. 4, 8 + Mock exam	Deploy a job and pipeline with `databricks bundle deploy -t dev` ; break a task and repair it via the SDK; build a SQL alert and use the SQL alert task to branch a job.

How to read the practice questions

Every domain chapter ends with 10–16 original scenario questions written in the style of the real exam, followed by an answer key that explains *why* the right answer is right and why the distractors are wrong. Do the questions **before** reading the answers, mark the ones you got wrong, and re-read the relevant section. The final chapter is a 20-question mixed mock; take it timed (40 minutes). The official guide also contains 10 retired sample questions aligned to the new outline — do those too.

EXAM-DAY TACTICS THAT WORK

- The Professional exam is scenario-based. Questions usually contain a constraint: "lowest cost", "least operational overhead", "minimal custom code", "without rewriting data", "exactly-once". The constraint decides between two plausible answers.
- When two answers both "work", the exam wants the **Databricks-native, managed** option (Lakeflow Connect over hand-written JDBC, SDP over hand-rolled streaming, UC managed tables over external, liquid clustering over partitioning, ABAC policies over per-table masks, system tables over scraping the UI, metric views over copy-pasted SQL).
- Eliminate answers that describe behaviour Databricks doesn't have (cross-task rollback, "trigger interval requires a new checkpoint", "VACUUM restores history", "streaming tables apply upstream deletes").
- Flag and move on. Under two minutes per question is enough only if you don't stall.

DOMAIN 1

Developing Code for Data Processing using Python and SQL

23% OF EXAM

The largest domain. It mixes software-engineering hygiene (project structure, dependencies, tests) with the two production pipeline technologies on Databricks: **Lakeflow Spark Declarative Pipelines** and **Lakeflow Jobs**. The new outline adds SQL and Unity Catalog UDFs, serverless compute configuration (performance mode) and, explicitly, **Structured Streaming stateful semantics**. Expect ~14 questions.

OFFICIAL OBJECTIVES (NEW EXAM)

- Implement scalable Python project structures for Declarative Automation Bundles (formerly DABs) to support modular development and CI/CD.
- Troubleshoot dependency conflicts and installation failures (PyPI, local wheels, source archives) across serverless, pipeline and bundle-deployed environments.
- Develop UDFs using Pandas, Python and SQL, including Unity Catalog functions, for given constraints.
- Create production-ready streaming pipelines with Lakeflow Declarative Pipelines and Auto Loader; create and automate ETL with Lakeflow Jobs via UI, API and CLI.
- Choose between a streaming table and a materialized view for a given latency, cost and refresh requirement. Implement CDC with AUTO CDC APIs (formerly APPLY CHANGES), SCD Type 1 and 2 via `stored_as_scd_type` .
- Choose between Spark Structured Streaming and Declarative Pipelines given operational constraints. Create jobs with If/Else and For Each control flow.
- Choose compute and configuration: serverless environments, dependency management, **performance mode**, high-memory notebook tasks, auto-optimization settings (e.g. disallowing retries).
- Unit and integration tests with `assertDataFrameEqual` , `assertSchemaEqual` , `DataFrame.transform` and testing frameworks.
- Apply Structured Streaming stateful-processing semantics: watermarks, output modes, `foreachBatch` , checkpoints for fault-tolerant exactly-once state recovery.

1.1 Project structure for Declarative Automation Bundles (formerly Databricks Asset Bundles)

A bundle is a folder with a `databricks.yml` at its root that declares *resources* (jobs, pipelines, clusters, dashboards, model endpoints) plus the source files they need. The recommended layout separates reusable library code from entry points so that code is unit-testable locally and deployable as a wheel:

```
my_project/
├── databricks.yml          # bundle name, targets (dev/staging/prod), variables, artifacts
├── resources/              # one YAML per resource: jobs, pipelines, clusters ...
│   ├── etl_job.yml
│   └── sdp_pipeline.yml
├── src/
│   └── my_project/         # importable Python package (built into a wheel)
│       ├── __init__.py
│       ├── transforms.py   # pure functions: DataFrame -> DataFrame
│       └── main.py         # entry point used by python_wheel_task
├── notebooks/              # thin notebooks that call the package
├── tests/
│   ├── unit/               # pytest, run locally or in CI (no cluster needed)
│   └── integration/        # run on Databricks against dev target
├── pyproject.toml / setup.py
└── requirements-dev.txt
```

Key ideas the exam checks: (1) business logic lives in a **package**, not in notebooks; (2) the package is built as a **wheel artifact** by the bundle (`artifacts:` block, `type: whl`) and referenced by tasks; (3) **targets** parameterise the same resources for dev/prod; (4) tests live beside the code and run in CI before `bundle deploy` .

```
# databricks.yml (excerpt)
bundle:
  name: my_project
artifacts:
  my_wheel:
    type: whl
    path: .
variables:
  catalog: {default: dev}
targets:
  dev:
    mode: development     # resources prefixed "[dev <user>]", schedules paused
    default: true
  prod:
    mode: production
    workspace: {host: https://prod.cloud.databricks.com}
    run_as: {service_principal_name: sp-etl}
    variables: {catalog: prod}
```

1.2 Libraries and dependencies

Mechanism	Scope	When to use / gotchas
`%pip install pkg==1.2`	Notebook session (all nodes)	Fast iteration. Must be at the top of the notebook; restarts the Python process (`dbutils.library.restartPython()`). Not for production jobs.
Task/cluster libraries (PyPI, Maven, wheel in a UC Volume or workspace file)	Cluster or job task	Production standard for classic compute. Wheels stored in **Unity Catalog volumes** (DBFS root is deprecated for libraries). Version-pin.
Serverless **environment** (`environment_key` , `dependencies:`)	Serverless job task / notebook	Serverless has no cluster libraries or init scripts; declare dependencies in the environment spec (PyPI names, wheel paths, `-r requirements.txt`). Same spec can set **memory: high** for notebook tasks.
`requirements.txt` as a library	Cluster / task	Supported directly as a library type; keeps one manifest for local, CI and Databricks.
Init scripts	Classic cluster only	For OS-level packages. Store in UC volumes. Not available on serverless. Avoid for Python deps.
Source archive (`.tar.gz`) / git URL	Notebook via pip	`%pip install https://.../pkg.tar.gz` or `git+https://...` . Needs network egress; build a wheel instead for reproducibility.

COMMON TROUBLESHOOTING QUESTIONS

- **ModuleNotFoundError on workers but not driver** → library was installed only in the driver's Python (e.g. `!pip` or `subprocess`). Use `%pip` or a cluster/task library so all nodes get it.
- **Version conflict with DBR-bundled package** → pin the version explicitly; check the Databricks Runtime release notes for pre-installed versions; prefer a wheel with pinned deps.
- **Wheel works in dev, fails in prod** → different DBR versions or a relative path; store the wheel in a UC volume and reference the absolute path; set `spark_version` in the bundle.
- **Library not found on serverless** → dependencies must be in the task's environment spec, not cluster libraries.

1.3 User-defined functions

Type	Execution	Performance	Use when
SQL UDF (`CREATE FUNCTION ... RETURN expr`)	Inlined by Catalyst; fully optimised, Photon-compatible	Best	Anything expressible in SQL. Also the mechanism for row filters and column masks (Ch. 6).
Python UDF (`@udf`)	Row-at-a-time; data serialised to a Python worker per row	Slowest; no Photon	Small data, complex logic with libraries, prototypes. Arrow-optimised variant (`useArrow=True`) reduces serialisation cost.

Type	Execution	Performance	Use when
Pandas UDF (`@pandas_udf`)	Vectorised via Apache Arrow; operates on `pd.Series` / `pd.DataFrame` batches	Much faster than row UDFs (often 10–100×)	Numeric / string transforms using pandas or ML libraries; batch inference.
Grouped map (`applyInPandas` , `mapInPandas`)	Whole group / partition as a pandas DataFrame	Fast, but whole group must fit in memory	Per-group model scoring, custom aggregations, complex reshaping.
UC Python UDF (`CREATE FUNCTION ... LANGUAGE PYTHON`)	Sandboxed; governed by Unity Catalog	Moderate	Share Python logic across SQL warehouses and clusters with UC permissions.

```
from pyspark.sql.functions import pandas_udf, udf
import pandas as pd

@pandas_udf("double")                    # Series -> Series, batched with Arrow
def to_celsius(f: pd.Series) -> pd.Series:
    return (f - 32) * 5 / 9

@udf("string")                           # row-at-a-time; avoid for large data
def classify(x):
    return "hot" if x > 30 else "ok"

df.withColumn("c", to_celsius("temp_f")).withColumn("label", classify("c"))
```

Unity Catalog functions (SQL and Python)

A UC function is a governed, reusable object (`catalog.schema.function`) with an owner and privileges (`EXECUTE`). It works from SQL warehouses, notebooks and pipelines alike, and it is the building block for row filters, column masks and ABAC policies (Ch. 6).

```
-- SQL UDF: inlined by the optimiser; can contain a scalar subquery; supports DEFAULT parameters
CREATE OR REPLACE FUNCTION prod.util.age_band(dob DATE) RETURNS STRING
  COMMENT 'Bucket a date of birth into 10-year bands'
  RETURN CONCAT(CAST(FLOOR(DATEDIFF(CURRENT_DATE(), dob) / 3652.5) * 10 AS STRING), 's');

-- SQL table-valued function
CREATE FUNCTION prod.util.recent_claims(days INT) RETURNS TABLE (claim_id BIGINT, amount DECIMAL(12,2))
  RETURN SELECT claim_id, amount FROM prod.silver.claims WHERE service_date >= CURRENT_DATE() - days;

-- Python UC UDF: runs in a secure sandbox; no network by default; can pip-install via ENVIRONMENT clause (DBR
17+)
CREATE OR REPLACE FUNCTION prod.util.normalize_npi(npi STRING) RETURNS STRING
  LANGUAGE PYTHON AS $$
import re
return re.sub(r'\D', '', npi or '')[:10]
$$;
GRANT EXECUTE ON FUNCTION prod.util.normalize_npi TO `etl_engineers`;
```

- Choose a **SQL UDF** when logic is expressible in SQL (fastest, Photon-compatible, usable in masks/filters). Choose a **UC Python UDF** when you need Python but want governance and reuse across SQL warehouses; accept the sandbox limits (per-row execution, limited libraries). Choose a **Pandas UDF** in a notebook/pipeline when throughput on large DataFrames matters and governance isn't required.
- Batch (vectorised) Python UC UDFs and the Arrow-optimised session UDFs reduce serialisation overhead; the exam mainly wants the ordering below.

> **EXAM RULE OF THUMB**
> Order of preference: built-in Spark functions → SQL UDF → Pandas UDF → Python UDF. Built-ins and SQL UDFs are visible to the optimiser and Photon; Python code is a black box. A question that says "a Python UDF is the bottleneck" wants you to vectorise (Pandas UDF) or replace with native functions.

1.4 Lakeflow Declarative Pipelines (SDP)

SDP is a declarative framework: you declare datasets and their dependencies; Databricks builds the DAG, manages checkpoints, retries, compute (incl. serverless), schema evolution, lineage and data-quality metrics. Two dataset types matter:

	Streaming table (ST)	Materialized view (MV)
Processing	Incremental; each update processes **only new rows** from an append-only source (Auto Loader, Kafka, another ST, Delta table).	Result of a query, **recomputed** on each update. With serverless, Databricks recomputes incrementally when it can (cost-based) — otherwise full recompute.
Source constraints	Source must be append-only. Updates/deletes upstream break it unless you use `skipChangeCommits` or a full refresh.	Any source, including tables that are updated/deleted, joins across many tables, aggregations.
State	Keeps checkpoint; exactly-once for the flow. Supports watermarks/windows.	No streaming state; always consistent with the current source snapshot.
Best for	Ingestion (bronze), append-only silver, low latency, high volume.	Aggregations, dimensions, gold layers, anything needing correctness over changing inputs.
Trade-off	Cheapest per row, lowest latency; cannot reflect upstream corrections.	Simplest correctness; can be expensive if a full recompute is triggered by a non-incrementalisable query.

```
-- SQL syntax (bronze -> silver -> gold)
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT *, current_timestamp() AS ingested_at, _metadata.file_path AS src
FROM STREAM read_files('/Volumes/raw/orders/', format => 'json');      -- Auto Loader

CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_id     EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_amount EXPECT (amount > 0)           ON VIOLATION FAIL UPDATE,
  CONSTRAINT has_email    EXPECT (email IS NOT NULL)            -- warn only, keep row
)
AS SELECT order_id, customer_id, CAST(amount AS DECIMAL(12,2)) amount, email, order_ts
FROM STREAM bronze_orders;

CREATE OR REFRESH MATERIALIZED VIEW gold_daily_revenue
AS SELECT DATE(order_ts) d, SUM(amount) revenue FROM silver_orders GROUP BY 1;
```

```
# Python syntax (same pipeline)
from pyspark import pipelines as dp          # legacy: import dlt
from pyspark.sql.functions import *

@dp.table(name="bronze_orders")              # streaming table because source is a stream
def bronze():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")
            .option("cloudFiles.schemaLocation", "/Volumes/raw/_schemas/orders")
            .load("/Volumes/raw/orders/"))

@dp.table(name="silver_orders")
@dp.expect_or_drop("valid_id", "order_id IS NOT NULL")
@dp.expect_or_fail("valid_amount", "amount > 0")
def silver():
    return spark.readStream.table("bronze_orders").select("order_id","customer_id","amount","order_ts")

@dp.materialized_view(name="gold_daily_revenue")
def gold():
    return spark.read.table("silver_orders").groupBy(to_date("order_ts").alias("d")).agg(sum("amount"))
```

Expectations (data quality) — three behaviours

- **Warn** (`EXPECT` / `@dp.expect`): row is kept, violation counted in the event log.
- **Drop** (`ON VIOLATION DROP ROW` / `expect_or_drop`): row discarded, counted.
- **Fail** (`ON VIOLATION FAIL UPDATE` / `expect_or_fail`): update aborts; the pipeline stops — use for invariants that must never be violated.
- `expect_all*` variants take a dict of rules. Expectations are visible in the UI and in `event_log()` (Ch. 4). Quarantining is covered in Ch. 3.

CDC with APPLY CHANGES INTO / AUTO CDC

Takes a change feed (rows with a key, a sequence column, and optionally an operation column) and maintains a target table as **SCD Type 1** (overwrite) or **SCD Type 2** (history with `__START_AT` / `__END_AT`). It handles out-of-order events using `SEQUENCE BY` and deduplicates by key — the main reason to use it instead of hand-written MERGE in `foreachBatch` .

```
CREATE OR REFRESH STREAMING TABLE customers_scd2;

CREATE FLOW customers_cdc AS AUTO CDC INTO customers_scd2   -- older syntax: APPLY CHANGES INTO
FROM STREAM cdc_customers_raw
KEYS (customer_id)
APPLY AS DELETE WHEN op = 'DELETE'
SEQUENCE BY event_ts
COLUMNS * EXCEPT (op, event_ts)
STORED AS SCD TYPE 2
TRACK HISTORY ON (email, address);       -- only these columns open a new version
```

```
dp.create_streaming_table("customers_scd2")
dp.create_auto_cdc_flow(            # legacy: dlt.apply_changes(...)
    target="customers_scd2", source="cdc_customers_raw",
    keys=["customer_id"], sequence_by=col("event_ts"),
    apply_as_deletes=expr("op = 'DELETE'"),
    except_column_list=["op", "event_ts"], stored_as_scd_type=2)
```

TRAPS

- The target of AUTO CDC is a streaming table you create first (`CREATE OR REFRESH STREAMING TABLE t;` with no query). You **cannot** read a CDC target as a streaming source in the same pipeline the normal way (it is not append-only) — read it as a batch source or use Change Data Feed.
- `SEQUENCE BY` must be monotonically meaningful (timestamp/version). Without it, out-of-order updates corrupt the target.
- SCD2 with `TRACK HISTORY ON` limits which column changes create a new row — a cost/accuracy tuning point.
- Full refresh on the target reprocesses all source data; a snapshot-based source requires `AUTO CDC FROM SNAPSHOT` instead.

Structured Streaming vs Spark Declarative Pipelines

	Structured Streaming (hand-written)	Spark Declarative Pipelines
You manage	Checkpoints, retries, cluster, schema evolution, error handling, monitoring, table creation, ordering of multiple streams	Only the transformations and expectations; the rest is managed (serverless optional, autoscaling, event log, lineage)
Flexibility	Maximum: `foreachBatch`, arbitrary sinks (Kafka, JDBC), `applyInPandasWithState`, custom triggers, multiple queries per job	Declarative datasets, sinks (Delta, Kafka via `create_sink`), expectations, AUTO CDC, flows. Less arbitrary code.
Latency	Any trigger interval, continuous mode (experimental)	Triggered (scheduled) or continuous; real-time mode available for serverless
Testing / CI	Standard PySpark tests	Development mode, validate-only runs, bundle deploys
Choose when	You need a non-Delta sink, custom stateful logic, or complex MERGE semantics not covered by AUTO CDC	Standard medallion ETL, CDC, data-quality rules, minimal ops — the default answer on the exam

1.5 Lakeflow Jobs: orchestration, control flow, configuration

A job is a DAG of tasks (notebook, Python script, Python wheel, SQL, SDP pipeline, dbt, JAR, Spark submit, **Run job**, **If/else**, **For each**, dashboard, Power BI). Tasks can share compute, run on **job clusters** (created per run, cheaper DBU rate, destroyed after) or **serverless**.

Control flow

- **If/else condition task**: evaluates an expression (e.g. `{{tasks.check.values.row_count}} > 0`) and routes to true/false branches.
- **For each task**: iterates over a JSON array (literal or from a task value) running a nested task per element, with a **concurrency** setting.
- **Run if** dependency rule per task: `ALL_SUCCESS` (default), `AT_LEAST_ONE_SUCCESS`, `NONE_FAILED`, `ALL_DONE`, `AT_LEAST_ONE_FAILED`, `ALL_FAILED` — use `ALL_DONE` for a cleanup task that must always run.
- **Task values**: `dbutils.jobs.taskValues.set(key, value)` in one task, `get(taskKey, key)` in another; referenced in UI as `{{tasks.<task>.values.<key>}}`.
- **Run job task** calls another job (modular orchestration).

Parameters & dynamic values

- **Job parameters** (key/value at job level) are pushed to every task; override them on `run-now` or repair.
- Dynamic value references: `{{job.id}}`, `{{job.run_id}}`, `{{job.start_time.iso_date}}`, `{{job.parameters.env}}`, `{{task.name}}`.
- Notebook tasks read them with `dbutils.widgets.get("env")`; wheel tasks receive them as CLI args; SQL tasks as `{{param}}`.

Triggers

- Scheduled (cron, timezone, pause status)
- **File arrival** on a UC volume/external location
- **Table update** trigger (run when Delta tables change)
- **Continuous** (job restarts automatically — for streaming)
- Manual / API `run-now`

Reliability settings

- **Retries**: `max_retries` per task (+ `min_retry_interval_millis`, `retry_on_timeout`). Set **0 retries for non-idempotent tasks** (e.g. appending without dedup, sending emails).
- **Timeout** per task/job; **max concurrent runs**; **queueing** when limits are hit.
- **Health rules**: duration warnings, streaming backlog thresholds → notifications (Ch. 4).
- **Run as** a service principal in prod so runs don't break when a user leaves.

Compute configuration choices

- **Job cluster** for scheduled batch: created for the run, lower DBU price than all-purpose, no idle cost. **All-purpose (interactive)** cluster: for development only; expensive and shared.
- **Serverless**: fastest start, autoscaling, no cluster to manage; dependencies come from the task's **environment** (base environment version + `dependencies` list). **Performance mode**: *standard* (cost-optimised; may queue/start slower) vs *performance optimized* (faster start and scaling at higher cost) — set per job (`performance_target`) and per pipeline. Notebook tasks on serverless can request **high memory** in the environment when pandas/driver-heavy code OOMs.
- Choose performance mode for latency-sensitive or SLA-bound jobs; standard for overnight batch where a few minutes of delay is free savings.
- **Photon** on for SQL/DataFrame-heavy work; off for pure Python/RDD workloads.
- Spot/preemptible workers for cost, with on-demand driver; autoscaling for variable loads; **single-node** for small pandas/ML tasks.

```
# Create / run / inspect a job with the CLI and REST API
databricks jobs create --json @job.json
databricks jobs run-now 123 --json '{"job_parameters": {"env": "prod", "run_date": "2026-09-22"}}'
databricks jobs list-runs --job-id 123 --active-only
curl -X GET "$HOST/api/2.2/jobs/runs/get?run_id=456" -H "Authorization: Bearer $TOKEN"
```

1.6 Testing and debugging

Write transformations as **pure functions** (`DataFrame → DataFrame`) and chain them with `DataFrame.transform`. Unit tests run with pytest on a local or Databricks Connect SparkSession; integration tests run the deployed job against the dev target.

```
# src/my_project/transforms.py
from pyspark.sql import DataFrame, functions as F
def clean_orders(df: DataFrame) -> DataFrame:
    return df.filter("amount > 0").withColumn("d", F.to_date("order_ts"))
def add_tax(rate: float):
    def _inner(df: DataFrame) -> DataFrame:
        return df.withColumn("tax", F.col("amount") * rate)
    return _inner

result = raw.transform(clean_orders).transform(add_tax(0.2))   # composable, testable

# tests/unit/test_transforms.py
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual
def test_clean_orders(spark):
    src = spark.createDataFrame([(1, 10.0, "2026-01-01 10:00:00"), (2, -5.0, "2026-01-01 10:00:00")],
                                "order_id INT, amount DOUBLE, order_ts STRING")
    expected = spark.createDataFrame([(1, 10.0, "2026-01-01 10:00:00", "2026-01-01")],
                                     "order_id INT, amount DOUBLE, order_ts STRING, d DATE")
    out = clean_orders(src)
    assertSchemaEqual(out.schema, expected.schema)
    assertDataFrameEqual(out, expected)              # row order ignored by default; checkRowOrder=True to
enforce
```

- `assertDataFrameEqual(actual, expected, checkRowOrder=False, rtol=1e-5, atol=1e-8, ignoreNullable=True)` — compares schema and data, with tolerance for floats; also accepts lists of Rows or pandas DataFrames.
- `assertSchemaEqual` compares StructTypes (nullability ignored by default).
- Use pytest fixtures for a SparkSession; on Databricks the `spark` global exists, so fixtures should `try: from databricks.connect import DatabricksSession` fallback.
- **Notebook debugger**: the Databricks notebook has an interactive Python debugger (breakpoints, step, variable explorer — pdb-based, "Debug cell") for driver-side code. It cannot step inside executors/UDFs; debug those with small local pandas samples.
- SDP: use **Development mode** (cluster reused, no retries, fast iteration) and **validate** (check the DAG and SQL without processing data).

1.7 Structured Streaming stateful semantics (explicit objective)

Even when you use Declarative Pipelines, the engine underneath is Structured Streaming. The exam now names four mechanisms and asks you to combine them for **bounded state** and **exactly-once recovery**.

Mechanism	What it does	Exam-relevant rules
Checkpoint	Write-ahead log of source offsets + committed batch ids + operator state (RocksDB by default on Databricks)	One checkpoint per query, never shared. On restart the query replays from the last committed offsets and restores state, so with a replayable source (Kafka, files, Delta) and an idempotent sink (Delta) you get **exactly-once**. Changing trigger interval or rate limits is safe; changing source, sink or a stateful operator's structure needs a new checkpoint.
Watermark	`withWatermark("event_ts", "30 minutes")` : the engine tracks max event time seen minus 30 min; events older than that are late	Required for append-mode aggregations, stream-stream joins and `dropDuplicatesWithinWatermark` ; the only way to **bound state** for event-time operations. Late events beyond the watermark are dropped (quarantine them upstream if you must keep them).
Output mode	What is emitted per micro-batch	**append**: only finalised rows (windows are emitted once the watermark passes their end) — the mode for Delta sinks and streaming tables. **update**: rows changed in this batch (needs an upsert-capable sink → foreachBatch/MERGE). **complete**: entire result every batch; only for aggregations; state unbounded, no watermark pruning.
foreachBatch	Custom batch function per micro-batch	Use for MERGE/upserts, multiple sinks, JDBC. At-least-once by default on retries; make it idempotent with MERGE, or pass `txnVersion` / `txnAppId` on Delta writes so replayed batches are skipped.

```
# Bounded state + exactly-once recovery for a windowed aggregation
(spark.readStream.table("silver.events")
   .withWatermark("event_ts", "30 minutes")                # late data tolerance -> state eviction
   .groupBy(window("event_ts", "10 minutes"), "device_id").count()
 .writeStream.outputMode("append")                         # emit finalised windows only
   .option("checkpointLocation", "/Volumes/prod/_chk/events_10m")   # offsets + state for restart
   .trigger(processingTime="1 minute")
   .toTable("gold.events_10m"))

# Custom upsert with idempotent writes
def upsert(batch_df, batch_id):
    (batch_df.write.format("delta")
        .option("txnVersion", batch_id).option("txnAppId", "events_upsert")   # replay-safe
        .mode("append").saveAsTable("silver.events_dedup"))
q = stream.writeStream.foreachBatch(upsert).option("checkpointLocation", chk).start()
```

DISTRACTORS TO RECOGNISE

- "Use complete mode so nothing is lost" — complete mode keeps all state forever; it does not solve lateness or recovery.
- "Disable checkpointing to restart faster" — you lose exactly-once and state.
- "Commit offsets manually to a Delta table" — reinventing the checkpoint; only `foreachBatch` idempotency is your job.
- "Increase shuffle partitions to bound state" — partitions change parallelism, not state retention.

Domain 1 — Practice Questions

16 questions · answers on the next page

Q1 **A team stores all transformation logic in notebooks that are called directly by job tasks. Unit testing requires a running cluster and PRs cannot be validated in CI. Which change best addresses this while preparing the project for deployment with Databricks Asset Bundles?**

- **A.** Convert every notebook to a SQL task so tests can run in a SQL warehouse.
- **B.** Move transformation functions into a Python package under `src/`, build it as a wheel artifact in the bundle, keep notebooks as thin entry points, and add pytest unit tests that run in CI.
- **C.** Add `%run` commands so notebooks share code, and run all tests in the production job.
- **D.** Store the notebooks in DBFS and reference them by absolute path from the bundle.

Q2 **A serverless job task fails with `ModuleNotFoundError: No module named 'great_expectations'`. The same code works on an all-purpose cluster where the library is installed as a cluster library. What should the engineer do?**

- **A.** Add an init script that pip-installs the library on the serverless compute.
- **B.** Declare the package in the task's serverless **environment** dependencies (or a `requirements.txt` referenced there).
- **C.** Install the library on the all-purpose cluster with "Install for all clusters".
- **D.** Copy the library source into DBFS root and add it to `sys.path` in the notebook.

Q3 **A Python UDF that parses a string column with a regular expression is the slowest stage of a 2-billion-row job. Which change gives the largest speed-up with the least rewriting?**

- **A.** Increase `spark.sql.shuffle.partitions`.
- **B.** Enable Photon on the cluster.
- **C.** Replace the UDF with the built-in `regexp_extract` function; if not possible, rewrite it as a Pandas UDF.
- **D.** Cache the DataFrame before applying the UDF.

Q4 **A silver table in a Spark Declarative Pipeline is defined as a streaming table reading from a bronze Delta table. A backfill job now runs `UPDATE` statements on bronze to fix historical prices. What happens on the next pipeline update?**

- **A.** The streaming table silently applies the updates.
- **B.** The flow fails because the source is no longer append-only; the team must either full-refresh the streaming table, set `skipChangeCommits`, or redefine silver as a materialized view.
- **C.** The pipeline converts the streaming table to a materialized view automatically.
- **D.** Only rows inserted after the update are processed; updated rows are ignored without error.

Q5 **Which statement about materialized views in Spark Declarative Pipelines is correct?**

- **A.** They can only be refreshed by a full recompute, regardless of compute type.
- **B.** They require an append-only source.
- **C.** They always reflect the current state of their sources after a refresh, and on serverless compute Databricks refreshes them incrementally when it determines that is cheaper.
- **D.** They maintain a streaming checkpoint that can be reset with `RESET CHECKPOINT`.

Q6 **A CDC feed from an operational database delivers `customer_id`, `op` (I/U/D), `event_ts` and attribute columns. Events can arrive out of order. The team needs a table that keeps full history of address changes with validity intervals. Which approach requires the least custom code?**

- **A.** A `foreachBatch` MERGE that closes and opens rows manually.
- **B.** A materialized view with a window function over the raw feed.
- **C.** AUTO CDC / APPLY CHANGES INTO a streaming table with `KEYS (customer_id)`, `SEQUENCE BY event_ts`, `APPLY AS DELETE WHEN op='D'`, `STORED AS SCD TYPE 2`.
- **D.** A streaming table that appends every event and a view that filters the latest.

Q7 **A job has three tasks: `extract` → `transform` → `notify_ops`. The ops team wants `notify_ops` to run whether or not `transform` succeeds, so they always get a message. What should be configured?**

- **A.** Set the job's retry policy to 3 on `transform`.
- **B.** Set the **Run if** dependency of `notify_ops` to `ALL_DONE`.
- **C.** Remove the dependency and run `notify_ops` in parallel with `extract`.
- **D.** Wrap `transform` in a try/except that swallows the error.

Q8 **A nightly job must process a list of 40 regions produced by an upstream task, with at most 5 regions processed at a time. Which job design meets this without creating 40 tasks?**

- **A.** A **For each** task whose inputs reference the upstream task value (`{{tasks.list_regions.values.regions}}`) with concurrency set to 5.
- **B.** An If/else task that loops until the list is empty.
- **C.** A Run job task with `max_concurrent_runs = 5`.
- **D.** A Python task using `ThreadPoolExecutor` inside the notebook, since jobs cannot iterate.

Q9 **A task appends daily records to a Delta table and then calls an external payment API. Retries were set to 3 and duplicate payments were sent after a transient failure. What is the correct fix?**

- **A.** Set the task's retries to 0 (disallow retries) for the non-idempotent task and make failures visible through notifications; make the append idempotent separately.
- **B.** Increase retries to 5 so the run eventually succeeds.
- **C.** Move the task to serverless, which never retries.
- **D.** Enable auto-optimisation on the target table.

Q10 **A notebook task on serverless compute fails with a driver out-of-memory error while building a large pandas DataFrame for a report. The logic cannot be changed this sprint. Which configuration is the most direct fix?**

- **A.** Set `spark.sql.shuffle.partitions` to 2000.
- **B.** Select the **high memory** option in the task's serverless environment settings.
- **C.** Enable Photon.
- **D.** Add an init script to increase the JVM heap.

Q11 **Which test assertion correctly compares two DataFrames while ignoring row order and small floating-point differences?**

- **A.** `assert actual.collect() == expected.collect()`
- **B.** `assertDataFrameEqual(actual, expected)` from `pyspark.testing` (default `checkRowOrder=False`, with rtol/atol tolerances)
- **C.** `assertSchemaEqual(actual, expected)`
- **D.** `assert actual.subtract(expected).count() == 0`

Q12 **A team must decide between hand-written Structured Streaming and Spark Declarative Pipelines for a pipeline that reads Kafka, applies validation rules with metrics, deduplicates late events, writes Delta, and must be maintained by analysts with limited Spark experience. Which recommendation is best?**

- **A.** Structured Streaming with `foreachBatch`, because SDP cannot read Kafka.
- **B.** Spark Declarative Pipelines: managed checkpoints and retries, expectations provide validation metrics, and streaming tables with watermarks handle dedup — lowest operational burden.
- **C.** Structured Streaming, because expectations are only available for batch tables.
- **D.** A scheduled batch job with COPY INTO.

Q13 **In a bundle, target `dev` uses `mode: development`. Which behaviour should the engineer expect after `databricks bundle deploy -t dev`?**

- **A.** Job schedules are paused and resource names are prefixed with the deploying user's name so several engineers can deploy the same bundle without collisions.
- **B.** Jobs run as a service principal and schedules are activated immediately.
- **C.** The deployment is rejected unless the Git branch is `main`.
- **D.** Resources are deployed with identical names to production, overwriting them.

Q14 **A Structured Streaming query aggregates events into 5-minute windows. Very late events cause the state store to grow without bound, and after a driver restart some windows were double-counted. Which configuration fixes both problems?**

- **A.** Set a watermark on the event-time column and keep the query's checkpoint location so offsets and state are restored on restart.
- **B.** Use complete output mode without a watermark.
- **C.** Delete the checkpoint before each restart and increase shuffle partitions.
- **D.** Wrap the aggregation in foreachBatch and commit offsets manually.

Q15 **A masking rule "keep only the last 4 characters" must be reused by SQL warehouse users, pipeline code and an ABAC column-mask policy, with EXECUTE governed centrally. What should be created?**

- **A.** A Pandas UDF registered in each notebook
- **B.** A Unity Catalog SQL function (`CREATE FUNCTION catalog.schema.last4(s STRING) RETURNS STRING RETURN right(s, 4)`) with EXECUTE granted to the relevant groups
- **C.** A `%run` -shared notebook containing the Python logic
- **D.** A Scala UDF in a JAR installed on every cluster

Q16 **A serverless job feeds an operational dashboard and must start within seconds of its trigger; another serverless job runs an overnight backfill where a slower start is acceptable. Which configuration matches each?**

- **A.** Performance-optimized mode for the dashboard job; standard (cost-optimised) mode for the backfill.
- **B.** Standard mode for both; performance mode only affects Photon.
- **C.** High-memory environment for the dashboard job; performance mode for the backfill.
- **D.** Classic job clusters for both, since serverless cannot be tuned.

Domain 1 — Answer Key

Q	Ans	Explanation
1	B	Package + wheel + thin notebooks + pytest in CI is the DAB-recommended structure. A and D change nothing about testability; C (`%run`) keeps logic un-importable and tests in prod are wrong.
2	B	Serverless compute has no cluster libraries or init scripts; dependencies are declared in the task environment. A is impossible on serverless, C affects only classic clusters, D is a hack that breaks on workers.
3	C	Native functions are optimiser- and Photon-visible; Pandas UDFs vectorise via Arrow. Photon (B) does not accelerate Python UDFs; shuffle partitions and caching don't address per-row Python overhead.
4	B	Streaming tables require append-only sources. Detecting non-append changes raises an error. Options: full refresh, `skipChangeCommits` (ignore the changed rows), or redefine as an MV which handles updates correctly.
5	C	MVs are always consistent with sources after a refresh; serverless SDP uses cost-based incremental refresh where possible. They have no append-only requirement (B) and no checkpoint (D). A is wrong because incremental refresh exists on serverless.
6	C	AUTO CDC/APPLY CHANGES handles ordering (SEQUENCE BY), deletes, dedup and SCD2 validity columns natively. A works but is far more code; B and D don't produce validity intervals with deletes handled.
7	B	`ALL_DONE` runs the task after all dependencies finish regardless of outcome. Retries don't guarantee notification; parallel execution loses the ordering; swallowing errors hides failures.
8	A	The For each task iterates over an array (literal or from a task value) with a concurrency limit. B, C and D misuse other features; D works but ignores the native control-flow operator the exam wants.
9	A	Retries re-execute the whole task; a non-idempotent side effect (payment call) must not be retried. The exam objective literally says "auto-optimization to disallow retries" for such tasks.
10	B	Serverless notebook tasks can be given high memory through the environment settings. Shuffle partitions and Photon don't change driver memory; init scripts don't exist on serverless.
11	B	`assertDataFrameEqual` compares schema and data with order-insensitivity and float tolerances by default. A is order-sensitive and exact; C compares only schema; D ignores duplicates and schema.
12	B	SDP reads Kafka, supports expectations with metrics in the event log, and streaming tables support watermarks for dedup. It minimises operational work for a less Spark-savvy team. A and C are factually wrong; D isn't streaming.
13	A	Development mode prefixes resource names with `[dev <username>]` , pauses schedules/triggers, and enables concurrent runs and code sync for iteration. Production mode is what enforces service principals and clean names.
14	A	Watermark bounds state by evicting old windows; the checkpoint restores offsets and state so re-execution is exactly-once. Complete mode retains everything; deleting the checkpoint reprocesses; manual offsets reinvent the checkpoint.
15	B	UC SQL functions are governed (EXECUTE), engine-agnostic and the only function type usable in ABAC/mask policies. Notebook-local UDFs and JARs are neither governed nor usable from warehouses.
16	A	Performance mode trades cost for fast start/scale; standard mode is the cost-optimised default. High memory addresses OOM, not latency.

DOMAIN 2

Data Ingestion & Acquisition

12% OF EXAM

Weight jumped from 7% to 12% and the scope widened: file formats now include **Iceberg**; message buses name Kafka, Kinesis and Pub/Sub; managed **Lakeflow Connect** CDC from SQL Server/MySQL/PostgreSQL is in; and the old "Sharing & Federation" domain (OpenSharing, Clean Rooms, Lakehouse Federation) now lives here. Expect ~7 questions.

OFFICIAL OBJECTIVES (NEW EXAM)

- Develop ingestion pipelines for Delta Lake, Parquet, JSON, Iceberg, CSV and Binary from message buses (Kafka, Kinesis, Pub/Sub) and cloud storage.
- Configure incremental CDC pipelines using Lakeflow Pipelines with Delta or Iceberg as the target table format.
- Configure CDC ingestion pipelines from relational databases (SQL Server, MySQL, PostgreSQL) using Lakeflow Connect.
- Implement OpenSharing (D2D and Databricks-to-Open) and Clean Rooms for privacy-preserving collaboration.
- Configure Lakehouse Federation with governance across supported source systems, applying UC permissions and connection-level credentials.

2.1 File formats and their read options

Format	Reader	Key options / notes
Delta	`spark.read.table()`, `.format("delta")`, `readStream.table()`	Schema in the log; time travel; streaming source options `ignoreDeletes`, `skipChangeCommits`, `startingVersion`, `maxFilesPerTrigger`, `readChangeFeed`.
Iceberg	`USING iceberg` (UC managed Iceberg), foreign Iceberg tables via Glue/HMS/Snowflake Horizon catalogs, `format("iceberg")`	Open table format with hidden partitioning, snapshots, schema evolution. On Databricks: **managed Iceberg tables** (UC owns storage + maintenance, DBR 16.4+), readable/writable by external engines through the **Iceberg REST Catalog**; or **UniForm** (`delta.universalFormat.enabledFormats = 'iceberg'`) to expose a Delta table with Iceberg metadata. Declarative pipeline tables can target Iceberg.
Parquet	`format("parquet")`	Columnar, schema in footer. `mergeSchema` when files differ. `CONVERT TO DELTA parquet.`path`` upgrades in place.
ORC	`format("orc")`	Columnar (Hive world). Same handling as Parquet; also supported by `CONVERT TO DELTA`.
Avro	`format("avro")`	Row-based, schema embedded; typical for Kafka payloads. `from_avro(col, jsonSchema)` / `to_avro`; Schema Registry integration.
JSON	`format("json")`	Schema inferred (expensive on big data — sample or provide schema). `multiLine` for pretty JSON; nested structs; `from_json` for JSON strings; use **VARIANT** type for semi-structured columns.
CSV	`format("csv")`	`header`, `inferSchema`, `sep`, `quote`, `escape`, `multiLine`, `mode` (PERMISSIVE default with `_corrupt_record`, DROPMALFORMED, FAILFAST), `badRecordsPath`.
XML	`format("xml")` (native since DBR 14.3)	`rowTag` mandatory; `from_xml` / `schema_of_xml`; Auto Loader supports `cloudFiles.format = "xml"`.
Text	`format("text")`	One row per line in a `value` column; `wholetext=true` for whole file per row.
Binary	`format("binaryFile")`	Columns `path, modificationTime, length, content` (bytes); for images/PDFs. `pathGlobFilter` to select extensions.

The `read_files()` table-valued function is the SQL equivalent (`SELECT * FROM read_files('/Volumes/...', format => 'csv', header => true)`) and, with `STREAM read_files(...)`, invokes Auto Loader. The `_metadata` column exposes `file_path`, `file_name`, `file_modification_time` — useful for lineage.

2.2 Auto Loader (cloudFiles)

Auto Loader incrementally and idempotently discovers new files in cloud storage and feeds them to Structured Streaming. It scales to **billions of files**, tracks processed files in a RocksDB store inside the checkpoint, and supports schema inference/evolution.

```
(spark.readStream.format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", "/Volumes/raw/_schemas/events")  # required for inference
   .option("cloudFiles.inferColumnTypes", "true")       # default: all strings (except nested)
   .option("cloudFiles.schemaHints", "amount DECIMAL(12,2), ts TIMESTAMP")
   .option("cloudFiles.schemaEvolutionMode", "addNewColumns")   # default
   .option("cloudFiles.maxFilesPerTrigger", 1000)
   .load("/Volumes/raw/events/")
 .writeStream.option("checkpointLocation", "/Volumes/raw/_chk/events")
   .option("mergeSchema", "true")
   .trigger(availableNow=True)                          # batch-style run over all new files
   .toTable("bronze.events"))
```

Option	Meaning
`cloudFiles.schemaEvolutionMode`	`addNewColumns` (default: stream **fails** once, schema updated, restart succeeds), `rescue` (new columns go to `_rescued_data`), `failOnNewColumns`, `none`.
`_rescued_data`	JSON column capturing data that didn't match the schema (type mismatch, extra columns, case mismatch). Always present when schema is inferred — use it for quarantining (Ch. 3).
Directory listing vs **file notification** (`cloudFiles.useNotifications`) vs **file events** (`cloudFiles.useManagedFileEvents`)	Listing is default (scales with incremental listing on lexically ordered names). Notification mode uses cloud queues for very large directories/low latency; managed file events on UC external locations is the newest, no-ops option.
`cloudFiles.backfillInterval`	Periodic full listing to catch files missed by notifications.
`cloudFiles.allowOverwrites`	Reprocess a file when it is overwritten (default false: files are processed exactly once by path).
`cloudFiles.includeExistingFiles`	Whether to process files already present (default true).

2.3 COPY INTO vs Auto Loader vs plain batch read

	COPY INTO	Auto Loader	Batch `spark.read` + append
Nature	SQL batch command, idempotent (tracks loaded files in the table's metadata)	Streaming source, incremental, checkpoint-tracked	Full re-read every run
Scale	Thousands of files; re-lists directory each run	Millions/billions of files; notifications/file events avoid listing	Small, one-off
Schema	`COPY_OPTIONS ('mergeSchema'='true')`, `FORMAT_OPTIONS`	Inference, hints, evolution, rescue column	Manual
Use when	SQL-only users, warehouse-driven loads, few files, scheduled	Continuous or near-real-time ingestion; the default ingestion answer	Ad hoc / one-time backfill

```
COPY INTO bronze.orders
FROM '/Volumes/raw/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');      -- re-running loads only new files
```

2.4 Message buses (Kafka, Event Hubs, Kinesis, Pub/Sub)

```
raw = (spark.readStream.format("kafka")
   .option("kafka.bootstrap.servers", "b1:9093")
   .option("subscribe", "orders")
   .option("startingOffsets", "earliest")           # only for a new checkpoint
   .option("maxOffsetsPerTrigger", 100000)          # rate limiting
   .option("kafka.security.protocol", "SASL_SSL")
   .load())
# Kafka schema: key, value (binary), topic, partition, offset, timestamp, timestampType
parsed = raw.select(col("value").cast("string").alias("json"), "timestamp", "partition", "offset")
# Bronze best practice: keep the raw payload + metadata, parse in silver with from_json / from_avro
```

- **Kinesis**: `format("kinesis")` with `streamName`, `region`, `initialPosition` (latest / trim_horizon / timestamp); Databricks provides an efficient-fan-out mode. Records arrive with `partitionKey`, `data` (binary), `sequenceNumber`,

`approximateArrivalTimestamp`.

- **Google Pub/Sub**: `format("pubsub")` with `subscriptionId`, `topicId`, `projectId` and a service-account credential; exactly-once via the checkpoint. **Azure Event Hubs**: use the Kafka connector on its Kafka-compatible endpoint. All three feed streaming tables in Declarative Pipelines directly.
- Lakeflow Connect also has managed **streaming connectors** (e.g. Kafka, RabbitMQ) and **Zerobus Ingest** for writing records straight into streaming tables from applications.
- Bronze should be **append-only and raw**: store the bytes/string, topic/partition/offset, ingestion time. Parse later so bad messages don't stop ingestion.
- Exactly-once into Delta: the checkpoint stores Kafka offsets; the Delta sink is idempotent per micro-batch (transaction id in the log).

2.5 The append-only pipeline that serves batch and streaming

Design a bronze Delta table as append-only; then the same table can be consumed by a streaming table (incremental) and by batch jobs/MVs. Two ways to run the same Auto Loader code in batch or streaming mode:

- **`trigger(availableNow=True)`**: runs as a stream until all available data is processed, then stops — batch semantics with streaming bookkeeping (checkpoint, exactly-once). Preferred over the deprecated `trigger(once=True)` because it respects rate-limit options and processes in multiple micro-batches.
- **Continuous** `trigger(processingTime="1 minute")` on a continuous job for low latency.

APPEND-ONLY GUARANTEES TO REMEMBER

- Delta tables used as streaming sources must be append-only; if you need to apply corrections upstream, either `ignoreDeletes` (partition deletes), `skipChangeCommits` (skip commits with updates/deletes) or propagate via Change Data Feed.
- Schema evolution: `mergeSchema` on write adds columns; `overwriteSchema` replaces (breaks streams). In SDP, streaming tables evolve automatically with Auto Loader.
- Auto Loader + Delta = idempotent re-runs: files are never ingested twice by path (unless `allowOverwrites`).

2.6 Lakeflow Connect: managed CDC from relational databases

Lakeflow Connect provides fully managed connectors for SaaS applications (Salesforce, Workday, ServiceNow…) and **databases**. The database connectors (SQL Server, MySQL, PostgreSQL, Oracle in newer releases) replicate inserts, updates and deletes with **change data capture** and almost no custom code — the exam's preferred answer over hand-written JDBC polling.

Component	Role
Connection	UC securable holding host, port and credentials (from a secret) for the source database. Governed with UC privileges (`USE CONNECTION`, `CREATE FOREIGN CATALOG`).
Ingestion gateway	A pipeline running as a **continuous** job that extracts initial snapshots and the change log (SQL Server change tracking / CDC, MySQL binlog, PostgreSQL logical replication) into **staging** storage in a UC volume. It must keep running so changes are captured before the source truncates its logs; a long stop can create a gap that requires a full refresh.
Ingestion pipeline	A Declarative Pipeline (serverless) that reads the staged changes and applies them to destination tables incrementally on a schedule or continuously; supports SCD Type 1 / Type 2 / append-only destination modes and schema evolution.
Destination tables	Streaming tables in the chosen catalog/schema, one per source table; governed and queryable like any UC table.

- Source prerequisites: enable CDC/change tracking (SQL Server), `binlog_format=ROW` (MySQL), `wal_level=logical` + replication slot (PostgreSQL); a minimum-privilege DB user; network path (Private Link/VPN/allow-list); primary keys on tables for SCD modes.
- Create via the **Data Ingestion** UI wizard, the CLI/SDK, a notebook, or a Declarative Automation Bundle; manage like any pipeline (event log, notifications, full refresh of a table). Staging data is auto-deleted after ~25–30 days.
- **Query-based connectors** (no gateway) poll the source on a schedule when CDC isn't available — cheaper to set up, but no deletes and higher latency. **Lakehouse Federation** is for querying in place, not replication (2.10).
- Cost shows up in `system.billing.usage` as pipeline (and gateway) DBUs — the gateway is a continuously running cost.

2.7 Incremental CDC pipelines with Delta or Iceberg targets

Whether the change feed comes from Lakeflow Connect staging, a Kafka topic, CDF of an upstream table or files, the same AUTO CDC construct (Ch. 1.4) applies it incrementally. New in this exam version: the target streaming table can be exposed in the **Iceberg** format when external Iceberg engines need to read it.

```
-- Target readable by Trino/Flink/Spark via the Iceberg REST Catalog
CREATE OR REFRESH STREAMING TABLE customers_current
  TBLPROPERTIES ('delta.universalFormat.enabledFormats' = 'iceberg');   -- UniForm: Iceberg metadata on a Delta
table
-- (UC managed Iceberg tables: CREATE TABLE ... USING iceberg for tables written outside pipelines)

CREATE FLOW customers_cdc AS AUTO CDC INTO customers_current
FROM STREAM staging.customers_changes
KEYS (customer_id) APPLY AS DELETE WHEN op = 'D' SEQUENCE BY change_seq
COLUMNS * EXCEPT (op, change_seq) STORED AS SCD TYPE 1;
```

- Choose **Delta** by default (all Databricks features: DVs, liquid clustering, CDF). Choose **Iceberg** (managed Iceberg or UniForm) when non-Databricks Iceberg clients must read or write the table; Unity Catalog still handles compaction and snapshot expiration for managed Iceberg.
- UniForm generates Iceberg metadata asynchronously after each Delta commit; Iceberg readers may lag slightly and the table must satisfy Iceberg compatibility (e.g. column mapping, no unsupported types).
- Idempotency: AUTO CDC deduplicates by key and `SEQUENCE BY`, so replaying staged changes is safe; a full refresh re-snapshots from source.

2.8 OpenSharing (Delta Sharing) — the model

Provider side (your metastore)

- **Share**: a named collection of assets. `CREATE SHARE sales_share; ALTER SHARE sales_share ADD TABLE prod.gold.orders [PARTITION (region = 'EU')] [WITH HISTORY];`
- Assets: tables, partitions of tables, views, dynamic views, materialized views/streaming tables, **volumes**, notebooks, UC models, and entire schemas.
- **Recipient**: who receives. `GRANT SELECT ON SHARE sales_share TO RECIPIENT partner_x;`
- Data never leaves your storage; recipients read via short-lived pre-signed URLs issued by the Delta Sharing server. Audit events land in `system.access.audit`.

Recipient side

- **D2D** (recipient has Unity Catalog): `CREATE RECIPIENT partner_x USING ID 'aws:us-east-1:<metastore-uuid>';` — authenticated by the recipient's metastore ID. Recipient runs `CREATE CATALOG partner_sales USING SHARE provider_org.sales_share;` and queries live data with UC governance.
- **D2O / open protocol** (no Databricks): `CREATE RECIPIENT partner_y;` generates an **activation link** and a bearer **token** (with configurable expiry). Recipient downloads a `config.share` credential file and reads with pandas (`delta_sharing.load_as_pandas`), Spark (`format("deltaSharing")`), Power BI, Tableau, etc.

	Databricks-to-Databricks (D2D)	Open sharing (D2O)
Auth	Metastore identity; no tokens to manage	Bearer token; rotate with `ALTER RECIPIENT ... ROTATE TOKEN`; set token lifetime; IP access lists
Extra features	Volumes, notebooks, models, views, streaming from shared tables, Change Data Feed, incremental sharing with history	Tables (and views with recent versions); CDF/streaming only when shared `WITH HISTORY`
Cost	Egress if cross-region/cloud; compute paid by recipient	Egress paid by provider's cloud account
Governance	Recipient applies its own UC grants on the shared catalog; provider controls the share contents	Provider controls via share & token; recipient governance is external

EXAM POINTS

- Delta Sharing is **read-only** and **live** — no replication, no ETL; recipients see the current version (and history if shared).
- To stream from or read CDF of a shared table, the provider must add it `WITH HISTORY`; otherwise only the latest snapshot is shared.
- Share a **view** or **partition** to limit rows/columns; dynamic views can filter per recipient using `current_recipient('property')`.
- Use Delta Sharing when the consumer is a different metastore, organisation or platform. Inside one metastore, just **GRANT** — sharing is unnecessary overhead.
- Clean Rooms build on Delta Sharing for privacy-safe collaboration; Databricks Marketplace is Delta Sharing-based distribution.

2.9 Clean Rooms — privacy-preserving collaboration

A **Clean Room** is a secure, Databricks-hosted environment built on OpenSharing where two or more parties bring data and approved code (notebooks) so that analyses run on the *combined* data without any party seeing the other's raw rows.

- Each collaborator shares tables/views (and optionally notebooks) into the clean room; only **approved notebooks** can run, and only their **outputs** (aggregates, model results) leave the room — raw data never does.
- Compute is isolated inside the clean room (Databricks-managed), collaborators can be on different clouds/regions, and the central clean-room catalog is read-only for participants.
- Use for partner overlap analysis, joint attribution, research consortia in healthcare — anywhere sharing raw PII/PHI is prohibited but joint computation is allowed. Not for simply granting a partner read access (that's OpenSharing) and not a substitute for de-identification of outputs.
- Audit: creation, notebook approvals and runs are logged in `system.access.audit`.

2.10 Lakehouse Federation

Federation lets Unity Catalog run queries against external systems **without ingesting** the data. You create a **connection** (credentials, host) and a **foreign catalog** that mirrors the external database's schemas and tables as read-only UC objects.

```
CREATE CONNECTION pg_crm TYPE postgresql
OPTIONS (host 'crm.db.internal', port '5432', user secret('crm-scope','user'), password secret('crm-
scope','pwd'));

CREATE FOREIGN CATALOG crm USING CONNECTION pg_crm OPTIONS (database 'crm_prod');

GRANT USE CATALOG, USE SCHEMA, SELECT ON CATALOG crm TO `analysts`;    -- UC governance applies
SELECT c.region, COUNT(*) FROM crm.public.customers c JOIN prod.gold.orders o USING (customer_id) GROUP BY 1;
```

- Supported sources include PostgreSQL, MySQL, SQL Server, Azure Synapse, Amazon Redshift, Snowflake, Google BigQuery, Oracle, Teradata, Salesforce Data Cloud, another Databricks workspace and Hive metastore/Glue.
- **Query pushdown**: filters, projections, limits and many aggregations are pushed to the source, so filter early. Pushdown capabilities vary by connector.
- Governance: privileges, lineage, tags, row filters/column masks apply to foreign tables like any UC table. Credentials live on the **connection** (from secrets or OAuth), never in notebooks; `USE CONNECTION` / `CREATE FOREIGN CATALOG` control who can build catalogs on it; analysts only ever receive UC grants on the foreign catalog.
- Best for exploration, joins with small/medium external reference data, migrations, and building a materialized view on top of a foreign table to **cache** it. Not for high-volume repeated ETL — that should ingest (Lakeflow Connect / JDBC into Delta).

Domain 2 — Practice Questions

20 questions

Q1 **A vendor drops ~200,000 small JSON files per hour into a cloud storage folder with lexically increasing names. Directory listing has become the bottleneck of the Auto Loader stream. Which change most directly fixes discovery latency without changing the file naming?**

- **A.** Switch to `COPY INTO` on a schedule.
- **B.** Enable file notification mode (`cloudFiles.useNotifications`) or managed file events on the external location so new files are discovered from events rather than listing.
- **C.** Increase `cloudFiles.maxFilesPerTrigger` .
- **D.** Set `cloudFiles.includeExistingFiles` to false.

Q2 **An Auto Loader stream using the default schema evolution mode stops with an `UnknownFieldException` after the producer added a new column. What is the expected behaviour and correct operator action?**

- **A.** The stream is broken permanently; delete the checkpoint and restart.
- **B.** This is expected in `addNewColumns` mode: the schema location has been updated with the new column; simply restart the stream (or rely on job retries) and it continues.
- **C.** Set `cloudFiles.schemaEvolutionMode` to `none` so the column is added silently.
- **D.** Convert the source files to Parquet.

Q3 **A team wants to ingest new CSV files every night using SQL only from a SQL warehouse, re-running safely without duplicating rows. Only a few hundred files land per day. What is the most appropriate approach?**

- **A.** `COPY INTO` with `FILEFORMAT = CSV` , which tracks already-loaded files and is idempotent.
- **B.** `INSERT INTO ... SELECT * FROM csv.`/path`` each night.
- **C.** `CREATE TABLE ... AS SELECT` every night.
- **D.** A Structured Streaming job with `foreachBatch` .

Q4 **Messages on a Kafka topic are Avro-encoded with a Confluent Schema Registry. The bronze table must retain the raw payload so that parsing errors can be replayed. Which design is correct?**

- **A.** Decode with `from_avro` in bronze and drop the binary `value` to save space.
- **B.** Store `key` , `value` (binary), `topic` , `partition` , `offset` , `timestamp` in an append-only bronze streaming table; decode with `from_avro` in the silver step.
- **C.** Convert Kafka messages to CSV files first and use Auto Loader.
- **D.** Use `COPY INTO` with `FILEFORMAT = AVRO` pointing at the broker.

Q5 **Which read configuration is required to ingest XML files where each `<patient>` element is one row?**

- **A.** `spark.read.format("xml").option("rowTag", "patient").load(path)`
- **B.** `spark.read.format("text").option("wholetext", "true").load(path)`
- **C.** `spark.read.format("json").option("multiLine", "true").load(path)`
- **D.** `spark.read.format("xml").load(path)` — rowTag is inferred.

Q6 **A pipeline runs hourly on a job cluster and must process all files that arrived since the last run, exactly once, then stop so the cluster is released. Which trigger should the Auto Loader write use?**

- **A.** `trigger(processingTime="1 hour")`
- **B.** `trigger(continuous="1 second")`
- **C.** `trigger(availableNow=True)`
- **D.** No trigger; the default runs micro-batches as fast as possible.

Q7 **A CSV feed occasionally contains malformed lines. The team wants the good rows loaded and the bad rows kept for inspection, without failing the batch read. Which option set is correct?**

- **A.** `mode="FAILFAST"`
- **B.** `mode="PERMISSIVE"` with `columnNameOfCorruptRecord` (or `badRecordsPath`) so bad lines are captured instead of dropped.
- **C.** `mode="DROPMALFORMED"`
- **D.** `inferSchema="false"`

Q8 **An existing Parquet directory of 50 TB must become a Delta table with minimal data movement. What is the most efficient command?**

- **A.** `CREATE TABLE t USING DELTA AS SELECT * FROM parquet.`/path``
- **B.** `CONVERT TO DELTA parquet.`/path`` (optionally with `PARTITIONED BY`), which writes a transaction log over the existing files.
- **C.** `COPY INTO t FROM '/path' FILEFORMAT = PARQUET`
- **D.** Read with Auto Loader in `availableNow` mode.

Q9 **A streaming table reads from a bronze Delta table. An operator ran `DELETE FROM bronze WHERE ingest_date < '2024-01-01'` to enforce retention, and the downstream stream now fails. Which option lets the stream continue while ignoring these retention deletes?**

- **A.** `option("skipChangeCommits", "true")` or `ignoreDeletes` on the streaming read (or a full refresh of the streaming table).
- **B.** `option("mergeSchema", "true")`
- **C.** `option("startingOffsets", "latest")`
- **D.** Delete the checkpoint directory.

Q10 **A hospital network must give a research partner (who does not use Databricks) read access to a de-identified gold table, refreshed continuously, without copying files. Which approach is correct?**

- **A.** Export the table nightly to CSV in a shared bucket.
- **B.** Create a Delta Share containing the table, create an **open sharing** recipient, and send the activation link so they read with the Delta Sharing client.
- **C.** Create a D2D recipient using the partner's email address.
- **D.** Grant SELECT on the table to an external account group.

Q11 **A recipient in another Databricks account wants to run a streaming table on top of a shared Delta table. The stream fails immediately. What must the provider change?**

- **A.** Share the table `WITH HISTORY` so table versions are available for streaming/CDF.
- **B.** Convert the recipient to open sharing.
- **C.** Grant MODIFY on the share.
- **D.** Enable deletion vectors on the shared table.

Q12 **Two business units in the same Unity Catalog metastore need to read each other's gold tables. What is the recommended mechanism?**

- **A.** Delta Sharing D2D between the workspaces.
- **B.** Standard Unity Catalog `GRANT`s on the catalogs/schemas/tables — sharing is unnecessary within one metastore.
- **C.** DEEP CLONE the tables into each other's catalogs nightly.
- **D.** Lakehouse Federation between the two workspaces.

Q13 **A provider wants each partner to see only rows for its own country in one shared view, without creating a view per partner. Which feature enables this?**

- **A.** Row filters on the recipient side.
- **B.** A dynamic view using `current_recipient('country')` with a recipient property set on each recipient.
- **C.** Separate metastores per partner.
- **D.** Sharing partitions with `WITH HISTORY`.

Q14 **Analysts need to join a 2 GB customer table in an on-prem PostgreSQL database with lakehouse orders a few times a day for exploration, with UC permissions enforced. What is the lowest-effort approach?**

- **A.** Build an SDP pipeline with JDBC ingestion into bronze.
- **B.** Create a UC `CONNECTION` of type postgresql and a `FOREIGN CATALOG`; grant SELECT to the analysts.
- **C.** Give analysts the PostgreSQL password and let them use `spark.read.jdbc`.
- **D.** Export the table as Parquet weekly.

Q15 **A federated query against Snowflake returns all 500 million rows before filtering on the Databricks side. What is the most likely cause?**

- **A.** Federation never pushes down predicates.
- **B.** The filter uses a function or expression that the connector cannot push down; rewrite it to a supported predicate (or materialise the table).
- **C.** The foreign catalog was created without `WITH HISTORY`.
- **D.** Photon is disabled.

Q16 **Security asks that an open-sharing recipient's credentials be invalidated immediately because a token leaked. Which action is correct?**

- **A.** `ALTER RECIPIENT partner ROTATE TOKEN` with expiry of the old token set to now (or drop the recipient).
- **B.** Drop the share.
- **C.** Revoke SELECT on the underlying table from the provider's users.
- **D.** Change the cloud storage key.

Q17 **A hospital must continuously replicate inserts, updates and deletes from an operational SQL Server database into the lakehouse with minimal custom code and no impact on the source beyond enabling change tracking. Which approach fits?**

- **A.** A nightly JDBC full-table read that overwrites the target.
- **B.** A Lakeflow Connect managed SQL Server connector: a UC connection, a continuously running ingestion gateway, and an ingestion pipeline applying CDC to streaming tables.
- **C.** A Lakehouse Federation foreign catalog over SQL Server queried by downstream jobs.
- **D.** Export CSV extracts hourly and load with Auto Loader.

Q18 **A Lakeflow Connect PostgreSQL gateway was stopped for two weeks to save cost; the source's replication slot was dropped and WAL was recycled. When the gateway restarts, what is the correct expectation and remedy?**

- **A.** The pipeline resumes exactly where it stopped because staging retains all changes.
- **B.** There is a change gap that cannot be recovered incrementally; trigger a full refresh of the affected destination tables (and keep the gateway running continuously going forward).
- **C.** Only deletes are lost; inserts and updates are recovered from the binlog.
- **D.** Convert the destination tables to Iceberg to recover the missing changes.

Q19 **A gold table must be written by Databricks pipelines and also updated by an external Flink job, with Unity Catalog governing it and performing compaction. Which table type satisfies this?**

- **A.** A Unity Catalog managed Iceberg table (`USING iceberg`) accessed by Flink through the Iceberg REST Catalog
- **B.** An external Parquet directory
- **C.** A Delta table with deletion vectors
- **D.** A Delta Sharing recipient for Flink

Q20 **Two health systems want to measure patient overlap and joint outcomes, but neither may expose patient-level records to the other. Which Databricks capability is designed for this?**

- **A.** OpenSharing (D2D) of both patient tables
- **B.** A Clean Room where both parties share tables and only approved notebooks run, releasing aggregate outputs
- **C.** Lakehouse Federation between the two workspaces
- **D.** A shallow clone exchanged via cloud storage

Domain 2 — Answer Key

Q	Ans	Explanation
1	B	Notification/file-event modes replace directory listing with event-driven discovery. COPY INTO also lists; maxFilesPerTrigger controls batch size, not discovery; includeExistingFiles only affects the first run.
2	B	In addNewColumns mode the stream fails once, records the new schema in the schema location, and succeeds on restart. This is why Auto Loader jobs are commonly configured with retries.
3	A	COPY INTO is the idempotent SQL batch loader for thousands of files. B/C duplicate or replace data; D is overkill for a SQL-only team.
4	B	Bronze keeps raw bytes and Kafka metadata so silver parsing can be re-run. A loses replayability; C and D are not real patterns.
5	A	The native XML reader requires `rowTag`. Text mode gives one string per file; JSON can't parse XML.
6	C	`availableNow` processes all available data in (possibly several) micro-batches with checkpointing and then terminates. processingTime keeps the stream (and cluster) alive; continuous is experimental.
7	B	PERMISSIVE retains malformed rows in the corrupt-record column (or writes them to badRecordsPath). DROPMALFORMED discards them; FAILFAST aborts.
8	B	CONVERT TO DELTA creates the log in place without rewriting 50 TB. CTAS and COPY INTO copy the data.
9	A	Delta streaming sources require append-only; `skipChangeCommits` (or `ignoreDeletes` for partition-aligned deletes) lets the stream skip those commits. Deleting the checkpoint would reprocess everything.
10	B	Open protocol serves non-Databricks consumers with live, no-copy access. D2D needs a metastore; UC grants don't cross organisations.
11	A	Streaming and CDF on shared tables require history; by default only the current snapshot is shared.
12	B	Within one metastore, UC grants are the mechanism; sharing adds no value. Clones copy data; federation is for external systems.
13	B	Dynamic views can reference recipient properties via `current_recipient()` to filter per recipient.
14	B	Federation exposes the table as a governed foreign catalog with pushdown — no ingestion, no credential sprawl.
15	B	Pushdown depends on the connector's supported predicates; unsupported expressions force a full pull. Consider an MV to cache.
16	A	Token rotation with immediate expiry invalidates leaked credentials. Dropping the share breaks all recipients; C/D don't affect sharing.
17	B	Managed database connectors deliver CDC (incl. deletes) with minimal code; federation queries in place rather than replicating; JDBC overwrite and CSV extracts are full reloads without deletes.
18	B	The gateway must run continuously so changes are captured before the source truncates its log; a gap is unrecoverable incrementally and needs a full refresh. Staging holds only what the gateway captured.
19	A	Managed Iceberg tables let external Iceberg engines read and write through the REST catalog while UC owns governance and maintenance. Parquet has no governance/ACID; Delta+DVs isn't Iceberg-writable; sharing is read-only.
20	B	Clean Rooms run approved code on combined data without exposing raw rows; sharing/federation/clones all expose records.

DOMAIN 3

Data Manipulation

12% OF EXAM

Renamed "Data Manipulation" and raised to 12%. Besides efficient window/join/aggregation code and the quarantine pattern, two new topics appear: the **VARIANT** type for semi-structured data and **AI functions** (`ai_query` and friends) for model inference inside pipelines. Expect ~7 questions.

OFFICIAL OBJECTIVES (NEW EXAM)

- Apply advanced transformations — window functions, joins, aggregations — with Spark SQL and PySpark on large datasets.
- Develop a model and query semi-structured data using the VARIANT data type and related functions (e.g. `parse_json`, `variant_get`).
- Apply AI functions, including `ai_query`, to perform model inference within pipelines for enrichment and classification.
- Implement data quality expectations in Lakeflow Declarative Pipelines to quarantine, drop, or fail on bad records.

3.1 Window functions

```
SELECT customer_id, order_ts, amount,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_ts DESC)         AS rn,      -- dedup / latest
  SUM(amount)  OVER (PARTITION BY customer_id ORDER BY order_ts
                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)        AS running_total,
  LAG(amount)  OVER (PARTITION BY customer_id ORDER BY order_ts)              AS prev_amount,
  AVG(amount)  OVER (PARTITION BY customer_id ORDER BY order_ts
                     RANGE BETWEEN INTERVAL 7 DAYS PRECEDING AND CURRENT ROW) AS avg_7d
FROM orders;
-- Latest record per key (classic dedup):
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) rn FROM src) WHERE rn = 1;
```

- `ROW_NUMBER` (unique), `RANK` (gaps), `DENSE_RANK` (no gaps), `NTILE`, `LAG/LEAD`, `FIRST_VALUE/LAST_VALUE`.
- `ROWS BETWEEN` counts rows; `RANGE BETWEEN` uses values of the ORDER BY column (ties included; supports interval literals on timestamps).
- A window without `PARTITION BY` pulls all rows to one partition — a classic performance trap. Partition by a key whenever possible.
- PySpark: `from pyspark.sql.window import Window; w = Window.partitionBy("customer_id").orderBy(F.col("order_ts").desc())`.

3.2 Joins — what Spark actually does

Strategy	When chosen	Cost
Broadcast hash join	One side < `spark.sql.autoBroadcastJoinThreshold` (default 10 MB) or hinted `/*+ BROADCAST(dim) */`; AQE can switch to it at runtime	No shuffle of the big side; small side copied to every executor (must fit in driver & executor memory)
Sort-merge join	Default for two large tables with equi-join keys	Shuffles both sides by key, sorts, merges — network heavy; skew on the key hurts
Shuffle hash join	When one side is small-ish relative to the other but too big to broadcast (`SHUFFLE_HASH` hint)	Shuffle + build hash per partition
Broadcast nested loop	Non-equi joins (`ON a.x > b.y`), cross joins	Very expensive; avoid or pre-bucket ranges

- **Adaptive Query Execution (AQE)**, on by default: coalesces shuffle partitions, converts SMJ→broadcast when a side turns out small, and splits **skewed** partitions (`spark.sql.adaptive.skewJoin.enabled`).
- Skew remedies: AQE skew join, `/*+ SKEW('t', 'key') */` hint, salting the hot key, filtering nulls before the join, broadcasting the small side.
- **Dynamic file pruning** lets a join filter push down into a Delta scan of the fact table (Ch. 5).
- Range/point-in-time joins: use `RANGE_JOIN` hint with a bin size.

3.3 Aggregations, dedup and higher-order functions

- `groupBy().agg()` is a wide transformation: a shuffle by the group key. Partial aggregation happens map-side automatically. Prefer `approx_count_distinct` over `count(distinct)` on huge data.
- Streaming dedup: `dropDuplicates(["id"])` keeps state forever; add a watermark and use `dropDuplicatesWithinWatermark(["id"])` to bound state.
- Complex types: `transform(arr, x -> x*2)`, `filter`, `aggregate`, `explode`, `posexplode`, `map_keys`; `:` operator / `variant_get` for JSON/VARIANT paths; `from_json` with a schema.
- `MERGE INTO` for upserts (idempotent batch loads): `WHEN MATCHED [AND cond] THEN UPDATE/DELETE`, `WHEN NOT MATCHED THEN INSERT`, `WHEN NOT MATCHED BY SOURCE THEN DELETE`. Source must have unique keys or MERGE fails with multiple matches.
- Pivot/unpivot: `PIVOT`, `UNPIVOT`, `stack()`. Set operations: `UNION` (dedups) vs `UNION ALL`, `EXCEPT`, `INTERSECT`.

3.4 Quarantining bad data

Pattern A — SDP expectations with an inverted rule

Expectations drop or fail; they don't route rows. The quarantine pattern defines the **same rules** twice: the clean table drops violations, the quarantine table drops rows that *pass*. Alternatively, flag rows with a boolean and split with two views.

```
CREATE OR REFRESH STREAMING TABLE silver_valid (
  CONSTRAINT rules EXPECT (order_id IS NOT NULL AND amount > 0) ON VIOLATION DROP ROW)
AS SELECT * FROM STREAM bronze_orders;

CREATE OR REFRESH STREAMING TABLE silver_quarantine (
  CONSTRAINT inverse EXPECT (NOT (order_id IS NOT NULL AND amount > 0)) ON VIOLATION DROP ROW)
AS SELECT *, current_timestamp() AS quarantined_at, 'order_rules' AS failed_rule FROM STREAM bronze_orders;
```

```
# Python variant with a single scan: flag then split
rules = {"valid_id": "order_id IS NOT NULL", "valid_amount": "amount > 0"}
quarantine_expr = " OR ".join(f"NOT({r})" for r in rules.values())

@dp.view
def flagged():
    return spark.readStream.table("bronze_orders").withColumn("is_quarantined", expr(quarantine_expr))

@dp.table
def silver_valid():      return spark.readStream.table("flagged").filter("NOT
is_quarantined").drop("is_quarantined")
@dp.table
def silver_quarantine(): return spark.readStream.table("flagged").filter("is_quarantined")
```

Pattern B — Auto Loader in classic jobs

- **`_rescued_data`**: rows where it is not null had schema/type problems. Split the micro-batch in `foreachBatch`: `bad = df.filter("_rescued_data IS NOT NULL")` → quarantine table; `good` → target.
- **`badRecordsPath`** (CSV/JSON): writes unparseable records to files for later inspection.
- PERMISSIVE mode with `_corrupt_record` column keeps raw text of bad lines.
- Quarantine tables should carry **why** (rule name), **when**, and the raw payload so records can be corrected and replayed.

> **CHOOSING THE RIGHT SEVERITY**
> Use **FAIL UPDATE** only for invariants where continuing would corrupt downstream data (e.g. primary key null). Use **DROP + quarantine** for recoverable row-level issues. Use **warn** for monitoring-only metrics. The exam expects you to avoid stopping a whole pipeline for a handful of bad rows.

3.5 Semi-structured data with VARIANT

VARIANT is a native Spark/Delta type (DBR 15.3+) that stores JSON-like data in an efficient binary encoding with a shredded index, so you can keep payloads of varying shape in one column and extract fields fast — without inferring a rigid schema up front and without paying string-parsing cost at every query.

```
-- Ingest raw JSON strings into VARIANT (bronze) and query paths (silver)
CREATE OR REFRESH STREAMING TABLE bronze_events AS
SELECT parse_json(value) AS payload, _metadata.file_path AS src
FROM STREAM read_files('/Volumes/raw/events/', format => 'text');

SELECT
  payload:patient.id::STRING              AS patient_id,     -- colon path + cast
  payload:vitals.hr::INT                  AS heart_rate,
  variant_get(payload, '$.encounter.ts', 'TIMESTAMP') AS encounter_ts,
  try_variant_get(payload, '$.notes[0].text', 'STRING') AS first_note,   -- NULL instead of error on mismatch
  is_variant_null(payload:discharge)      AS no_discharge,
  schema_of_variant(payload)              AS inferred_schema
FROM bronze_events;

-- Explode arrays inside a VARIANT, or shred to columns once shape stabilises
SELECT e.patient_id, v.value:code::STRING AS dx_code
FROM silver_events e, LATERAL variant_explode(e.payload:diagnoses) v;
```

Function / feature	Use
`parse_json(str)` / `try_parse_json`	String → VARIANT (try_ returns NULL on invalid JSON instead of failing the batch).
`col:path`, `variant_get(v, '$.path', 'TYPE')`, `try_variant_get`	Field extraction with type cast; JSONPath-style with arrays (`$.a[0].b`).
`variant_explode`, `variant_explode_outer`	Turn arrays/objects into rows.
`schema_of_variant`, `schema_of_variant_agg`	Discover the shape of the data; input for deciding when to shred into typed columns.
`to_json(variant)`, `variant → STRING cast`	Serialise back for export.
Auto Loader / `read_files` with `singleVariantColumn`	Ingest an entire JSON document into one VARIANT column.

VARIANT VS STRING VS STRUCT — THE EXAM'S DECISION

- **STRING + regexp/from_json at query time**: cheapest to write, slowest to read, no type safety. The "before" state in most questions.
- **STRUCT with a fixed schema**: fastest and enables stats/data skipping on the fields, but every schema change breaks the pipeline. Use for stable, well-known shapes (silver/gold).
- **VARIANT**: flexible *and* fast — the right answer when payload shape varies or evolves and you want efficient storage plus quick field access without pre-declaring a schema. Limitations to remember: not a clustering/partition key, no min/max stats for skipping on inner fields (shred hot fields to columns), limited support in some external readers.

3.6 AI functions in pipelines (`ai_query` and task-specific functions)

AI functions are built-in SQL functions that call model-serving endpoints — foundation models hosted by Databricks or your own endpoints — so enrichment and classification become ordinary SQL transformations inside Declarative Pipelines, MERGE statements or ad hoc queries.

```
-- General-purpose: any served model, custom prompt, optional structured output
SELECT claim_id,
  ai_query('databricks-meta-llama-3-3-70b-instruct',
           CONCAT('Classify this denial reason into one of: CODING, ELIGIBILITY, AUTH, OTHER. Reason: ',
denial_text),
           returnType => 'STRING') AS denial_category
FROM silver.claims WHERE status = 'DENIED';

-- Batch inference against a custom endpoint with a schema for the response
SELECT ai_query('risk-scoring-endpoint', named_struct('age', age, 'dx', dx_codes),
                returnType => 'STRUCT<score: DOUBLE, tier: STRING>') AS risk FROM gold.members;

-- Task-specific functions (no prompt engineering needed)
SELECT ai_classify(note_text, ARRAY('urgent','routine')) AS triage,
       ai_extract(note_text, ARRAY('medication','dosage')) AS meds,
       ai_mask(note_text, ARRAY('person','address'))       AS deidentified_note,
       ai_analyze_sentiment(feedback)                     AS sentiment,
       ai_summarize(discharge_summary, 50)                AS short_summary,
       ai_translate(instructions, 'es')                   AS instructions_es
FROM silver.encounters;
```

- `ai_query(endpoint, request, [returnType], [failOnError], [modelParameters])` is the general function; `failOnError => false` returns an error struct per row instead of failing the batch — the pattern for production pipelines.
- Task functions: `ai_classify`, `ai_extract`, `ai_mask`, `ai_analyze_sentiment`, `ai_summarize`, `ai_translate`, `ai_fix_grammar`, `ai_gen`, `ai_similarity`, `ai_parse_document`, `vector_search`. They use Databricks-hosted foundation models; governance and cost follow the endpoint (pay-per-token or provisioned throughput).
- In a Declarative Pipeline, wrap AI calls in a streaming table so each row is scored **once** (incremental) rather than in an MV that would re-score everything on refresh; batch inference on very large tables should use provisioned-throughput endpoints or the batch inference API for cost and rate limits.
- Combine with expectations: `CONSTRAINT valid_class EXPECT (denial_category IN ('CODING','ELIGIBILITY','AUTH','OTHER')) ON VIOLATION DROP ROW` catches malformed model output; `ai_mask` is a legitimate PII-redaction step (Ch. 6).
- Requires a workspace/region where model serving and AI functions are available; Unity Catalog governs the endpoint (`CAN QUERY`).

Domain 3 — Practice Questions

14 questions

Q1 **A query computes a running total with `SUM(amount) OVER (ORDER BY order_ts)` over 3 billion rows and runs on a single task for hours. What is the root cause?**

- **A.** The window lacks `PARTITION BY`, so all rows are moved into one partition.
- **B.** `SUM` is not supported in window functions.
- **C.** The cluster has Photon disabled.
- **D.** `ROWS BETWEEN` was not specified.

Q2 **Which expression returns exactly one row per `patient_id` — the most recent by `updated_at` — even when two rows share the same timestamp?**

- **A.** `RANK() OVER (PARTITION BY patient_id ORDER BY updated_at DESC) = 1`
- **B.** `DENSE_RANK() OVER (PARTITION BY patient_id ORDER BY updated_at DESC) = 1`
- **C.** `ROW_NUMBER() OVER (PARTITION BY patient_id ORDER BY updated_at DESC, ingest_id DESC) = 1`
- **D.** `MAX(updated_at) OVER (PARTITION BY patient_id)`

Q3 **A join between a 5 TB fact table and a 4 MB reference table uses a sort-merge join and shuffles the fact table. What is the most likely reason and fix?**

- **A.** The reference table statistics are missing or the broadcast threshold was lowered; add a `BROADCAST` hint or ensure AQE can detect the small size.
- **B.** Sort-merge is always chosen for Delta tables; nothing can be done.
- **C.** Increase `spark.sql.shuffle.partitions`.
- **D.** Convert the reference table to CSV.

Q4 **In the Spark UI, one task in a join stage processes 40 GB while the median task processes 200 MB. Which two mitigations address this? (choose the best combined answer)**

- **A.** Disable AQE and increase executor memory.
- **B.** Ensure AQE skew join handling is enabled and, if the hot key is legitimate, salt the key or broadcast the smaller side.
- **C.** Use `coalesce(1)` before the join.
- **D.** Cache both tables.

Q5 **A streaming deduplication uses `dropDuplicates(["event_id"])` and the job's state store grows without bound. Which change bounds state while still removing duplicates that arrive within an hour?**

- **A.** Add `.withWatermark("event_ts", "1 hour")` and use `dropDuplicatesWithinWatermark(["event_id"])`.
- **B.** Increase the driver memory.
- **C.** Replace it with `distinct()`.
- **D.** Use output mode `complete`.

Q6 **An SDP silver table drops rows failing `amount > 0`. Auditors require that dropped rows be retained with the reason. Which is the simplest compliant design?**

- **A.** Change the expectation to `FAIL UPDATE` so nothing is lost.
- **B.** Add a second streaming table from the same source whose expectation is the inverse rule with `ON VIOLATION DROP ROW`, adding a `failed_rule` column.
- **C.** Query the event log later to reconstruct the dropped rows.
- **D.** Remove the expectation and clean data in gold.

Q7 **Which statement about the `_rescued_data` column produced by Auto Loader is true?**

- **A.** It contains rows that failed an SDP expectation.
- **B.** It holds, as JSON, data that could not be parsed into the inferred/provided schema (type mismatches, unexpected columns), so it is a natural quarantine filter.
- **C.** It is only available with CSV.
- **D.** It is written to `badRecordsPath` automatically.

Q8 **A `MERGE INTO dim_customer USING updates ON dim.id = updates.id WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *` fails with an error that multiple source rows matched a target row. What is the correct fix?**

- **A.** Add `WHEN NOT MATCHED BY SOURCE THEN DELETE`.
- **B.** Deduplicate `updates` to one row per `id` (e.g. latest by timestamp with `ROW_NUMBER`) before the MERGE.
- **C.** Change the join to `ON dim.id <> updates.id`.
- **D.** Enable deletion vectors on the target.

Q9 **A 7-day moving average per device is needed where readings arrive at irregular intervals. Which window frame is correct?**

- **A.** `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`
- **B.** `RANGE BETWEEN INTERVAL 7 DAYS PRECEDING AND CURRENT ROW` ordered by the timestamp
- **C.** `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
- **D.** No frame; the default is 7 days

Q10 **A non-equi join `ON e.ts BETWEEN s.start_ts AND s.end_ts` between two large tables takes hours. Which technique is most effective?**

- **A.** Increase broadcast threshold to 10 GB.
- **B.** Use the `RANGE_JOIN` hint with an appropriate bin size (or bucket both sides by a coarse time bucket and add an equi-join on the bucket).
- **C.** Convert to a Python UDF.
- **D.** Use `UNION ALL` instead of a join.

Q11 **A bronze table stores device telemetry as a JSON string whose keys change from firmware to firmware. Analysts complain that `from_json` with a fixed schema drops new fields and that `regexp_extract` queries are slow. Which redesign is most appropriate?**

- **A.** Store the payload in a VARIANT column with `parse_json` and extract fields with colon-path syntax / `variant_get`; shred stable hot fields into typed columns in silver.
- **B.** Re-infer a STRUCT schema on every batch and overwrite the table schema.
- **C.** Keep the string and add a Z-order on it.
- **D.** Convert the payloads to XML.

Q12 **Which expression returns NULL rather than raising an error when `payload:vitals.hr` contains a non-numeric value?**

- **A.** `variant_get(payload, '$.vitals.hr', 'INT')`
- **B.** `try_variant_get(payload, '$.vitals.hr', 'INT')`
- **C.** `payload:vitals.hr::INT`
- **D.** `schema_of_variant(payload)`

Q13 **A silver pipeline must tag each new customer-support ticket with a category from a fixed list using a Databricks-hosted foundation model, processing each ticket exactly once and never failing the whole update because of one bad model response. Which design is best?**

- **A.** A materialized view calling `ai_query` so results are always current.
- **B.** A streaming table that applies `ai_classify` (or `ai_query` with `failOnError => false`) to new rows only, with an expectation validating the category.
- **C.** A Python UDF that calls the endpoint with `requests` from every executor.
- **D.** A nightly notebook that exports tickets to CSV for manual labelling.

Q14 **Clinical notes must be stored in silver with names and addresses redacted before analysts can query them. Which built-in function performs this within a SQL transformation?**

- **A.** `ai_mask(note_text, ARRAY('person','address'))`
- **B.** `ai_summarize(note_text, 10)`
- **C.** `sha2(note_text, 256)`
- **D.** `ai_translate(note_text, 'en')`

Domain 3 — Answer Key

Q	Ans	Explanation
1	A	An unpartitioned window forces a single partition (the classic `WindowExec` warning). Partition by a key or compute cumulative sums differently.
2	C	RANK/DENSE_RANK return several rows on ties; ROW_NUMBER with a deterministic tie-breaker guarantees one row. D returns a value, not a filter.
3	A	A 4 MB side should be broadcast; if stats are stale or the threshold was reduced, hint it. Shuffle partitions don't change the strategy.
4	B	Skew is handled by AQE splitting large partitions; salting or broadcasting removes the shuffle hot spot. Disabling AQE makes it worse; coalesce(1) is catastrophic.
5	A	Watermark-bounded dedup evicts old keys from state. distinct() is the same unbounded operation; complete mode is for aggregations.
6	B	The inverse-expectation quarantine pattern. FAIL UPDATE stops the pipeline; the event log stores counts, not rows.
7	B	The rescued-data column captures schema-mismatched data for JSON, CSV, Avro, Parquet etc. Expectations are an SDP concept; badRecordsPath is a separate option.
8	B	MERGE requires at most one source row per target row. Deduplicate the source first. A/C/D don't address the multiple-match error.
9	B	Irregular intervals need a value-based (RANGE) frame with a time interval; ROWS counts records regardless of time.
10	B	Range joins are the documented optimisation for interval/point-in-interval joins; broadcasting a large table is impossible.
11	A	VARIANT keeps flexibility with efficient binary storage and fast path access; shredding hot fields restores stats-based skipping. Schema overwrite breaks readers; Z-order on a string blob is useless.
12	B	The `try_` variant returns NULL on cast/path errors; the others raise or return schema text.
13	B	Streaming tables score only new rows (exactly once, cost-bounded); `failOnError => false` and expectations isolate bad responses. An MV would re-score everything; a UDF with raw HTTP bypasses governance and rate limiting.
14	A	`ai_mask` redacts named entity types in text; hashing destroys the note and summarising/translating don't remove PII.

DOMAIN 4

Monitoring and Alerting

10% OF EXAM

Observability has three layers on Databricks: **system tables** (account-wide history), **UI tools** (Query Profile, Spark UI, pipeline event log) and **programmatic access** (REST API, CLI and now the **Databricks SDK**). The new outline groups alerting under **Lakehouse alerts** — SQL alerts applied to business metrics, data-quality anomalies, cost, warehouse health, audit events, AI-agent quality and **branching inside Lakeflow Jobs**.

OFFICIAL OBJECTIVES (NEW EXAM)

- Use system tables (billing, compute, access, lakeflow) for cost analysis, auditing and workload monitoring.
- Use REST APIs / CLI and SDK for monitoring and analysing jobs and pipelines.
- Use Declarative Pipelines event logs to monitor pipeline health and data quality.
- Use Lakehouse alerts to monitor governed business metrics, data quality and anomaly signals, usage and cost, SQL warehouse/query health, audit and security events, AI agent quality, and Lakeflow Job branching.
- Use Lakeflow Jobs UI and Jobs API to monitor job status and performance metrics.

4.1 System tables (`system.*`)

Delta tables in the `system` catalog, populated by Databricks for the whole account, governed by UC (metastore admin grants `USE SCHEMA` / `SELECT`). Most schemas keep 365 days (some less, e.g. query history 90 days). Some schemas must be enabled first.

Table	Use it for
`system.billing.usage`	DBU consumption per workspace, SKU, cluster/warehouse/job/pipeline id, custom tags, hourly. Join with `system.billing.list_prices` for $ cost.
`system.access.audit`	Audit log: who did what (logins, permission changes, table access, Delta Sharing events, job runs) — `service_name`, `action_name`, `user_identity`, `request_params`.
`system.access.table_lineage`, `column_lineage`	Read/write lineage between tables, notebooks, jobs, pipelines.
`system.compute.clusters`, `node_timeline`, `warehouses`, `warehouse_events`	Cluster configs over time; per-node CPU/memory utilisation per minute; warehouse scaling events.
`system.lakeflow.jobs`, `job_tasks`, `job_run_timeline`, `job_task_run_timeline`	Job definitions and every run/task run with result state, start/end, trigger type, compute — the way to build a "failed jobs last 24h" dashboard.
`system.lakeflow.pipelines`	Pipeline definitions (SDP).
`system.query.history`	SQL warehouse query history: text, duration, rows, user, warehouse; find slow/expensive queries.
`system.storage.predictive_optimization_operations_history`	What OPTIMIZE/VACUUM/ANALYZE predictive optimisation ran and cost.
`system.information_schema`	Metadata of every catalog (tables, columns, grants).

```
-- Cost per job over the last 30 days
SELECT u.usage_metadata.job_id, SUM(u.usage_quantity * p.pricing.default) AS usd
FROM system.billing.usage u
JOIN system.billing.list_prices p
  ON u.sku_name = p.sku_name AND u.cloud = p.cloud
 AND u.usage_start_time >= p.price_start_time AND (p.price_end_time IS NULL OR u.usage_start_time <
p.price_end_time)
WHERE u.usage_date >= current_date() - 30 AND u.usage_metadata.job_id IS NOT NULL
GROUP BY 1 ORDER BY usd DESC;

-- Failed task runs in the last day
SELECT job_id, task_key, result_state, period_start_time, period_end_time
FROM system.lakeflow.job_task_run_timeline
WHERE result_state IN ('FAILED','TIMED_OUT','ERROR') AND period_start_time > now() - INTERVAL 1 DAY;
```

4.2 Query Profile and Spark UI

Query Profile (SQL warehouses & DBSQL)

- Graph/tree of operators with per-operator **time, rows, memory, spill, bytes read, files read vs files pruned**.
- Look for: scans reading most files (poor data skipping/clustering), **shuffle** nodes with large bytes, **spill to disk**, join type (nested loop = bad), low pruning ratio, wall-clock dominated by a single operator, "waiting for compute" (warehouse start/scale).
- Available from Query History → query → Query Profile; DAG and details can be downloaded/shared.

Spark UI (clusters, jobs, pipelines)

- **Jobs → Stages → Tasks**: task duration distribution (skew: max ≫ median), **shuffle read/write**, **spill (memory/disk)**, GC time, input size.
- **SQL/DataFrame** tab: physical plan with metrics per node (rows output, size of files read, number of partitions).
- **Executors**: memory, GC, task failures, lost executors (spot loss).
- **Storage**: cached datasets; **Environment**: effective configs; **Streaming**: batch duration, input rate, processing rate, watermark, state size.

4.3 Programmatic monitoring: REST API, CLI and SDK

```
# Jobs API 2.2
GET  /api/2.2/jobs/runs/list?job_id=123&active_only=true&limit=25   # runs (paginated with next_page_token)
GET  /api/2.2/jobs/runs/get?run_id=456                              # state.life_cycle_state / result_state,
tasks[]
GET  /api/2.2/jobs/runs/get-output?run_id=789                       # task-level output (notebook exit value,
error)
POST /api/2.2/jobs/runs/cancel     {"run_id": 456}
POST /api/2.2/jobs/runs/repair     {"run_id": 456, "rerun_all_failed_tasks": true, "job_parameters": {...}}

# Pipelines API
GET  /api/2.0/pipelines/{pipeline_id}                # state, latest_updates
GET  /api/2.0/pipelines/{pipeline_id}/updates        # update history
GET  /api/2.0/pipelines/{pipeline_id}/events         # event log (same as event_log())

# CLI equivalents
databricks jobs list-runs --job-id 123 --active-only
databricks jobs get-run 456
databricks pipelines get <pipeline_id>
databricks pipelines list-pipeline-events <pipeline_id>
```

- Run states: `life_cycle_state` (PENDING, RUNNING, TERMINATING, TERMINATED, SKIPPED, INTERNAL_ERROR) and `result_state` (SUCCESS, FAILED, TIMEDOUT, CANCELED). For multi-task runs, inspect `tasks[]` and call `get-output` per task `run_id`.
- Authenticate with a service principal (OAuth M2M) in automation, not a personal token.

The **Databricks SDK** (Python, Go, Java) wraps the same endpoints with typed objects, pagination and unified auth (env vars, profiles, OAuth). It is the exam's expected tool for monitoring scripts and notebooks:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.jobs import RunResultState
w = WorkspaceClient()                       # auth from DATABRICKS_HOST / CLIENT_ID / CLIENT_SECRET or a profile

failed = [r for r in w.jobs.list_runs(job_id=123, expand_tasks=True)      # paginates automatically
          if r.state and r.state.result_state == RunResultState.FAILED]
for r in failed:
    for t in r.tasks or []:
        if t.state.result_state == RunResultState.FAILED:
            out = w.jobs.get_run_output(run_id=t.run_id)                  # task-level error
            print(r.run_id, t.task_key, out.error)
w.jobs.repair_run(run_id=failed[0].run_id, rerun_all_failed_tasks=True, job_parameters={"date": "2026-10-01"})

for e in w.pipelines.list_pipeline_events(pipeline_id="abc-123", max_results=50):   # SDP event log via API
    if e.level == "ERROR": print(e.timestamp, e.message)
w.statement_execution.execute_statement(warehouse_id="w1",
    statement="SELECT * FROM system.lakeflow.job_task_run_timeline WHERE result_state = 'FAILED' LIMIT 10")
```

- SDK objects map 1:1 to REST resources (`w.jobs`, `w.pipelines`, `w.clusters`, `w.warehouses`, `w.alerts_v2`, `w.policies`, `w.statement_execution`); the CLI is built on the same SDK. Prefer SDK in code, CLI in shell/CI, REST only where no SDK exists.

4.4 Spark Declarative Pipelines event log

Every pipeline writes a structured event log (a Delta table). Query it with the `event_log` table-valued function or publish it to Unity Catalog (pipeline setting `event_log`: catalog/schema/name) so alerts and dashboards can read it.

```
SELECT timestamp, event_type, level, message, details
FROM event_log(TABLE(prod.silver.orders))          -- or event_log('<pipeline_id>')
WHERE level = 'ERROR' ORDER BY timestamp DESC;

-- Data-quality metrics per expectation (flow_progress events)
SELECT timestamp, e.name, e.dataset, e.passed_records, e.failed_records
FROM (SELECT timestamp,
        explode(from_json(details:flow_progress:data_quality:expectations,
          'array<struct<name:string,dataset:string,passed_records:int,failed_records:int>>')) e
      FROM event_log(TABLE(prod.silver.orders)) WHERE event_type = 'flow_progress');
```

- Event types: `flow_definition` (lineage/schema), `flow_progress` (metrics, expectations, backlog), `user_action`, `cluster_resources` (autoscaling), `maintenance`, `planning_information` (MV incremental vs full refresh reason).
- `details:flow_progress:metrics:num_output_rows`, `backlog_bytes`; use `origin.update_id` to scope to a run.

4.5 Lakehouse alerts: one mechanism, many signals

A Databricks SQL alert is a saved query on a SQL warehouse + a condition on a result column + a schedule + notification destinations (email, Slack, Teams, PagerDuty, webhook). The new outline lists the *signals* you should know how to alert on; almost all are "write the right query against the right table":

Signal	Source to query / feature	Example condition
Governed business metrics	A Unity Catalog **metric view** (Ch. 9): `SELECT MEASURE(net_revenue) FROM gold.kpi_mv WHERE d = current_date()-1`	Value below yesterday's by > 20%
Data quality & anomalies	**Data quality monitoring** (formerly Lakehouse Monitoring): **anomaly detection** at schema level (freshness/completeness health per table, root-cause hints, alerts creatable from its UI) and **data profiling** metrics tables per table; plus expectation counts from `event_log()`	Table marked unhealthy; null % above threshold; failed_records > 0
Usage and cost	`system.billing.usage` × `list_prices`; budget policies	Daily serverless spend > budget; a job's cost doubled vs 7-day average
SQL warehouse / query health	`system.query.history`, `system.compute.warehouse_events`	p95 duration > 60 s; failed queries > N; warehouse scaling to max for > 30 min
Audit & security events	`system.access.audit`	Grants to `account users`; permission changes on PHI schemas; failed logins; policy deletions
AI agent quality	MLflow evaluation / inference (`ai_gateway` usage & payload) tables, Agent Evaluation metrics, data-profiling on inference tables	Groundedness or safety score below threshold; error rate up
Lakeflow Job branching	The **SQL alert task** in Lakeflow Jobs evaluates an alert as a task and exposes its state (`TRIGGERED` / `OK`) as a task value, so an **If/else** task can branch on it	Skip the publish task when the freshness alert is triggered; page on-call otherwise

Alert mechanics

- Condition on one aggregated value (or per-row with notification once/each time), schedule (cron), notification destinations configured at workspace level, custom templated message body.
- Can be defined in bundles/Terraform and managed via `w.alerts_v2` in the SDK.
- Freshness/volume checks (`max(updated_at) < now() - INTERVAL 2 HOURS`, `count(*) = 0`) are the classic exam examples; anomaly detection now does these automatically for whole schemas.

Job notifications & health rules (still on the exam)

- Job/task notifications on start / success / failure / duration warning to users or notification destinations.
- **Health rules**: `RUN_DURATION_SECONDS` and, for streaming tasks, `STREAMING_BACKLOG_BYTES / RECORDS / SECONDS / FILES`.
- `notification_settings.no_alert_for_skipped_runs / no_alert_for_canceled_runs`; system notifications on retries; pipeline notifications on update failure / flow failure.

Domain 4 — Practice Questions

14 questions

Q1 **Finance asks for the monthly dollar cost of each job across all workspaces, attributed by a `cost_center` cluster tag. Which data source answers this with the least effort?**

- **A.** Scrape the Jobs UI run pages.
- **B.** Join `system.billing.usage` (custom tags, `usage_metadata.job_id`) with `system.billing.list_prices`.
- **C.** Parse cloud provider invoices.
- **D.** Use the Spark UI Executors tab.

Q2 **A security reviewer needs to know which users queried the `patients` table in the last 90 days. Which table should be queried?**

- **A.** `system.billing.usage`
- **B.** `system.access.audit` (and `system.access.table_lineage` for read events by job/notebook)
- **C.** `system.compute.clusters`
- **D.** `system.lakeflow.jobs`

Q3 **In a Query Profile, the scan operator shows "files read: 12,000 / files pruned: 30" for a query filtering on `event_date`. What does this indicate and what is the remedy?**

- **A.** Data skipping is ineffective for `event_date`; cluster (liquid clustering) or partition/Z-order the table by that column and run OPTIMIZE.
- **B.** The warehouse is too small; scale up.
- **C.** The query needs a broadcast hint.
- **D.** Deletion vectors should be disabled.

Q4 **A job stage shows one task with 25 minutes duration while the median is 40 seconds and a large "spill (disk)" value. What is the most likely diagnosis?**

- **A.** Network throttling.
- **B.** Data skew on the join/aggregation key causing one partition to exceed memory.
- **C.** A missing checkpoint.
- **D.** Too many small files in the source.

Q5 **An operations script must page on-call when any task of a multi-task job fails and include the task error message. Which API sequence is correct?**

- **A.** `runs/list` for the job → `runs/get` on the run to read `tasks[]` and their `state` → `runs/get-output` with each failed task's `run_id` for the error.
- **B.** `jobs/get` only; it returns errors.
- **C.** `runs/get-output` with the parent run id, which returns all task outputs.
- **D.** Query `system.query.history`.

Q6 **A data steward wants a daily report of how many rows each SDP expectation dropped. Where is this information?**

- **A.** In the target table's `_rescued_data` column.
- **B.** In `flow_progress` events of the pipeline event log (`details:flow_progress:data_quality:expectations`), queryable with `event_log()`.
- **C.** In `system.billing.usage`.
- **D.** In the cluster driver log.

Q7 **A streaming job normally keeps up, but on some days backlog grows for hours before anyone notices. Which configuration gives the earliest automatic warning?**

- **A.** A job health rule on `STREAMING_BACKLOG_SECONDS` (or bytes/records) with a notification destination.
- **B.** A success notification on the job.
- **C.** Increasing the trigger interval.
- **D.** A SQL alert on `system.billing.usage`.

Q8 **Which is the best way to alert when a gold table has not been updated in the last 2 hours?**

- **A.** A SQL alert on a query such as `SELECT max(updated_at) < now() - INTERVAL 2 HOURS AS stale FROM gold.kpi` scheduled every 15 minutes, notifying Slack.
- **B.** A job failure notification.
- **C.** A Spark UI bookmark.
- **D.** A `VACUUM` schedule.

Q9 **Which statement about system tables is correct?**

- **A.** They are per-workspace and only visible to workspace admins.
- **B.** They are account-wide Delta tables in the `system` catalog, governed by Unity Catalog; a metastore admin must grant access (and enable some schemas).
- **C.** They retain data indefinitely.
- **D.** They are only accessible through the REST API.

Q10 **A materialized view took much longer than usual to refresh. Which event log detail explains whether it was recomputed fully and why?**

- **A.** `cluster_resources` events
- **B.** `planning_information` events showing the refresh technique chosen (incremental vs complete) and the reason
- **C.** `user_action` events
- **D.** `system.access.audit`

Q11 **The team wants failure notifications for a job but not for runs that were skipped because a previous run was still active. Which setting applies?**

- **A.** `notification_settings.no_alert_for_skipped_runs = true`
- **B.** Set `max_retries = 0`
- **C.** Set `max_concurrent_runs = 10`
- **D.** Disable queueing

Q12 **A daily job must publish a gold table only if a freshness check passes, and page on-call otherwise — all inside the job's DAG rather than in a separate script. Which design is native?**

- **A.** A **SQL alert task** that evaluates the freshness alert, followed by an **If/else** task branching on the alert task's result value to either run the publish task or a notification task.
- **B.** A notebook that polls the alert's REST endpoint in a loop.
- **C.** A job health rule on run duration.
- **D.** Two separate jobs triggered by the same schedule.

Q13 **A steward wants every table in the `prod.silver` schema watched for late or missing loads without writing a query per table, with alerts when a table becomes unhealthy. Which feature is designed for this?**

- **A.** Enable **anomaly detection** (data quality monitoring) on the schema and create alerts from its results.
- **B.** Add `EXPECT` constraints to every table.
- **C.** Increase job retries.
- **D.** Query the Spark UI nightly.

Q14 **A Python monitoring script must list failed runs of several jobs across pages of results, fetch each failed task's error, and repair the runs with a corrected parameter, authenticating as a service principal. Which approach is recommended?**

- **A.** Hand-written `requests` calls with a personal access token.
- **B.** The Databricks SDK (`WorkspaceClient`) using `jobs.list_runs`, `jobs.get_run_output` and `jobs.repair_run` with OAuth M2M credentials.
- **C.** Screen-scraping the Jobs UI.
- **D.** `DESCRIBE HISTORY` on the output tables.

Domain 4 — Answer Key

Q	Ans	Explanation
1	B	Billing system tables carry job id and custom tags per usage record; list_prices converts DBUs to currency. The other options are manual or wrong scope.
2	B	Audit logs record table access events; lineage tables show which workloads read it. Billing/compute tables have no access data.
3	A	A tiny pruned count means the file statistics don't help for that predicate; physical layout by the filter column fixes it.
4	B	Long-tail task + spill is the signature of skew. Small files cause many short tasks, not one long one.
5	A	get-output works on task run ids, not multi-task parent runs; the parent run's tasks[] array supplies them.
6	B	Expectation metrics are emitted per flow update in the event log. Rescued data is an Auto Loader schema concept.
7	A	Streaming backlog health rules fire while the run is still active; success notifications never fire for a healthy-but-lagging stream.
8	A	Freshness is a data condition — a SQL alert. Job notifications don't detect an upstream silently producing nothing.
9	B	System tables are UC-governed, account-wide, with finite retention (mostly a year) and SQL-accessible.
10	B	Planning events report why an MV was fully recomputed (e.g. non-incrementalisable operator, source change).
11	A	Notification settings let you mute skipped and cancelled runs; the other options change execution, not alerting.
12	A	The SQL alert task returns its evaluation state as a task value; If/else consumes it. This is the "Lakeflow Job branching" objective.
13	A	Anomaly detection monitors freshness and completeness for a whole schema with no per-table configuration and supports alerts; expectations are per pipeline table.
14	B	The SDK handles pagination, typed responses and unified auth; service principals with OAuth are the automation standard.

DOMAIN 5

Cost & Performance Optimization

15% OF EXAM

Now 15% — the second-largest domain and the one where "reasonable-sounding" distractors are most dangerous. You need to know how Delta physically stores and skips data, what **Predictive Optimization** automates for UC managed tables, when to pick deletion vectors vs liquid clustering vs `CLUSTER BY AUTO`, how the **Delta cache** helps repeated reads, and how to read a query profile to name the bottleneck. Expect ~9 questions.

OFFICIAL OBJECTIVES (NEW EXAM)

- Explain how Unity Catalog managed tables, Predictive Optimization and Liquid Clustering reduce operational and maintenance overhead for a given workload.
- Choose the appropriate Delta optimization technique (deletion vectors, Liquid Clustering, `CLUSTER BY AUTO`) for a given table access pattern.
- Understand how caching (Delta cache) improves query performance for repeated reads of the same data.
- Apply Change Data Feed to expose row-level changes (updates/deletes) for efficient incremental downstream processing.
- Use the query profile to identify bottlenecks: data-skipping inefficiencies, join strategies, shuffle operations.
- Compare Liquid Clustering vs partitioning/Z-ORDER for a given table size and query pattern.

5.1 Why Unity Catalog managed tables

	UC managed table	External table
Storage	Databricks-managed location (metastore or catalog/schema storage root); path abstracted	Your path in an external location; other tools may write there
Maintenance	**Predictive optimisation**: automatic OPTIMIZE, VACUUM, ANALYZE (statistics), automatic liquid clustering (`CLUSTER BY AUTO`); file-size tuning; deletion vectors on by default	You schedule OPTIMIZE/VACUUM/ANALYZE yourself
Performance	Faster metadata & reads (Databricks knows the layout), auto stats for the CBO	Depends on your maintenance
Lifecycle	`DROP TABLE` deletes data (recoverable with `UNDROP TABLE` within 7 days)	DROP removes only metadata; files remain
Use external when	Data must be read/written by non-Databricks tools in place, regulatory location constraints, or migrating existing paths. Otherwise default to managed.	

Predictive Optimization — what it actually does

- Enabled per account/catalog/schema (`ALTER CATALOG c ENABLE PREDICTIVE OPTIMIZATION`); applies to **UC managed tables** only. Databricks watches read/write patterns and runs, on its own serverless compute, **OPTIMIZE** (compaction and incremental liquid clustering), **VACUUM** (default retention), and **ANALYZE** (column statistics for the CBO) when the expected benefit exceeds the cost.
- With **automatic liquid clustering** (`CLUSTER BY AUTO`) it also *chooses and changes clustering keys* based on query predicates — keys are re-evaluated as workloads shift, without rewrites until an OPTIMIZE reclusters.
- Cost is billed as serverless DBUs and visible in `system.storage.predictive_optimization_operations_history`; tables that are never queried are left alone.
- What you still own: retention settings (`delta.deletedFileRetentionDuration`), purge for compliance (REORG PURGE), and maintenance of **external** tables.

5.2 Delta physical layout: how queries get fast

Data skipping and file pruning

- Every Delta file has **min/max/null-count statistics** in the transaction log for the first **32 columns** (`delta.dataSkippingNumIndexedCols`; or choose columns with `delta.dataSkippingStatsColumns`). Long strings are truncated in stats. A query with a predicate on such a column skips files whose range cannot match — this is "file pruning".
- Skipping works only if related values are **co-located** in files: sorted/clustered on the predicate column. Random layout → every file's min/max covers everything → nothing pruned.
- **Partition pruning**: partition directories are eliminated by the predicate (needs the partition column in the filter). **Dynamic file pruning**: at join time, the small side's keys are used to prune files of the big table.

- Column pruning: Parquet reads only referenced columns — `SELECT *` on wide tables wastes IO.

Liquid clustering (the modern default)

```
CREATE TABLE prod.silver.claims (claim_id BIGINT, member_id BIGINT, service_date DATE, ...)
CLUSTER BY (member_id, service_date);         -- up to 4 columns; or CLUSTER BY AUTO
ALTER TABLE prod.silver.claims CLUSTER BY (service_date, provider_id);   -- change keys, no rewrite
OPTIMIZE prod.silver.claims;                  -- incrementally clusters unclustered data
OPTIMIZE prod.silver.claims FULL;             -- recluster everything after a key change
```

	Liquid clustering	Hive-style partitioning	Z-ORDER
How	Adaptive data layout tree; OPTIMIZE clusters new data **incrementally**	Physical directories per value	Multi-dimensional sort within OPTIMIZE (space-filling curve)
Change keys	Yes, metadata-only; new writes use new keys	No — rewrite whole table	Re-run OPTIMIZE ZORDER (rewrites files)
Cardinality	Any, including high (ids, timestamps)	Low only; high → small-file explosion	Any, but full rewrites on every OPTIMIZE
Skew	Handles skewed keys	Skewed partitions → giant/small partitions	OK
Concurrency	Row-level concurrency; compatible with DVs and streaming writes	Partition-disjoint writes don't conflict	OPTIMIZE conflicts with writers less than rewrites
Verdict	**Use this** (exam default). Incompatible with partitioning/Z-order on the same table.	Only for >1 TB tables with a low-cardinality column and legal/lifecycle needs (e.g. per-day retention)	Legacy; still fine on existing partitioned tables

Deletion vectors

- Without DVs, deleting one row rewrites the whole Parquet file (copy-on-write). With DVs, DELETE/UPDATE/MERGE write a small bitmap marking rows as removed (**merge-on-read**): writes are far faster; reads apply the mask.
- Enabled by default on new tables in recent runtimes and for UC managed tables (`delta.enableDeletionVectors = true`). Also enables **row-level concurrency** (fewer write conflicts) and photonised MERGE.
- Rows are physically removed only when files are rewritten: `OPTIMIZE` (compaction) or `REORG TABLE t APPLY (PURGE)`, then `VACUUM`. Relevant for GDPR purges (Ch. 6).
- Readers must support DVs (DBR 12.2+/14.1+ for streaming; external engines need reader version 3). Delta Sharing to older clients may need DVs disabled.

Choosing the technique for an access pattern

Access pattern	Pick	Why
Frequent small MERGE/UPDATE/DELETE on large files (CDC targets, GDPR deletes)	**Deletion vectors** (+ row-level concurrency)	Avoids rewriting whole files per change; compaction reconciles later
Selective filters/joins on 1–4 known columns, any cardinality	**Liquid clustering** with explicit keys	Data skipping on those keys, incremental OPTIMIZE, keys changeable
Query patterns unknown or shifting; many tables; small team	`CLUSTER BY AUTO` (managed tables + Predictive Optimization)	Databricks picks and adapts keys from actual predicates; zero tuning
Same data read repeatedly by BI/ad hoc on classic compute	**Delta cache** (automatic) / `CACHE SELECT` to pre-warm	Local SSD copies of remote Parquet remove object-store latency
> 1 TB, coarse low-cardinality lifecycle key (drop a month)	Partitioning (only then), optionally with clustering inside	Partition-level deletes/retention; otherwise avoid
Existing partitioned table, no migration budget	Z-ORDER on secondary filter columns	Legacy but effective; full rewrites on each OPTIMIZE

OPTIMIZE, file sizes, VACUUM

- **Small files** are the #1 performance killer (open/close overhead, huge metadata, poor stats). `OPTIMIZE` compacts to the target size (~1 GB default, auto-tuned down for tables with frequent MERGE via `delta.tuneFileSizesForRewrites` or explicit `delta.targetFileSize`).
- **Optimized writes** (`delta.autoOptimize.optimizeWrite`, on by default for UC managed) shuffle before write to produce fewer, larger files. **Auto compaction** (`autoCompact`) runs a mini-OPTIMIZE after writes.

- `VACUUM` removes files no longer referenced and older than the retention (default 7 days, `delta.deletedFileRetentionDuration`). It **reduces storage cost and time travel**; running with a low retention while streams/readers are active risks `FileNotFound` errors — a safety check blocks < 7 days unless `spark.databricks.delta.retentionDurationCheck.enabled = false`.
- Predictive optimisation does OPTIMIZE/VACUUM/ANALYZE for managed tables automatically based on access patterns — the reason the exam says managed tables "reduce maintenance burden".

5.3 Compute-side performance

Shuffle, partitions, spill

- Wide transformations (joins, groupBy, distinct, repartition, window) trigger a **shuffle** (exchange). Reduce them: broadcast small tables, pre-aggregate, filter early, avoid repeated `distinct`.
- `spark.sql.shuffle.partitions` (default 200; **auto** with AQE on Photon/serverless) — too few → big partitions & spill; too many → tiny tasks and overhead.
- **Spill** = a partition didn't fit in executor memory. Fix: more memory per core (memory-optimised instances), more partitions, remove skew, avoid collecting wide rows/huge strings.
- `repartition(n)` shuffles (balanced); `coalesce(n)` only merges (no shuffle, can create imbalance). Writing a specific number of files: `repartition` before write, or rely on optimized writes.

Engine & caching

- **Photon**: vectorised C++ engine; big wins for SQL/DataFrame on Delta/Parquet (joins, aggregations, MERGE); no benefit for Python UDFs or RDDs; higher DBU rate but usually lower total cost.
- **Disk cache** (Delta cache) caches remote Parquet on local SSD automatically on supported instances; `CACHE SELECT` to pre-warm. **Spark cache** (`df.cache()`) is manual and memory-hungry — use only when a DataFrame is reused several times in one job.
- **AQE**: runtime re-optimisation (coalesce partitions, skew join, join strategy switch). Keep it on.
- Statistics: `ANALYZE TABLE t COMPUTE STATISTICS FOR ALL COLUMNS` feeds the cost-based optimiser (automatic under predictive optimisation).

5.4 Delta cache (disk cache) and other caches

The **Delta cache** (a.k.a. disk cache; "Delta caching" in older docs) automatically copies remote Parquet/Delta data that a query reads onto the **local SSDs** of the worker nodes, in a decompressed columnar layout. Subsequent reads of the same files are served locally — no object-store round trips — which is why repeated reads of the same data (dashboards, iterative analysis, repeated joins to the same dimension) get dramatically faster.

	Delta / disk cache	Spark cache (`df.cache()` / `CACHE TABLE`)	Query result cache (SQL warehouses)
What is stored	File contents on local SSD	DataFrame partitions in executor memory (spill to disk)	Final result set of an identical query
Managed by	Automatic on instance types with local SSDs (enabled by default there; `spark.databricks.io.cache.enabled`); pre-warm with `CACHE SELECT * FROM t WHERE ...`	You, explicitly; must `unpersist`	Automatic; invalidated when the underlying table changes
Consistency	Detects file changes; new Delta versions read new files	Stale until recomputed	Exact-match queries only
Best for	Repeated scans of the same tables on classic clusters / warehouses (remote result & local caches on serverless)	A DataFrame reused several times in one job	Dashboards re-running identical SQL
Doesn't help	First read; data that changes every query; workloads on instances without SSDs	Wide tables that don't fit; single-use DataFrames	Parameterised or changing queries

- Photon reads from the disk cache too; the cache is per cluster (each cluster warms its own), so a long-lived warehouse or a cluster pool benefits most.
- Exam pattern: "the same 200 GB table is scanned by dozens of dashboard queries an hour; queries are IO-bound" → choose SSD-backed instance types / rely on the Delta cache and pre-warm, before reaching for bigger clusters.

5.5 Change Data Feed (CDF)

CDF records row-level changes (`insert`, `update_preimage`, `update_postimage`, `delete`) per commit. It solves the streaming-table limitation: a streaming source must be append-only, but with CDF a downstream stream can consume **updates and deletes** from an upstream table incrementally instead of recomputing a materialized view.

```
ALTER TABLE prod.silver.customers SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

-- Batch read of changes
SELECT * FROM table_changes('prod.silver.customers', 1200)     -- from version, or timestamp
WHERE _change_type IN ('update_postimage','delete','insert');

-- Streaming consumption (Structured Streaming or SDP)
df = (spark.readStream.option("readChangeFeed", "true").option("startingVersion", 1200)
        .table("prod.silver.customers"))
# columns: ..., _change_type, _commit_version, _commit_timestamp
```

- Use CDF to propagate SCD1 changes to gold, to feed AUTO CDC downstream, to sync to external systems, or to audit. Overhead: extra `_change_data` files only for updates/deletes (inserts are derived from the log).
- Latency benefit: instead of a full MV recompute after a small update, a CDF-driven streaming table applies just the changed rows.
- CDF data is retained with the table history — VACUUM removes it; time-travel limits apply.

5.6 Reading a query profile like an examiner

Symptom in profile / Spark UI	Diagnosis	Fix
Scan reads nearly all files despite a selective filter; "files pruned" tiny	Bad data skipping (layout not aligned with predicate, or column beyond the 32 stats columns)	Liquid cluster / Z-order on the filter column; move column into stats columns; OPTIMIZE
Thousands of tiny files in the scan; high "time spent listing"	Small-file problem	OPTIMIZE, optimized writes/auto compaction, fewer partitions
Sort-merge join with a side of a few MB	Missed broadcast	BROADCAST hint / stats / AQE
Nested loop join	Non-equi or missing join condition	Add equality predicate, range join hint
Huge shuffle bytes; one long task; spill	Skew or too few partitions	AQE skew join, salting, more partitions, memory
Time dominated by "Result fetching"/collect	Driver bottleneck (collect, toPandas)	Aggregate on cluster; limit; write to table
Long "waiting for compute"	Warehouse/cluster start-up or scaling	Serverless, min clusters, auto-stop tuning
Python UDF operator dominates	Row-by-row Python	Native functions, Pandas UDF

COST TRAPS THE EXAM LOVES

- All-purpose clusters for scheduled jobs (use job clusters or serverless); clusters without auto-termination; oversized fixed clusters instead of autoscaling.
- Partitioning small tables or on high-cardinality columns; running `OPTIMIZE ZORDER` on every micro-batch.
- Full MV recomputes where a streaming table or CDF would be incremental; `trigger(processingTime)` streams kept alive for hourly data (use `availableNow`).
- VACUUM never run (storage bloat) or run too aggressively (breaks time travel and streams).

Domain 5 — Practice Questions

14 questions

Q1 **Which is the strongest reason the exam gives for preferring Unity Catalog managed tables over external tables?**

- **A.** Managed tables can only be read by Databricks, which is more secure.
- **B.** Databricks performs predictive optimisation (OPTIMIZE, VACUUM, statistics, automatic liquid clustering) and file-size tuning automatically, removing maintenance jobs and improving performance.
- **C.** Managed tables do not support time travel, which saves storage.
- **D.** External tables cannot use deletion vectors.

Q2 **A 20 TB claims table is queried mostly by `member_id` (high cardinality) and `service_date`. It is currently partitioned by `member_id` and has millions of tiny files. What is the best redesign?**

- **A.** Keep the partitioning and run OPTIMIZE hourly.
- **B.** Recreate the table unpartitioned with `CLUSTER BY (member_id, service_date)` (liquid clustering) and let OPTIMIZE cluster incrementally.
- **C.** Partition by `member_id` and Z-order by `service_date`.
- **D.** Partition by both columns.

Q3 **After enabling deletion vectors, a nightly GDPR delete of a few hundred rows completes in seconds instead of an hour. A compliance officer asks whether the deleted rows are physically gone from storage. What is the accurate answer?**

- **A.** Yes, DVs rewrite the files immediately.
- **B.** No — rows are masked by a deletion vector; they are physically removed only after the files are rewritten (OPTIMIZE or `REORG TABLE ... APPLY (PURGE)`) and the old files are VACUUMed past the retention period.
- **C.** No, and they can never be removed while DVs are enabled.
- **D.** Yes, as soon as the transaction commits.

Q4 **A query filtering on `diagnosis_code` reads every file of a table clustered by `member_id`. Which statement explains the behaviour?**

- **A.** Data skipping uses per-file min/max statistics; because `diagnosis_code` values are spread across all files, no file's range excludes the predicate.
- **B.** Delta only supports one filter column per query.
- **C.** String columns never have statistics.
- **D.** Photon is required for data skipping.

Q5 **Which change lets a table's clustering keys be changed from `(date)` to `(date, region)` without rewriting existing data immediately?**

- **A.** `ALTER TABLE t CLUSTER BY (date, region)` — later OPTIMIZE (or OPTIMIZE FULL) applies the new layout.
- **B.** `ALTER TABLE t PARTITIONED BY (date, region)`
- **C.** `OPTIMIZE t ZORDER BY (date, region)`
- **D.** Drop and recreate the table with CTAS.

Q6 **A gold materialized view aggregates a 3 TB silver table that receives a few thousand updates per hour; the MV fully recomputes every run and misses its latency SLA. Which redesign lowers latency with the least cost?**

- **A.** Enable Change Data Feed on silver and feed a streaming table (or AUTO CDC target) from the change feed so only changed rows are applied downstream.
- **B.** Increase the MV refresh frequency to every minute.
- **C.** Convert silver to an external table.
- **D.** Add Z-order on the aggregation key.

Q7 **Which set of `_change_type` values does Change Data Feed emit for a row whose value was modified by an UPDATE?**

- **A.** `update`
- **B.** `update_preimage` and `update_postimage`
- **C.** `delete` then `insert`
- **D.** `modify`

Q8 **A Query Profile shows a join implemented as a broadcast nested loop join dominating runtime. The SQL is `JOIN dim d ON f.region_code = d.code OR f.legacy_code = d.code`. What should be done?**

- **A.** Increase the warehouse size.
- **B.** Rewrite the OR condition as two equi-joins combined with UNION (or COALESCE the keys) so a hash/sort-merge join can be used.
- **C.** Disable AQE.
- **D.** Add `DISTINCT`.

Q9 **An engineer sets `delta.deletedFileRetentionDuration = 'interval 1 hour'` and runs VACUUM every hour to cut storage cost. A downstream streaming job now fails intermittently with file-not-found errors. Why?**

- **A.** VACUUM removed files that the streaming query (or a time-travel/long-running reader) still referenced; retention must exceed the longest reader/stream lag.
- **B.** VACUUM corrupts the transaction log.
- **C.** Streaming requires deletion vectors to be disabled.
- **D.** The checkpoint must be on DBFS.

Q10 **A scheduled ETL job runs for 10 minutes every hour on an all-purpose cluster with auto-termination set to 120 minutes. Which change most reduces cost without affecting the SLA?**

- **A.** Run the job on a job cluster (or serverless) created per run.
- **B.** Enable Photon on the all-purpose cluster.
- **C.** Increase auto-termination to 240 minutes to avoid cold starts.
- **D.** Add more worker nodes so the job finishes faster.

Q11 **Which statement about `OPTIMIZE` on a liquid-clustered table is true?**

- **A.** It rewrites the entire table every time.
- **B.** It is incremental: only data not yet clustered (new writes) is rewritten, so it can run frequently at low cost.
- **C.** It requires a `ZORDER BY` clause.
- **D.** It is not allowed while streams write to the table.

Q12 **A Structured Streaming job writes micro-batches every 30 seconds to a Delta table, producing thousands of ~2 MB files per day; downstream queries slow down. Which is the most appropriate remedy set?**

- **A.** Enable optimized writes and auto compaction on the table (or rely on predictive optimisation for a managed table) and run OPTIMIZE regularly.
- **B.** Lower the trigger interval to 5 seconds.
- **C.** Convert the table to Parquet.
- **D.** Partition by ingestion minute.

Q13 **Thirty dashboards scan the same 300 GB gold table every few minutes on a classic SQL cluster; the query profile shows time dominated by cloud storage reads. Which change most directly improves repeated-read latency?**

- **A.** Run the workload on instance types with local SSDs so the Delta (disk) cache serves repeated reads, and pre-warm with `CACHE SELECT` after each refresh.
- **B.** Increase `spark.sql.shuffle.partitions`.
- **C.** Partition the table by dashboard name.
- **D.** Convert the table to CSV.

Q14 **A platform team manages 4,000 managed tables and cannot hand-pick clustering keys for each. Query patterns differ per team and change over time. Which option minimises tuning effort while keeping data skipping effective?**

- **A.** Partition every table by date.
- **B.** Enable Predictive Optimization and create tables with `CLUSTER BY AUTO` so clustering keys are selected and adapted from actual query predicates.
- **C.** Z-order every table by its primary key nightly.
- **D.** Disable statistics collection to save cost.

Domain 5 — Answer Key

Q	Ans	Explanation
1	B	That is exactly the objective wording: managed tables reduce operational overhead through automated maintenance. A, C, D are false.
2	B	High-cardinality partitioning is the anti-pattern; liquid clustering handles both keys, any cardinality, and clusters incrementally. C/D keep the bad partitioning.
3	B	DVs are merge-on-read; physical removal needs a rewrite (OPTIMIZE / REORG PURGE) plus VACUUM after retention.
4	A	Skipping depends on co-location. B, C, D are false (strings do get truncated stats; Photon is unrelated).
5	A	Liquid clustering keys are metadata; subsequent OPTIMIZE reclusters. Partitioning can't be altered in place; Z-order is incompatible with liquid clustering.
6	A	CDF turns a non-append source into an incremental change stream — the stated objective. More frequent full recomputes raise cost; Z-order doesn't avoid recompute.
7	B	Updates emit pre- and post-image rows sharing the same commit version.
8	B	OR-joined keys prevent hash-based joins; splitting into equi-joins restores a sort-merge/broadcast hash plan.
9	A	Files removed by VACUUM can still be referenced by lagging readers; retention must cover them (default 7 days is the safety check).
10	A	Job clusters cost less per DBU and exist only for the run; the all-purpose cluster idles 110 of 120 minutes at full cost.
11	B	Incremental clustering is the headline advantage over Z-order. OPTIMIZE FULL is the explicit full rewrite.
12	A	Small files from frequent micro-batches are fixed by write-time optimisation and compaction; a shorter trigger makes it worse.
13	A	The Delta cache keeps remote files on local SSD; repeated scans become local reads. Shuffle partitions don't affect scans; partitioning by dashboard is meaningless.
14	B	Automatic liquid clustering under Predictive Optimization chooses keys from workload predicates; date partitioning and PK Z-order ignore actual query patterns and cost rewrites.

DOMAIN 6

Ensuring Data Security and Compliance

8% OF EXAM

Directly relevant to healthcare work. The exam tests **mechanisms** (least-privilege ACLs, row filters and column masks — now primarily through **ABAC policies driven by governed tags** — secrets, masking functions) and **process** (PII detection + masking across batch and streaming, retention/purge that truly removes data for GDPR erasure). Expect ~5 questions.

OFFICIAL OBJECTIVES (NEW EXAM)

- Apply least-privilege ACLs to secure Unity Catalog securable objects and workspace resources.
- Apply attribute-based access control (ABAC) policies with governed tags to enforce row filters and column masks at scale.
- Apply anonymization and pseudonymization (hashing, tokenization, suppression, generalization) to confidential data using column masks and related Unity Catalog features.
- Implement a compliant pipeline that enforces PII detection and masking across batch and streaming workloads using Unity Catalog features.
- Implement a data purging strategy that satisfies retention and deletion policies (GDPR right-to-erasure / right-to-be-forgotten) using Delta Lake and Unity Catalog features.

6.1 Access control layers

Layer	Objects	Permissions
Unity Catalog (data)	Metastore → catalog → schema → table/view/volume/function/model; external locations, storage credentials, connections, shares	`USE CATALOG`, `USE SCHEMA`, `SELECT`, `MODIFY`, `CREATE TABLE/SCHEMA/FUNCTION/VOLUME`, `EXECUTE`, `READ/WRITE VOLUME`, `BROWSE`, `MANAGE`, `ALL PRIVILEGES`; ownership
Workspace ACLs	Notebooks/files/folders, jobs, pipelines, clusters, cluster policies, pools, SQL warehouses, dashboards, queries, alerts, secrets scopes, repos/Git folders, MLflow experiments	e.g. notebooks: `CAN READ / CAN RUN / CAN EDIT / CAN MANAGE`; jobs: `CAN VIEW / CAN MANAGE RUN / IS OWNER / CAN MANAGE`; clusters: `CAN ATTACH TO / CAN RESTART / CAN MANAGE`; secrets: `READ / WRITE / MANAGE`
Account & identity	Users, groups, service principals (SCIM), workspace assignment, entitlements (workspace access, DBSQL access, cluster creation)	Account-level groups are the unit of least-privilege design; avoid user-level grants
Compute policies	Cluster policies limit instance types, DBR versions, tags, max workers, allow only UC-enabled access modes (standard/dedicated)	Policy enforcement = prevent expensive or non-compliant clusters

- **Least privilege**: grant on the smallest scope that works, to groups, with time-bound elevation. Give job **CAN MANAGE RUN** (not owner) to operators; give analysts **SELECT on views**, not underlying tables; give service principals ownership of production jobs.
- **Secrets**: `dbutils.secrets.get(scope, key)`; values are redacted as `[REDACTED]` in notebook output; use secret ACLs; reference secrets in Spark configs as `{{secrets/scope/key}}`; UC connections use `secret('scope','key')`.
- Compute access modes: **standard** (shared, UC-enforced, most restrictive), **dedicated** (single user/group). Table ACLs are only enforced on UC-enabled access modes.

6.2 Row filters and column masks

Both are SQL UDFs attached to a table. They are enforced for every reader and every engine (SQL, Python, R, Scala) — unlike dynamic views, which protect only the view. The function owner controls logic; consumers need only SELECT on the table.

```
-- Row filter: nurses see only their facility; admins see everything
CREATE FUNCTION sec.facility_filter(facility_id STRING)
RETURN is_account_group_member('phi_admins')
    OR facility_id IN (SELECT f FROM sec.user_facilities WHERE user = current_user());
ALTER TABLE clinical.encounters SET ROW FILTER sec.facility_filter ON (facility_id);

-- Column mask: reveal SSN only to a privileged group
CREATE FUNCTION sec.mask_ssn(ssn STRING)
RETURN CASE WHEN is_account_group_member('phi_admins') THEN ssn
            ELSE CONCAT('***-**-', RIGHT(ssn, 4)) END;
ALTER TABLE clinical.patients ALTER COLUMN ssn SET MASK sec.mask_ssn;

-- Masks can take other columns as inputs: ALTER COLUMN dob SET MASK sec.mask_dob USING COLUMNS (consent_flag);
ALTER TABLE clinical.patients ALTER COLUMN ssn DROP MASK;  ALTER TABLE clinical.encounters DROP ROW FILTER;
```

DYNAMIC VIEWS VS ROW FILTERS/COLUMN MASKS

- **Dynamic view**: `CREATE VIEW v AS SELECT CASE WHEN is_account_group_member('g') THEN col ELSE 'REDACTED' END ...` . Protects only the view; the base table must be locked down separately. Good for combining logic and joins; the pattern used in the official sample question.
- **Row filter / column mask**: applied to the **table itself**; every query path sees filtered/masked data. Prefer these when the table is queried directly by many tools.
- Use `is_account_group_member()` (account-level groups) in UC; `is_member()` checks workspace-local groups. `current_user()` returns the caller.
- Limitations to remember: tables with filters/masks have restrictions as streaming sources, in Delta Sharing, MERGE targets in some cases, and with shallow clones — check docs when the question mentions them.

6.3 ABAC: governed tags + policies at catalog/schema scope

Table-level masks and filters (6.2) don't scale: every new table needs its own `ALTER TABLE` , logic drifts, and reviewers cannot see one rule. **Attribute-based access control** (GA in Unity Catalog since 2026) fixes this: a governance admin defines a **policy once** at catalog or schema scope; the policy matches tables and columns by **governed tags**; protection then follows the data automatically — including tables created and tagged later. This is now the recommended approach for consistent row filtering and column masking across many tables.

Building block	Details
Governed tags	Account-level tag keys with an optional fixed list of allowed values (e.g. `pii` ∈ {ssn, email, phone}, `sensitivity` ∈ {low, high}); permissions `ASSIGN` (who may apply the tag) and `APPLY TAG` on the object. Tags inherit down catalog → schema → table (overridable) but **column tags must be set directly on the column**. Automated **data classification** can detect PII and apply governed tags for you.
Policy	Attached to a catalog, schema or table; type **row filter**, **column mask** (or GRANT/DENY policies for privileges). Specifies **TO** principals (who is subject), optional **EXCEPT** principals, a **WHEN** table condition and **MATCH COLUMNS** column conditions using `has_tag('k')` / `has_tag_value('k','v')` , and the UC function that implements the logic (`USING COLUMNS` passes matched columns or constants).
Function	A UC SQL or Python UDF you have `EXECUTE` on (or one defined inline). Row filter → returns BOOLEAN; column mask → returns a value castable to the column type.
Permissions	Creating/altering a policy needs `MANAGE` or ownership on the scope object; viewing needs `READ METADATA` / `MANAGE` . Separation of duties: stewards tag, governance admins write policies, data owners never touch each table.
Compute	Serverless, or DBR 16.4+ standard/dedicated compute (dedicated needs fine-grained access control enabled). Older runtimes cannot read ABAC-protected tables. Cross-engine enforcement exists for external Iceberg/Delta clients reading managed tables.

```
-- 1) Governed tag taxonomy (account admin), then tag columns (steward)
ALTER TABLE prod.clinical.patients ALTER COLUMN ssn SET TAGS ('pii' = 'ssn');
ALTER TABLE prod.clinical.patients SET TAGS ('sensitivity' = 'high');

-- 2) Masking function (UC), owned by governance
CREATE FUNCTION prod.sec.last_n(v STRING, n INT) RETURNS STRING RETURN CONCAT('***', RIGHT(v, n));

-- 3) One policy for every current and future table in the catalog
CREATE POLICY mask_ssn ON CATALOG prod
COMMENT 'Mask SSN for everyone except PHI admins'
COLUMN MASK prod.sec.last_n
TO `account users` EXCEPT `phi_admins`
FOR TABLES
MATCH COLUMNS has_tag_value('pii', 'ssn') AS ssn
ON COLUMN ssn
USING COLUMNS (4);

-- Row filter policy: restrict high-sensitivity tables to the user's facility
CREATE FUNCTION prod.sec.my_facility(f STRING) RETURNS BOOLEAN
  RETURN is_account_group_member('phi_admins') OR f IN (SELECT facility FROM prod.sec.user_facilities WHERE user
= current_user());
CREATE POLICY facility_rows ON SCHEMA prod.clinical
ROW FILTER prod.sec.my_facility TO `account users` FOR TABLES
WHEN has_tag_value('sensitivity', 'high')
MATCH COLUMNS has_tag('facility_col') AS fac
USING COLUMNS (fac);

SHOW EFFECTIVE POLICIES ON TABLE prod.clinical.patients;     -- includes inherited catalog/schema policies
SELECT * FROM system.information_schema.abac_policy_definitions;   -- inventory across catalogs
```

WHEN TO USE WHICH — THE COMPARISON THE EXAM WANTS

	ABAC policy	Table-level row filter / column mask	Dynamic view
Scope	Catalog / schema / table, matched by tags	One table	One view
Covers new tables	**Yes**, once tagged	No	No
Who maintains	Governance admin (central)	Table owner	View owner
Protects base table	Yes	Yes	No — base table must be locked separately
Use when	Many tables, consistent rules, separation of duties, PII that appears over time	A single special table; quick fix	Curated/joined/reshaped view for a specific audience

A policy's **EXCEPT** list is how you exempt admins/compliance; a table can be under several effective policies (all apply). Audit every tag assignment and policy change from `system.access.audit` (`createPolicy`, `createEntityTagAssignment`).

6.4 Anonymisation vs pseudonymisation

Technique	Type	Reversible?	Example
Suppression	Anonymisation	No	Drop the column / null it out: `SELECT * EXCEPT (ssn)`
Generalisation (bucketing, truncation)	Anonymisation	No	Age → age band (`floor(age/10)*10`), ZIP → ZIP3, timestamp → date, lat/long rounding
Hashing (salted/keyed)	Pseudonymisation	Not directly, but joinable and linkable; unsalted hashes of low-entropy data are reversible by dictionary	`sha2(concat(secret_salt, email), 256)`; keep the salt in a secret
Tokenisation	Pseudonymisation	Yes, via a protected lookup table/vault	Replace `member_id` with a random token; mapping table in a restricted schema
Encryption	Pseudonymisation	Yes, with the key	`aes_encrypt(col, key)` / `aes_decrypt`; key in a secret scope or KMS

Technique	Type	Reversible?	Example
Masking (partial)	Presentation control	Original retained	`***-**-1234` via column mask

Regulatory framing (GDPR/HIPAA): **anonymised** data is no longer personal data; **pseudonymised** data still is, because re-identification is possible with the key/mapping — so it still needs access control, but reduces risk and can be shared more freely.

6.5 A compliant PII pipeline (batch and streaming)

1. **Land raw data in a restricted bronze schema** (only the pipeline service principal and a small admin group). Apply column masks there too if humans can query it.
2. **Detect PII**: automated **data classification** (scans columns and applies governed tags), pattern-based checks (regex for SSN/email/phone, `ai_classify` / `ai_extract` on free text), or known-schema tagging. Governed tags are then the switch that turns on ABAC masking policies automatically.
3. **Transform in silver**: pseudonymise identifiers (keyed hash or token), generalise quasi-identifiers, suppress free text if not needed, keep a mapping table in a locked schema. In SDP this is a normal streaming table with masking expressions; in Structured Streaming, the same DataFrame logic or `foreachBatch`.
4. **Enforce at read time**: ABAC column-mask/row-filter policies keyed on tags (or table-level masks for one-offs) on silver/gold; grants to groups; OpenSharing/Clean Rooms only for de-identified views. For streaming, the same policies apply because enforcement is at query time, independent of how the table was written.
5. **Prove it**: expectations such as `EXPECT (email NOT RLIKE '^[^@]+@')` on de-identified tables, lineage and audit logs, tags in the catalog.

```
-- SDP silver with de-identification and a guard-rail expectation
CREATE OR REFRESH STREAMING TABLE silver_patients (
  CONSTRAINT no_raw_email EXPECT (email_hash NOT LIKE '%@%') ON VIOLATION FAIL UPDATE)
AS SELECT
  sha2(concat(secret('phi', 'salt'), lower(email)), 256) AS email_hash,
  floor(datediff(current_date(), dob) / 365.25 / 10) * 10 AS age_band,
  left(zip, 3) AS zip3,
  * EXCEPT (email, dob, zip, ssn)
FROM STREAM bronze_patients;
```

6.6 Purging data for retention compliance

Deleting a row in Delta does **not** remove the bytes: old file versions stay for time travel, deletion vectors keep the row in the file, CDF keeps change records, and downstream copies exist. A compliant purge covers all of them:

1. **Logical delete**: `DELETE FROM t WHERE patient_id IN (SELECT id FROM erasure_requests)` or via `MERGE ... WHEN MATCHED THEN DELETE`. For SDP targets use AUTO CDC with `APPLY AS DELETE WHEN`.
2. **Physical removal**: if DVs are enabled, `REORG TABLE t APPLY (PURGE)` (or OPTIMIZE) to rewrite files; then `VACUUM t` after `delta.deletedFileRetentionDuration` (lower it deliberately for PII tables, e.g. 1 day, balanced against stream/time-travel needs). Set `delta.logRetentionDuration` so history doesn't retain the data longer than allowed.
3. **Propagate**: downstream tables (streaming consumers use CDF or a full refresh; MVs recompute automatically), clones, shares, backups, and the pseudonymisation mapping table (deleting the mapping = crypto-shredding).
4. **Time-based retention**: schedule `DELETE ... WHERE event_date < current_date() - 2555` (7 years) + VACUUM; or partition by month to drop cheaply; document with a job that logs counts to an audit table.
5. **Streams that read the purged table** must tolerate deletes: `skipChangeCommits` / `ignoreDeletes` when the deletes shouldn't be propagated; CDF when they should.

TRAPS

- "We deleted the rows and VACUUMed with default settings" — still recoverable for 7 days; the exam may ask what is missing (retention tuning, DVs purge, downstream copies).
- `VACUUM RETAIN 0 HOURS` needs the safety check disabled and can break concurrent readers — not a routine solution.
- Hashing without a secret salt is not adequate pseudonymisation for low-entropy identifiers (phone numbers, SSNs).

Domain 6 — Practice Questions

14 questions

Q1 **Analysts must query `clinical.encounters` from SQL warehouses, notebooks and a BI tool, seeing only rows for their own facility. Which implementation enforces this for every access path?**

- **A.** A dynamic view per facility.
- **B.** A row filter function attached to the table with `ALTER TABLE ... SET ROW FILTER f ON (facility_id)`, evaluated per user/group.
- **C.** A WHERE clause in the BI dashboard.
- **D.** Separate tables per facility with CAN READ notebook permissions.

Q2 **A view uses `CASE WHEN is_member('phi_admins') THEN ssn ELSE 'REDACTED' END`. A member of the account-level group `phi_admins` that was never added to the workspace-local group still sees REDACTED. Why?**

- **A.** `is_member()` checks workspace-local groups; use `is_account_group_member()` for Unity Catalog account groups.
- **B.** Views cannot use group functions.
- **C.** The user lacks MODIFY on the base table.
- **D.** Column masks override the view.

Q3 **Which technique produces data that is no longer considered personal data under GDPR?**

- **A.** Salted hashing of the email address
- **B.** Tokenisation with a mapping vault
- **C.** Generalising age to 10-year bands and suppressing direct identifiers (irreversible anonymisation)
- **D.** AES encryption of the SSN

Q4 **Two datasets from different departments must be joined on customer email without either side seeing plain emails. Which method keeps joinability?**

- **A.** Suppress the email column in both.
- **B.** Apply the same keyed hash (`sha2(concat(shared_secret_salt, lower(email)), 256)`) in both pipelines and join on the hash.
- **C.** Generalise email to its domain.
- **D.** Mask emails to `***@domain`.

Q5 **A DELETE removed a patient's rows from a Delta table with deletion vectors enabled and default retention. Legal asks for proof that the data is unrecoverable within 24 hours. Which sequence achieves this?**

- **A.** Run `OPTIMIZE` only.
- **B.** Set `delta.deletedFileRetentionDuration` (and log retention) to ≤ 24 hours for this table, run `REORG TABLE t APPLY (PURGE)`, then `VACUUM t` after the retention window; verify with time travel failing.
- **C.** Run `VACUUM t RETAIN 168 HOURS`.
- **D.** Disable Change Data Feed.

Q6 **A job operator must be able to trigger and repair production job runs but must not change the job definition. Which permission?**

- **A.** IS OWNER
- **B.** CAN MANAGE
- **C.** CAN MANAGE RUN
- **D.** CAN VIEW

Q7 **A notebook prints a JDBC password obtained with `dbutils.secrets.get`. What appears in the output?**

- **A.** The plain-text password
- **B.** `[REDACTED]` — secret values are redacted in notebook output
- **C.** A base64-encoded value
- **D.** An error; secrets cannot be assigned to variables

Q8 **Which column-mask definition reveals the full value only to `hr_admins` and shows the last four characters to everyone else?**

- **A.** `CREATE FUNCTION m(s STRING) RETURN CASE WHEN is_account_group_member('hr_admins') THEN s ELSE CONCAT('****', RIGHT(s,4)) END;` then `ALTER TABLE t ALTER COLUMN ssn SET MASK m;`
- **B.** `ALTER TABLE t SET ROW FILTER m ON (ssn)`
- **C.** `GRANT SELECT (ssn) ON t TO hr_admins`
- **D.** `CREATE VIEW v AS SELECT RIGHT(ssn,4) FROM t`

Q9 **A retention policy requires deleting events older than 7 years from a 40 TB append-only table every month, cheaply, and downstream streaming consumers must keep running. Which design is best?**

- **A.** Partition (or cluster) by `event_month`, run a monthly `DELETE WHERE event_month < ...` followed by VACUUM, and configure downstream streams with `ignoreDeletes` / `skipChangeCommits`.
- **B.** Recreate the table monthly with CTAS excluding old data.
- **C.** Set `delta.logRetentionDuration` to 7 years.
- **D.** Use a SHALLOW CLONE and delete from the clone.

Q10 **Which statement about row filters and column masks is correct?**

- **A.** They apply only when querying through Databricks SQL.
- **B.** They are SQL UDFs bound to the table and enforced for all readers regardless of language or compute (UC-enabled).
- **C.** Readers need EXECUTE on the mask function.
- **D.** A column mask can only reference the masked column, never other columns.

Q11 **The pipeline that pseudonymises members stores `member_id → token` mappings. A member exercises the right to erasure. Which action makes the historical pseudonymised records effectively anonymous without rewriting every downstream table?**

- **A.** Delete the member's row from the mapping/vault table (crypto-shredding) so tokens can no longer be resolved, and purge direct identifiers from bronze.
- **B.** Re-hash all downstream tables.
- **C.** Grant BROWSE instead of SELECT on the tables.
- **D.** Enable deletion vectors.

Q12 **An organisation has hundreds of tables containing PII across several schemas and expects more. Governance wants one centrally maintained rule that masks any column tagged `pii = 'email'` for everyone except `privacy_admins`, including tables created next year. What should be implemented?**

- **A.** An `ALTER TABLE ... SET MASK` on every existing table, repeated by a nightly job for new tables.
- **B.** A governed tag `pii`, tagged columns, and one ABAC column-mask policy at catalog scope: `CREATE POLICY ... COLUMN MASK fn TO \`account users\` EXCEPT privacy_admins FOR TABLES MATCH COLUMNS has_tag_value('pii','email') AS c ON COLUMN c`.
- **C.** Dynamic views duplicating every base table.
- **D.** Revoking SELECT on all tables and granting it per request.

Q13 **After an ABAC policy was created on schema `prod.clinical`, analysts on a cluster running Databricks Runtime 14.3 in standard access mode report errors when querying the protected tables, while serverless users are fine. What is the cause?**

- **A.** ABAC requires serverless or DBR 16.4+ compute; older runtimes cannot access ABAC-protected tables.
- **B.** The policy must be created on the table, not the schema.
- **C.** Analysts need EXECUTE on the masking function.
- **D.** Governed tags are not visible to clusters.

Q14 **A steward applied the governed tag `sensitivity = 'high'` to a schema and expected the column-mask policy matching `has_tag_value('pii','ssn')` to start masking the `ssn` column of a new table. Nothing is masked. Why?**

- **A.** Column tags do not inherit from parent objects; the `pii` tag must be applied directly to the `ssn` column (e.g. via data classification or `ALTER COLUMN ... SET TAGS`).
- **B.** Policies apply only to tables that existed when the policy was created.
- **C.** Column masks cannot be defined in ABAC policies.
- **D.** The schema must be re-created.

Domain 6 — Answer Key

Q	Ans	Explanation
1	B	Row filters bind to the table and apply to every engine/tool. Views and dashboard filters protect one path only; per-facility tables don't scale.
2	A	The two functions check different group scopes; UC governance is account-level.
3	C	Hashing, tokenisation and encryption are reversible/linkable → pseudonymisation; generalisation + suppression is anonymisation.
4	B	A deterministic keyed hash preserves equality for joins while hiding values; the others destroy joinability or leak.
5	B	Purge requires rewriting DV-masked files, then VACUUM after a shortened retention; OPTIMIZE alone leaves old files for 7 days; option C keeps the default 7 days.
6	C	CAN MANAGE RUN allows run/cancel/repair without editing settings; CAN MANAGE and IS OWNER allow edits.
7	B	Databricks redacts secret values in cell output (the official sample question tests this).
8	A	That is the column-mask syntax; row filters remove rows, column-level GRANT doesn't exist, the view exposes only a fragment to everyone.
9	A	Deleting by a coarse time key is cheap (whole files/partitions), VACUUM reclaims storage, and streaming readers must be told to ignore deletes.
10	B	Enforcement is table-level and language-agnostic; readers don't need EXECUTE on the function; masks may use other columns via USING COLUMNS.
11	A	Removing the mapping (or key) is crypto-shredding: tokens become meaningless. Rewriting everything is unnecessary; BROWSE/DVs don't address erasure.
12	B	ABAC = governed tag + one policy at the highest applicable scope, with EXCEPT for exemptions; it covers future tables automatically. Per-table masks and duplicated views don't scale; blanket revocation isn't least privilege, it's no access.
13	A	ABAC enforcement needs serverless or DBR 16.4+; scope at schema level is valid; readers don't need EXECUTE on policy functions.
14	A	Table/schema tags inherit downward, but column tags must be set on the column itself; the policy matches columns by their own tags.

DOMAIN 7

Data Governance

5% OF EXAM

Now 5% (≈3 questions). Two objectives: **tags and comments** as the metadata mechanisms that make data discoverable (and, with governed tags, drive ABAC in Domain 6), and the **Unity Catalog permission inheritance model** — predict what a user can see after a set of GRANTs.

OFFICIAL OBJECTIVES (NEW EXAM)

- Demonstrate understanding of Unity Catalog tags and comments as mechanisms for adding metadata to securable objects to improve discoverability.
- Demonstrate understanding of the Unity Catalog permission inheritance model for managing access across catalogs, schemas and objects.

7.1 Unity Catalog object model

```
Metastore (one per region, attached to workspaces)
 └── Catalog (e.g. prod, dev, system)              ← USE CATALOG
      └── Schema / database (e.g. silver)          ← USE SCHEMA
           ├── Table (managed / external), View, Materialized view, Streaming table
           ├── Volume (files, managed / external)
           ├── Function (SQL / Python UDF)          ← EXECUTE
           └── Model (MLflow registered model)
 Non-hierarchical securables: storage credential, external location, connection, share, recipient, provider,
service credential
```

7.2 Privilege inheritance — the rules

- Privileges granted on a **parent** are inherited by **all current and future children**: `GRANT SELECT ON CATALOG prod TO analysts` gives SELECT on every table in every schema of `prod`.
- To read a table you need the **full chain**: `USE CATALOG` on the catalog **and** `USE SCHEMA` on the schema **and** `SELECT` on the table (or inherited from above). USE privileges alone reveal nothing; they are the "door" privileges.
- **Owners** of an object (user, group, or service principal) have all privileges on it and can grant them. Ownership can be transferred (`ALTER TABLE t OWNER TO ...`). The **metastore admin** and catalog owners can grant on anything beneath them.
- **Revoking** a privilege on a child does not remove an inherited grant from the parent — you must revoke at the level where it was granted. There is no DENY.
- **Views**: the view **owner** must have SELECT on the underlying tables; a reader needs only USE CATALOG/SCHEMA + SELECT on the view. This is how views hide base tables.
- `BROWSE` lets users see metadata (names, comments) without reading data — key for discoverability. `MANAGE` lets a non-owner grant/revoke and alter the object. `ALL PRIVILEGES` is a bundle, not a super-role.
- Special principals: `account users` (everyone), `admins`. Grant to **groups**.

```
GRANT USE CATALOG ON CATALOG prod TO `data_analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA prod.gold TO `data_analysts`;     -- covers all current + future tables in
gold
GRANT BROWSE ON CATALOG prod TO `account users`;                     -- everyone can discover, nobody can read
GRANT MODIFY ON TABLE prod.silver.orders TO `etl_sp`;
SHOW GRANTS ON SCHEMA prod.gold;     REVOKE SELECT ON SCHEMA prod.gold FROM `data_analysts`;
```

7.3 Discoverability: descriptions, tags, lineage, classification

- **Comments**: `COMMENT ON TABLE prod.gold.kpi IS 'Daily KPIs, refreshed 06:00 UTC, owner: analytics'`; `ALTER TABLE t ALTER COLUMN c COMMENT '...'`; `COMMENT ON SCHEMA/CATALOG`. Catalog Explorer can generate **AI-suggested comments** for review.
- **Tags** (key/value on catalogs, schemas, tables, columns, volumes, functions, models): `ALTER TABLE t SET TAGS ('domain' = 'claims')`, `ALTER TABLE t ALTER COLUMN c SET TAGS ('pii' = 'email')`; searchable in Catalog Explorer; queryable via `information_schema.table_tags / column_tags / catalog_tags / schema_tags`.
- **Governed tags** vs free-form tags: governed tags are defined at the **account** level with an allowed-value list and `ASSIGN` / `APPLY TAG` permissions, so the taxonomy is consistent across metastores; they inherit catalog → schema → table (not to columns) and

are the attributes ABAC policies match on (`has_tag` , `has_tag_value`). Free-form tags are for ad hoc labelling and cost attribution. Automated **data classification** applies governed tags to detected sensitive columns.

- **Lineage** is captured automatically for tables, columns, notebooks, jobs, pipelines, dashboards — visible in Catalog Explorer and in `system.access.table_lineage` / `column_lineage` . Requires UC-enabled compute.
- **Search** in the workspace uses names, comments, tags, and column names. **Certified**/favourites and the **Insights** tab (frequent users, joins) help discovery.
- **Data classification** can auto-detect and tag sensitive columns; **Lakehouse Monitoring** profiles tables and tracks drift/quality metrics with dashboards.
- `information_schema` in every catalog exposes tables, columns, privileges, tags for programmatic catalogs/docs.

Domain 7 — Practice Questions

11 questions

Q1 **A user was granted `SELECT ON SCHEMA prod.silver` but gets a permission error querying `prod.silver.orders`. What is most likely missing?**

- **A.** `USE CATALOG` on `prod` (and/or `USE SCHEMA` on `prod.silver`)
- **B.** `MODIFY` on the table
- **C.** Ownership of the schema
- **D.** `EXECUTE` on the catalog

Q2 **An admin grants `SELECT ON CATALOG prod` to `analysts`, then runs `REVOKE SELECT ON TABLE prod.gold.salaries FROM analysts`. Can analysts still read `salaries`?**

- **A.** No — the revoke overrides the catalog grant.
- **B.** Yes — inherited privileges from the catalog still apply; you must revoke at the catalog level or restructure (move the table to another catalog/schema, or restrict via a view).
- **C.** Only through SQL warehouses.
- **D.** Only if they own the table.

Q3 **Which privilege lets a data steward see table names, comments and columns in Catalog Explorer without being able to query data?**

- **A.** SELECT
- **B.** BROWSE
- **C.** USE SCHEMA
- **D.** READ VOLUME

Q4 **A view `gold.v_patients_deid` is created by user `alice` over `silver.patients`. Which is required for analyst `bob` to query the view without accessing the base table?**

- **A.** bob needs SELECT on `silver.patients`.
- **B.** alice (the view owner) needs SELECT on `silver.patients`; bob needs USE CATALOG/SCHEMA and SELECT on the view only.
- **C.** bob must be granted ownership of the view.
- **D.** Both need MODIFY on the view.

Q5 **Which statement about Unity Catalog privileges is true?**

- **A.** A DENY on a table blocks inherited grants.
- **B.** Privileges granted on a schema apply to tables created in that schema in the future.
- **C.** Grants must be repeated for every new table.
- **D.** Only the metastore admin can grant privileges.

Q6 **The company wants every gold table to show its business owner, refresh cadence, and a PII flag in search results. Which combination is most appropriate?**

- **A.** Put the information in the table name.
- **B.** Table/column **comments** for descriptions and **tags** (e.g. `owner`, `refresh`, `pii`) for structured, searchable metadata.
- **C.** A README in a Git folder.
- **D.** Store the metadata in a separate Delta table only.

Q7 **Which is the correct SQL to document a column?**

- **A.** `ALTER TABLE t ALTER COLUMN claim_status COMMENT 'Status of the claim: OPEN, PAID, DENIED';`
- **B.** `COMMENT ON COLUMN t.claim_status = 'Status ...';`
- **C.** `SET COLUMN COMMENT t.claim_status 'Status ...';`
- **D.** `DESCRIBE COLUMN t.claim_status 'Status ...';`

Q8 **A compliance team wants to find all columns tagged `pii = 'true'` across the metastore programmatically. Where should they query?**

- **A.** `system.billing.usage`
- **B.** `system.information_schema.column_tags` (or each catalog's `information_schema.column_tags`)
- **C.** The pipeline event log
- **D.** `DESCRIBE HISTORY` on each table

Q9 **Who can grant SELECT on `prod.gold.kpi` to a group?**

- **A.** Only workspace admins
- **B.** The table owner, anyone with MANAGE on it, the owner of the parent schema/catalog, or the metastore admin
- **C.** Any user with SELECT on the table
- **D.** Only the user who created the metastore

Q10 **Lineage for a gold table shows no upstream sources although a job populates it nightly. What is the most likely reason?**

- **A.** Lineage is only captured for SQL warehouses.
- **B.** The job runs on compute without Unity Catalog (e.g. no-isolation/legacy access mode) so lineage is not captured.
- **C.** Lineage requires Delta Sharing.
- **D.** Comments are missing on the table.

Q11 **Which statement distinguishes governed tags from free-form tags in Unity Catalog?**

- **A.** Governed tags are defined at the account level with allowed values and assignment permissions, inherit through the object hierarchy (except to columns), and can be matched by ABAC policies; free-form tags are arbitrary key/value labels.
- **B.** Governed tags can only be applied to catalogs.
- **C.** Free-form tags are enforced by row filters.
- **D.** Governed tags replace comments.

Domain 7 — Answer Key

Q	Ans	Explanation
1	A	The USE chain is required in addition to SELECT. MODIFY/EXECUTE/ownership are unrelated.
2	B	No DENY semantics; inherited grants persist until revoked at their source.
3	B	BROWSE = metadata visibility without data access.
4	B	Views run with the owner's privileges on base tables; readers need only the view (plus USE privileges).
5	B	Inheritance covers future children. No DENY; owners/MANAGE holders can grant.
6	B	Comments + tags are the native, searchable metadata mechanisms.
7	A	Column comments use ALTER TABLE ... ALTER COLUMN ... COMMENT (or COMMENT ON COLUMN t.c IS '...').
8	B	Tags are exposed in information_schema views.
9	B	Owners, MANAGE holders and parent owners/admins can grant; plain SELECT holders cannot.
10	B	Lineage capture needs UC-enabled compute (standard/dedicated access modes).
11	A	Governed tags are the controlled taxonomy (account-level, permissions, inheritance) that policies key on; free-form tags are just metadata.

DOMAIN 8

Debugging and Deploying

10% OF EXAM

Half diagnostics (where to look, what a symptom means, how to recover a run) and half delivery (Asset Bundles and Git-based CI/CD). Expect questions that describe an error and ask for the fastest safe recovery.

OFFICIAL OBJECTIVES (NEW EXAM)

- Identify diagnostic information using Spark UI, cluster logs, system tables and query profiles to troubleshoot errors.
- Analyse errors and remediate failed job runs with job repairs and parameter overrides.
- Use event logs and the Spark UI to debug pipelines and Spark workloads.
- Deploy Databricks resources using Declarative Automation Bundles (formerly Databricks Asset Bundles).
- Configure Git-based CI/CD workflows using Databricks Git folders (formerly Repos) to deploy notebooks and code.

8.1 Where to look

Source	What you find
Job run page → task → output/error	Notebook stack trace, exit value, which task failed, run parameters, compute used, duration.
Driver logs (stdout, stderr, log4j)	Python exceptions, print output, library install messages, OOM on driver ("Driver is up but not responsive"), "Spark context stopped".
Cluster event log	Start/terminate reasons, autoscaling, spot instance loss, init script failures, driver unresponsive.
Cluster log delivery	Configure a UC volume (or DBFS) destination so driver/executor logs persist after termination for post-mortems.
Spark UI	Failed stages with exception, task skew, spill, shuffle, executor loss, GC, SQL plan metrics.
Query profile	Per-operator bottlenecks for SQL workloads (Ch. 4/5).
System tables	Historical view across jobs/clusters (`job_task_run_timeline.result_state` , `node_timeline` CPU/memory, audit).
SDP event log / UI	Errors per flow, expectation failures, schema changes, autoscaling, refresh reason. Development vs production mode, "Validate" button.
`DESCRIBE HISTORY t`	Who/what wrote each version, operation metrics, isolation level — essential for "who overwrote my table" questions.

8.2 Common errors and their meaning

Error / symptom	Cause	Remedy
`ConcurrentAppendException` / `ConcurrentDeleteReadException`	Two writers conflicted on the same files (optimistic concurrency)	Partition/cluster so writers touch disjoint data; enable deletion vectors for row-level concurrency; retry with backoff; serialise writers
`FileNotFoundException` on a Delta read	VACUUM removed a file a query/stream still referenced; or external overwrite	Increase retention; restart stream from latest checkpoint; avoid `RETAIN 0`
`AnalysisException: A schema mismatch detected`	Writing columns that don't match the table	`mergeSchema` for additive changes; `overwriteSchema` for full overwrite; fix source
`OutOfMemoryError` / executor lost / "Task killed"	Skew, huge partitions, collect/toPandas, broadcasting too big a table, memory-hungry UDF	Spark UI to find the stage; fix skew; smaller broadcast; memory-optimised nodes; high-memory serverless
Streaming: `Detected a data update ... not supported`	Non-append change in a streaming source	`skipChangeCommits` , CDF, or full refresh
Streaming: checkpoint incompatibility	Changed source, sink, or a stateful operator's structure	New checkpoint (reprocess) — trigger interval and rate limits **can** change safely
Job `SKIPPED` tasks	Upstream task failed and run_if = ALL_SUCCESS	Repair run reruns failed + dependent skipped tasks

Error / symptom	Cause	Remedy
Library / `ModuleNotFoundError`	Dependency not on the compute	See Ch. 1.2

8.3 Repairing runs and overriding parameters

- **Repair run** (UI or `POST /api/2.2/jobs/runs/repair`): reruns only failed tasks and the tasks that depended on them (skipped), reusing successful results — cheaper than rerunning the whole job. You can pick specific tasks (`rerun_tasks`) and override **job parameters** for the repair.
- **Parameter overrides**: `run-now` accepts `job_parameters` that override job-level defaults for that run only (e.g. re-run for a backfill date). Task-level notebook params are legacy; prefer job parameters.
- Idempotent design (MERGE, overwrite by partition, Auto Loader checkpoints) is what makes repair safe. Non-idempotent tasks: disable retries (Ch. 1) and repair deliberately.
- SDP: **Full refresh** (all or selected tables) rebuilds from source; use after logic fixes or bad data. **Refresh** processes new data only. Full refresh on a streaming table over Kafka with expired retention loses data — beware.

```
databricks jobs repair-run --run-id 456 --rerun-all-failed-tasks \
  --json '{"job_parameters": {"run_date": "2026-09-21"}}'
databricks jobs run-now 123 --json '{"job_parameters": {"run_date": "2026-09-20", "mode": "backfill"}}'
```

8.4 Declarative Automation Bundles (formerly Databricks Asset Bundles)

Renamed from "Databricks Asset Bundles" in March 2026 (same CLI, same `databricks.yml`; the exam still says "formerly DABs"). Bundles are the standard for deploying jobs, pipelines, alerts, dashboards and supporting code as versioned, reviewable YAML plus source. The CLI turns the declaration into workspace resources (via Terraform under the hood).

```
databricks bundle init               # scaffold from a template
databricks bundle validate -t dev    # syntax + variable resolution
databricks bundle deploy   -t dev    # upload files, build wheel, create/update resources
databricks bundle run      -t dev etl_job   # run a job or pipeline defined in the bundle
databricks bundle summary  -t prod   # what is deployed where
databricks bundle destroy  -t dev
```

```
# resources/etl_job.yml
resources:
  jobs:
    etl_job:
      name: "etl-${bundle.target}"
      tasks:
        - task_key: ingest
          pipeline_task: {pipeline_id: ${resources.pipelines.orders_pipeline.id}}
        - task_key: publish
          depends_on: [{task_key: ingest}]
          python_wheel_task: {package_name: my_project, entry_point: publish}
          libraries: [{whl: ../dist/*.whl}]
          environment_key: default
      environments:
        - environment_key: default
          spec: {client: "2", dependencies: ["../dist/*.whl"]}
      schedule: {quartz_cron_expression: "0 0 6 * * ?", timezone_id: UTC}
```

- **Targets** hold per-environment differences: workspace host, `mode`, `run_as`, variable values, overrides of resource fields (e.g. bigger cluster in prod).
- **Variables** (`${var.catalog}`) and **substitutions** (`${bundle.target}`, `${workspace.current_user.userName}`) avoid duplicating YAML.
- **Development mode**: name prefix, paused schedules, concurrent runs allowed, deploys to the user's home folder. **Production mode**: validates that `run_as` is set, no prefixes, clean state, typically deployed only from CI on `main`.
- Bundle state is stored in the workspace; deploy is idempotent (only diffs applied). Manual UI edits to deployed resources are overwritten on next deploy — a common trap.
- Pipelines in bundles: `resources.pipelines` with `libraries: [{notebook: {path: ...}}]` or `glob` patterns, `catalog`, `schema` / `target`, `serverless: true`, `development: true`.

8.5 Git folders and CI/CD

- **Databricks Git folders** (formerly Repos) clone a remote repo into the workspace: branch, pull, commit, push, PR link. Each developer works on a feature branch in their own Git folder; PRs are reviewed in GitHub/GitLab/Azure DevOps/Bitbucket.
- Production should **not** run from a developer's Git folder. Two supported patterns: (1) CI pipeline runs tests then `databricks bundle deploy -t prod` using a service principal; (2) jobs configured with a **Git source** (repo URL + branch/tag/commit) so each run checks out that revision.
- Typical GitHub Actions flow: on PR → `pytest tests/unit` + `bundle validate`; on merge to `main` → `databricks/setup-cli` action, authenticate with OAuth M2M secrets (`DATABRICKS_CLIENT_ID/SECRET`), `bundle deploy -t prod`, optionally `bundle run integration_job`.
- Secrets never in the repo; environment-specific values via bundle variables/targets; tag releases for rollback (`bundle deploy` of a previous tag).
- Notebook diffs: store notebooks as source files (`.py` with `# Databricks notebook source` or `.ipynb`) for reviewable diffs.

Domain 8 — Practice Questions

11 questions

Q1 **A 12-task job failed at task 9 after 3 hours; tasks 10–12 were skipped. The fix is a parameter typo. What is the fastest, cheapest recovery?**

- **A.** Run the job again from the start with the corrected parameter.
- **B.** Use **Repair run** with the corrected job parameter; only task 9 and its dependents rerun.
- **C.** Create a new job containing only tasks 9–12.
- **D.** Increase retries to 3 and wait.

Q2 **A nightly MERGE job intermittently fails with `ConcurrentAppendException` since a new streaming job began writing to the same table. Which approach reduces conflicts without serialising the two workloads?**

- **A.** Run VACUUM before the MERGE.
- **B.** Enable deletion vectors (row-level concurrency) and, if possible, cluster/partition so the two writers touch disjoint files; add retry logic.
- **C.** Switch the table to Parquet.
- **D.** Disable AQE.

Q3 **A job cluster terminates mid-run and the task fails with "executor lost" errors. Where should the engineer look first to confirm whether spot instance reclamation caused it?**

- **A.** The SQL query history
- **B.** The cluster **event log** (and `system.compute.node_timeline`) for spot loss / termination reasons
- **C.** The pipeline event log
- **D.** `DESCRIBE HISTORY`

Q4 **A gold table's values changed unexpectedly overnight. Which is the quickest way to find which job or user wrote the change?**

- **A.** `DESCRIBE HISTORY gold.kpi` — shows operation, user, job/notebook and metrics per version; then `RESTORE` if needed.
- **B.** Read all notebooks in the workspace.
- **C.** Check the Spark UI of every cluster.
- **D.** Run VACUUM to see removed files.

Q5 **An SDP update fails. The UI shows a red flow for `silver_claims`. Which query gives the error details programmatically?**

- **A.** `SELECT * FROM system.billing.usage WHERE ...`
- **B.** `SELECT timestamp, message, details:error FROM event_log(TABLE(prod.silver.claims)) WHERE level='ERROR' ORDER BY timestamp DESC`
- **C.** `DESCRIBE DETAIL prod.silver.claims`
- **D.** `SHOW TBLPROPERTIES prod.silver.claims`

Q6 **After `databricks bundle deploy -t prod`, a job schedule that an operator had changed in the UI reverted to the YAML value. What explains this?**

- **A.** A bug in the CLI.
- **B.** Bundles own their deployed resources declaratively; manual UI changes are overwritten on the next deploy — the change must be made in the bundle source.
- **C.** The operator lacked CAN MANAGE.
- **D.** Production mode pauses all schedules.

Q7 **Which bundle target configuration is appropriate for production?**

- **A.** `mode: development` with `default: true`
- **B.** `mode: production`, `run_as: {service_principal_name: ...}`, deployed by CI from the main branch
- **C.** No target; deploy from each developer's laptop
- **D.** `mode: production` deployed as a personal user with personal access tokens stored in the repo

Q8 **A stage in the Spark UI shows 2,000 tasks each reading ~1 MB with high scheduling overhead. What is the likely problem and fix?**

- **A.** Too few executors; add nodes.
- **B.** Small files / over-partitioned input; compact with OPTIMIZE and rely on AQE/coalescing.
- **C.** Data skew; salt the key.
- **D.** Driver OOM; increase driver memory.

Q9 **Which CI pattern ensures a job always runs the reviewed code from the `main` branch rather than a developer's working copy?**

- **A.** Point the job's notebook path at the developer's Git folder.
- **B.** On merge to main, CI deploys the bundle to prod with a service principal (or configure the job with a Git source pinned to main/tag).
- **C.** Ask developers to push before each scheduled run.
- **D.** Export notebooks to DBFS weekly.

Q10 **A Structured Streaming job needs its trigger changed from 1 minute to 5 minutes and its `maxFilesPerTrigger` reduced. What must the engineer do about the checkpoint?**

- **A.** Nothing — trigger interval and rate-limit options can change across restarts with the same checkpoint.
- **B.** Delete the checkpoint and reprocess from the beginning.
- **C.** Create a new checkpoint and set `startingVersion = latest`.
- **D.** Convert the job to SDP.

Q11 **To make driver and executor logs of short-lived job clusters available for later analysis, what should be configured?**

- **A.** Cluster log delivery to a Unity Catalog volume path in the job cluster spec.
- **B.** A longer auto-termination.
- **C.** `spark.eventLog.enabled = false`
- **D.** Notebook output export.

Domain 8 — Answer Key

Q	Ans	Explanation
1	B	Repair reruns only failed and dependent tasks and accepts parameter overrides — the objective's wording.
2	B	Deletion vectors enable row-level concurrency; disjoint layouts and retries handle the rest. VACUUM/Parquet/AQE are irrelevant.
3	B	Termination and instance-loss reasons are in the cluster event log; node_timeline shows the node disappearing.
4	A	Delta history records the writer identity and operation per version; RESTORE reverts.
5	B	The event log TVF exposes error events with details. DESCRIBE DETAIL and TBLPROPERTIES show metadata only.
6	B	Declarative deployment reconciles to the YAML; edit the source, not the UI.
7	B	Production mode + service principal + CI is the documented pattern; secrets never live in the repo.
8	B	Many tiny tasks signal small files; compaction fixes it. Adding nodes increases cost without addressing the cause.
9	B	Deployment from CI (or Git source on the job) decouples production from developer folders.
10	A	Trigger and rate limits are not part of checkpointed state; only source/sink/stateful-plan changes require a new checkpoint.
11	A	Log delivery persists logs beyond cluster termination.

DOMAIN 9

Data Modeling

5% OF EXAM

Short domain (≈3 questions) that overlaps with Domain 5. The new wording is precise: map **partitioning to data grain**, align **clustering to access patterns**, keep **file sizes balanced through compaction** — for Delta *and* Iceberg — and build dimensional models where **materialized views** pre-compute aggregates and **Unity Catalog Metric Views** hold governed, reusable metric definitions.

OFFICIAL OBJECTIVES (NEW EXAM)

- Design scalable Delta/Iceberg table layouts for large data assets by mapping partitioning to data grain, aligning clustering to relationship access patterns, and maintaining balanced file sizes through compaction.
- Design dimensional models for analytical workloads, leveraging Materialized Views for pre-computed aggregation and Unity Catalog Metric Views for governed, reusable metric definitions.

9.1 Medallion layers as a modelling decision

Layer	Model	Table type / layout
Bronze	Raw, append-only, source schema + metadata (`_ingested_at`, file/offset)	Streaming table; cluster by ingestion time; keep rescued data; long retention
Silver	Cleaned, conformed, deduplicated, typed; one row per business entity/event; SCD applied	Streaming table (append events) or AUTO CDC target (entities); cluster by business key + date; constraints
Gold	Consumption models: star schemas, wide denormalised tables, aggregates, metric views	Materialized views or batch-built Delta; cluster by common filter columns; PK/FK declared

9.2 Physical layout decision tree

1. **Default: no partitioning + liquid clustering** on the 1–4 columns most used in filters/joins (e.g. date + key). Works for any cardinality, adapts as query patterns change, incremental maintenance.
2. **Partition** only when: table > ~1 TB, the column is low-cardinality (date/month/region), each partition holds ≥ 1 GB, and you need partition-level lifecycle (drop a month) or partition-disjoint writers. Never partition on ids, timestamps, or columns with thousands of values.
3. **Z-order** only for existing partitioned tables where liquid clustering is not adopted; it must be re-run and rewrites data.
4. **Generated columns** (`event_date DATE GENERATED ALWAYS AS (CAST(event_ts AS DATE))`) let queries on `event_ts` benefit from pruning on `event_date`.
5. File size: aim for 128 MB–1 GB per file; enable optimized writes/auto compaction; predictive optimisation for managed tables.

Iceberg layouts on Databricks

- **Managed Iceberg tables** (`USING iceberg`) follow the same principles: partition only at a coarse grain that matches how data is loaded and dropped (Iceberg **hidden partitioning** transforms such as `days(ts)`, `bucket(16, id)` keep partition columns out of queries), sort/cluster within files for the dominant filters, and rely on Unity Catalog to run **compaction** and **snapshot expiration** so file counts and metadata stay balanced.
- Delta tables exposed via **UniForm** keep Delta's layout (liquid clustering, DVs) while Iceberg readers see snapshots; deletion vectors are surfaced as position-delete files.
- Rule of thumb for both formats: "partitioning = grain and lifecycle; clustering = access pattern; compaction = keep 128 MB–1 GB files". Small files are the failure mode in either format.

9.3 Dimensional modelling on Delta and Iceberg

```
CREATE TABLE gold.dim_member (
  member_sk     BIGINT GENERATED ALWAYS AS IDENTITY,          -- surrogate key
  member_id     STRING NOT NULL,                              -- natural/business key
  name          STRING, plan_code STRING, region STRING,
  effective_from TIMESTAMP, effective_to TIMESTAMP, is_current BOOLEAN,   -- SCD2
  CONSTRAINT pk_member PRIMARY KEY (member_sk) RELY,
  CONSTRAINT chk_region CHECK (region IN ('US','EU','APAC'))
) CLUSTER BY (member_id);

CREATE TABLE gold.fact_claims (
  claim_id BIGINT NOT NULL, member_sk BIGINT NOT NULL, provider_sk BIGINT, service_date DATE,
  amount DECIMAL(12,2),
  CONSTRAINT pk_claim PRIMARY KEY (claim_id) RELY,
  CONSTRAINT fk_member FOREIGN KEY (member_sk) REFERENCES gold.dim_member (member_sk)
) CLUSTER BY (service_date, member_sk);
```

- **Star schema**: fact tables (measures, foreign keys, grain = one row per event/transaction/snapshot) surrounded by **dimensions** (descriptive attributes). Conformed dimensions are shared across facts. Snowflaking (normalised dims) is usually avoided in the lakehouse; denormalise for query speed.
- **Surrogate keys** via identity columns decouple from source ids and make SCD2 possible (each version gets a new SK). Identity columns cannot be used with concurrent writers that both insert (identity generation isn't concurrency-safe across writers) — generate SKs in one writer.
- **Slowly changing dimensions**: Type 1 overwrite (no history), Type 2 add a row with validity window + `is_current` (AUTO CDC does this), Type 3 keep previous value in a column. Facts join to dims on SK; point-in-time joins use `effective_from <= txn_ts < effective_to`.
- **Constraints**: `NOT NULL` and `CHECK` are **enforced** (writes fail). `PRIMARY KEY` / `FOREIGN KEY` are **informational** (not enforced) but document the model, power BI tools, and with `RELY` allow the optimiser to eliminate redundant joins/aggregations.
- **Metric views** / semantic layer and **materialized views** pre-aggregate gold for dashboards; **views** present column subsets/renames for consumers (the "share with different names" question → a view, not a clone).
- **Clones**: `DEEP CLONE` for full copies (backups, migrations, test environments; incremental re-clone syncs); `SHALLOW CLONE` for cheap metadata-only copies to experiment (source VACUUM can invalidate).

9.4 Materialized views and Unity Catalog Metric Views in the gold layer

The exam separates two jobs a gold layer must do: **pre-compute** heavy aggregations (materialized views) and **define metrics once** so every dashboard, Genie space and notebook computes "net revenue" identically (metric views). Use both together.

	Materialized view	Metric view
What it is	A physically stored query result, refreshed by a Declarative Pipeline (incrementally on serverless when possible)	A **semantic definition**: source, optional joins, dimensions and **measures** in YAML; computed at query time (optionally materialised)
Solves	Slow dashboards recomputing large joins/aggregations	Inconsistent KPI definitions across reports; reusable, governed measures
Query	Like a table	`SELECT dim, MEASURE(measure_name) FROM mv GROUP BY ALL` — measures must be wrapped in `MEASURE()`; the engine applies the right aggregation per grouping (no double counting of ratios)
Governance	UC table privileges	UC view privileges; lineage; tags/comments on measures; visible to Genie and AI/BI dashboards

```
CREATE OR REPLACE VIEW gold.claims_metrics
WITH METRICS LANGUAGE YAML AS $$
  version: 1.1
  source: gold.fact_claims
  joins:
    - name: member
      source: gold.dim_member
      on: source.member_sk = member.member_sk
  filter: service_date >= '2024-01-01'
  dimensions:
    - name: Service Month
      expr: DATE_TRUNC('MONTH', service_date)
    - name: Region
      expr: member.region
  measures:
    - name: Paid Amount
      expr: SUM(amount)
    - name: Net Revenue
      expr: SUM(amount) - SUM(adjustments)
    - name: Avg Paid per Member
      expr: SUM(amount) / COUNT(DISTINCT member_sk)     -- ratio computed correctly at any grain
$$;

SELECT `Region`, `Service Month`, MEASURE(`Net Revenue`), MEASURE(`Avg Paid per Member`)
FROM gold.claims_metrics GROUP BY ALL;
```

- Metric views support star and snowflake joins, window measures (YTD, moving averages), comments and tags per measure, and can be materialised for performance; requires DBR 16.4+/SQL warehouses.
- Exam pattern: "different SQL for the same KPI in every dashboard *and* slow queries" → metric view for the definition + materialized view (or aggregated gold table) for the pre-computed numbers. A comment or a shared SQL snippet is not governance.

WHY LIQUID CLUSTERING "BEATS" PARTITIONING AND Z-ORDER (THE EXAM'S LIST)

- No small-file explosion from high-cardinality or skewed keys; no need to predict cardinality up front.
- Clustering keys can be changed without rewriting; OPTIMIZE is incremental (Z-order rewrites, partitioning cannot change).
- Handles skewed distributions; better concurrency (row-level with DVs) for streaming + batch writers.
- Automatic mode (`CLUSTER BY AUTO`) chooses keys from query patterns for managed tables.

Domain 9 — Practice Questions

12 questions

Q1 **A new 8 TB fact table will be filtered by `service_date` and joined on `member_sk` (millions of distinct values). Which layout should be chosen?**

- **A.** Partition by `member_sk`
- **B.** `CLUSTER BY (service_date, member_sk)` with no partitioning
- **C.** Partition by `member_sk` and Z-order by `service_date`
- **D.** No layout; rely on the disk cache

Q2 **Which column is the best partitioning candidate for a 3 TB events table, if partitioning must be used?**

- **A.** `event_id` (unique)
- **B.** `event_timestamp`
- **C.** `event_date` (≈ 1,000 values, several GB each)
- **D.** `user_id` (10 million values)

Q3 **A dimension must retain the full history of address changes and support point-in-time joins from a fact table. Which design is correct?**

- **A.** SCD Type 1 with an updated_at column
- **B.** SCD Type 2 with surrogate key, `effective_from`, `effective_to`, `is_current`; the fact stores the surrogate key valid at transaction time
- **C.** SCD Type 3 with a `previous_address` column
- **D.** A separate table per year

Q4 **Which constraint types are enforced on write in Delta Lake?**

- **A.** PRIMARY KEY and FOREIGN KEY
- **B.** NOT NULL and CHECK
- **C.** UNIQUE only
- **D.** All constraints

Q5 **The analytics team's queries filter by `event_ts` (timestamp) but the table is clustered by a date. Without changing the queries, how can pruning be improved?**

- **A.** Add a generated column `event_date DATE GENERATED ALWAYS AS (CAST(event_ts AS DATE))` and cluster/partition on it; Delta maps filters on `event_ts` to it.
- **B.** Increase the number of stats columns to 64.
- **C.** Cast `event_ts` to STRING.
- **D.** Disable deletion vectors.

Q6 **What is the main purpose of declaring `PRIMARY KEY ... RELY` on a gold table?**

- **A.** To reject duplicate keys on insert
- **B.** To document the model for tools and let the optimiser assume uniqueness for join/aggregation elimination
- **C.** To enable liquid clustering automatically
- **D.** To create an index

Q7 **Two teams need the same aggregate data but with different column names and a subset of columns. Which is the simplest correct solution?**

- **A.** A view over the table with aliased, selected columns
- **B.** A DEEP CLONE refreshed hourly
- **C.** A second write path in the pipeline
- **D.** A CTAS copy plus a sync job

Q8 **Which statement about Z-ORDER is accurate?**

- **A.** It is incremental and only touches new files.
- **B.** It co-locates data for multiple columns but must be re-run via OPTIMIZE and rewrites files; it cannot be combined with liquid clustering on the same table.
- **C.** It changes the table's partitioning.
- **D.** It is required for data skipping to work at all.

Q9 **Surrogate keys are generated with `GENERATED ALWAYS AS IDENTITY`. Two jobs insert concurrently and one fails. Why?**

- **A.** Identity columns require Z-order.
- **B.** Identity value generation is not safe for concurrent inserters; only one writer should insert into an identity table at a time (or generate keys differently).
- **C.** Identity columns are not supported on managed tables.
- **D.** Deletion vectors must be disabled for identity columns.

Q10 **A gold model must serve dashboards with sub-second filters on region and month over billions of rows, and the aggregates are deterministic. Which approach is most appropriate?**

- **A.** Query the silver fact table directly with a large warehouse
- **B.** A materialized view (or aggregated gold table) at the region×month grain, clustered by (region, month)
- **C.** A Python UDF that caches results
- **D.** A shallow clone of silver

Q11 **Finance dashboards compute "net revenue" with slightly different SQL, producing inconsistent numbers, and each dashboard recomputes large joins at query time. Which implementation addresses both problems?**

- **A.** A metric view defining net revenue once (governed, reusable measures queried with `MEASURE()`) plus materialized views for the pre-aggregated results dashboards read.
- **B.** A comment on each dashboard describing the formula.
- **C.** A shared SQL snippet file every analyst can edit.
- **D.** A Python UDF copied into each notebook.

Q12 **An Iceberg table maintained in Unity Catalog and read by Trino is loaded daily and purged by month after seven years; queries filter on `member_id`. Which layout follows the guide's principles?**

- **A.** Partition at the month grain (matching load/purge), sort or cluster within partitions by `member_id` for the access pattern, and let UC compaction and snapshot expiration keep files balanced.
- **B.** Partition by `member_id`.
- **C.** No partitioning and no compaction to avoid rewrites.
- **D.** Partition by day and by `member_id`.

Domain 9 — Answer Key

Q	Ans	Explanation
1	B	Liquid clustering supports both filter and join columns regardless of cardinality; partitioning on a high-cardinality key is the anti-pattern.
2	C	Low cardinality, large partitions, and a natural lifecycle key — the official sample's logic.
3	B	Type 2 with validity intervals is the standard for history + point-in-time joins.
4	B	PK/FK are informational; NOT NULL and CHECK reject violating writes.
5	A	Generated columns give Delta a pruning column derived from the filtered expression.
6	B	RELY tells the optimiser it may trust the (unenforced) constraint.
7	A	Views are the simplest way to present renamed/subset columns without data copies (also the official sample's answer).
8	B	Z-order is a rewrite-based OPTIMIZE option, incompatible with liquid clustering.
9	B	Identity columns don't support concurrent transactions that both allocate values.
10	B	Pre-aggregation at the query grain plus clustering on filter columns gives dashboard-grade latency at low cost.
11	A	That is the official sample's logic: metric view = one governed definition; materialized view = pre-computed aggregates. Comments, snippets and copied UDFs are not governance.
12	A	Partitioning maps to grain/lifecycle (month), clustering to access pattern (member_id), compaction keeps file sizes healthy. High-cardinality or double partitioning explodes file counts.

CHAPTER 10

Cross-Domain Cheat Sheets

ALL DOMAINS

Delta Lake internals underpin every domain, and Structured Streaming is now an explicit objective in Domain 1 (see 1.7 for the full treatment). Review these pages the night before the exam, then run through the checklist.

10.1 Delta Lake internals

Transaction log

- `_delta_log/` holds JSON commits (`00000000000000000123.json`) with actions: `add`, `remove`, `metaData`, `protocol`, `txn` (streaming idempotency), `commitInfo`.
- Every 10 commits a Parquet **checkpoint** of the full state is written; readers load the last checkpoint + newer JSON.
- The log is the single source of truth: a file is part of the table only if it has an `add` without a later `remove`. Stats (min/max/null count) live in `add` actions.
- `RENAME` of a table only updates the metastore reference; the log and files are untouched.

ACID & concurrency

- **Optimistic concurrency**: read snapshot → write files → try to commit; if another commit touched the same data, conflict detection raises `Concurrent*Exception`; disjoint partitions/files don't conflict; **row-level concurrency** with DVs reduces conflicts for MERGE/UPDATE/DELETE.
- Isolation: **WriteSerializable** (default) and **Serializable** (`ALTER TABLE t SET TBLPROPERTIES ('delta.isolationLevel'='Serializable')`). Readers always see a consistent snapshot; no dirty reads.
- Multi-task jobs do **not** share a transaction: a failed task C leaves A and B's commits intact and C's partial writes are simply un-committed (Delta is atomic per table, per commit — not across tables/tasks).

Time travel, restore, clones

- `SELECT * FROM t VERSION AS OF 42` / `TIMESTAMP AS OF '2026-09-01'`; `DESCRIBE HISTORY t`; `RESTORE TABLE t TO VERSION AS OF 41` (a new commit; nothing is deleted).
- Retention limits: `delta.logRetentionDuration` (30 days default) and `delta.deletedFileRetentionDuration` (7 days) + VACUUM define how far back you can travel.
- **DEEP CLONE**: full copy of data + metadata; re-running syncs incrementally. **SHALLOW CLONE**: copies only metadata, references source files — cheap for tests; VACUUM on the source can break it.
- `CONVERT TO DELTA`, `DESCRIBE DETAIL` (location, num files, size, properties), `UNDROP TABLE` within 7 days for managed tables.

Schema & properties

- Schema enforcement rejects mismatched writes; `mergeSchema` adds columns (evolution), `overwriteSchema` replaces (with `mode("overwrite")`); `spark.databricks.delta.schema.autoMerge.enabled` for MERGE.
- Column mapping (`delta.columnMapping.mode = 'name'`) enables RENAME/DROP COLUMN without rewrite.
- Type widening, generated columns, identity columns, VARIANT type, collations.
- Properties you should recognise: `delta.enableChangeDataFeed`, `delta.enableDeletionVectors`, `delta.autoOptimize.optimizeWrite/autoCompact`, `delta.targetFileSize`, `delta.dataSkippingStatsColumns`, `delta.appendOnly` (blocks deletes/updates — a compliance option), `delta.minReaderVersion/minWriterVersion`.

10.2 Structured Streaming essentials (summary of 1.7)

Concept	What to know
Micro-batches & triggers	Default: next batch as soon as the previous finishes. `processingTime="30 seconds"` fixed interval; `availableNow=True` process everything then stop (batch on streaming engine; replaces deprecated `once`); `continuous` experimental (ms latency, limited ops). Real-time mode in SDP/serverless for very low latency.
Checkpoint	Stores source offsets, committed batch ids and state; enables **exactly-once** with replayable sources (Kafka, files) and idempotent sinks (Delta). Each query needs its own checkpoint. Changing source/sink/stateful operator structure = new checkpoint; changing trigger, rate limits, cluster size = fine.
Output modes	**append** (only new final rows; aggregations require a watermark), **update** (changed rows), **complete** (whole result each batch; aggregations only, unbounded state). Delta tables support append and complete (and update via foreachBatch MERGE).
Watermark	`withWatermark("event_ts", "10 minutes")`: how late data may be; enables state cleanup for windows, dedup, stream-stream joins. Rows older than watermark are dropped.

Concept	What to know
Windows	`window(col, "10 minutes")` tumbling; `window(col, "10 minutes", "5 minutes")` sliding; `session_window(col, "5 minutes")` session.
Stateful ops	Aggregations, dedup, stream-stream joins (need watermarks on both sides), `applyInPandasWithState` / `transformWithState` for custom state. State store default RocksDB on Databricks; monitor state size in the Streaming tab.
`foreachBatch`	Run arbitrary batch code per micro-batch: MERGE into Delta, write to several sinks, JDBC. Not idempotent by default — set `txnVersion` / `txnAppId` options on the Delta write, or MERGE, to be safe on retries.
Rate limiting	`maxFilesPerTrigger`, `maxBytesPerTrigger` (Delta/Auto Loader), `maxOffsetsPerTrigger` (Kafka). Smaller batches = more predictable latency and less spill.
Delta as source	Append-only by default; `ignoreDeletes`, `skipChangeCommits`, `startingVersion/startingTimestamp`, `readChangeFeed`.
Backlog & SLA	If batch duration > trigger interval the stream falls behind. Fixes: shorter trigger with smaller batches (smoother), more parallelism (shuffle partitions/cores), rate limits, remove skew — not "trigger once every N seconds" via job scheduler.

```
(spark.readStream.table("silver.events")
   .withWatermark("event_ts", "15 minutes")
   .groupBy(window("event_ts", "1 hour"), "region").agg(count("*").alias("n"))
 .writeStream.outputMode("append")
   .option("checkpointLocation", "/Volumes/prod/_chk/gold_hourly")
   .trigger(processingTime="5 minutes")
   .toTable("gold.hourly_by_region"))

def upsert(batch_df, batch_id):
    batch_df.createOrReplaceTempView("u")
    batch_df.sparkSession.sql("""MERGE INTO silver.customers t USING u ON t.id = u.id
      WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *""")
stream.writeStream.foreachBatch(upsert).option("checkpointLocation", chk).start()
```

10.3 Configs and commands to recognise on sight

Setting / command	Meaning
`spark.sql.shuffle.partitions`	Number of partitions after a shuffle (200 default; auto with AQE on Photon/serverless)
`spark.sql.files.maxPartitionBytes`	Max bytes per input split when reading files (128 MB default) — controls read parallelism/file sizing without shuffling
`spark.sql.autoBroadcastJoinThreshold`	Broadcast size limit (10 MB default; -1 disables)
`spark.sql.adaptive.enabled`, `...skewJoin.enabled`, `...advisoryPartitionSizeInBytes`	AQE switches and target post-shuffle partition size
`spark.databricks.delta.retentionDurationCheck.enabled`	Safety check preventing VACUUM < 7 days
`spark.databricks.delta.properties.defaults.*`	Default table properties for new tables in the session
`OPTIMIZE t [WHERE ...] [ZORDER BY (c)] [FULL]`	Compaction / clustering
`VACUUM t [RETAIN n HOURS] [DRY RUN]`	Remove unreferenced files; DRY RUN lists them
`REORG TABLE t APPLY (PURGE)`	Rewrite files to physically drop DV-deleted rows
`ANALYZE TABLE t COMPUTE STATISTICS [FOR ALL COLUMNS]`	CBO stats
`CREATE TABLE ... CLUSTER BY (a, b)` / `CLUSTER BY AUTO`	Liquid clustering keys / automatic
`ALTER TABLE t SET TBLPROPERTIES (...)`	CDF, DVs, retention, file size, append-only
`table_changes('t', ver)`, `event_log(TABLE(t))`	CDF read; SDP event log
`dbutils.secrets.get`, `dbutils.jobs.taskValues`, `dbutils.widgets.get`, `dbutils.notebook.exit`	Secrets, task values, parameters, notebook exit value (task output)

Setting / command	Meaning
`databricks bundle validate/deploy/run`, `databricks jobs run-now/repair-run/list-runs`, `databricks pipelines start-update --full-refresh`	CLI you should recognise
`CREATE POLICY p ON SCHEMA s COLUMN MASK fn TO grp EXCEPT admins FOR TABLES MATCH COLUMNS has_tag_value('pii','ssn') AS c ON COLUMN c USING COLUMNS (4)`, `SHOW EFFECTIVE POLICIES`	ABAC policy on governed tags (Ch. 6)
`CREATE VIEW v WITH METRICS LANGUAGE YAML AS $$ ... $$`, `SELECT MEASURE(m) FROM v GROUP BY ALL`	Metric view definition and query (Ch. 9)
`parse_json`, `col:path::TYPE`, `variant_get`, `try_variant_get`, `variant_explode`, `schema_of_variant`	VARIANT semi-structured handling (Ch. 3)
`ai_query('endpoint', prompt, returnType => 'STRING', failOnError => false)`, `ai_classify`, `ai_extract`, `ai_mask`	AI functions in SQL pipelines (Ch. 3)
`CREATE TABLE t (...) USING iceberg`, `delta.universalFormat.enabledFormats = 'iceberg'`	Managed Iceberg table / UniForm (Ch. 2, 9)
`WorkspaceClient().jobs.list_runs / get_run_output / repair_run`, `w.pipelines.list_pipeline_events`	Databricks SDK monitoring (Ch. 4)
`ALTER CATALOG c ENABLE PREDICTIVE OPTIMIZATION`, `CLUSTER BY AUTO`, `CACHE SELECT * FROM t`	Managed-table automation and Delta cache pre-warm (Ch. 5)

10.4 Last-day checklist

- ☐ I can explain streaming table vs materialized view trade-offs and when a source breaks a streaming table.
- ☐ I can write an AUTO CDC / APPLY CHANGES statement with KEYS, SEQUENCE BY, APPLY AS DELETE, SCD TYPE 2.
- ☐ I know the three expectation behaviours and the inverse-rule quarantine pattern.
- ☐ I know why liquid clustering beats partitioning/Z-order, and the partitioning rule (>1 TB, low cardinality, ≥1 GB partitions).
- ☐ I can describe what deletion vectors change for writes, reads, concurrency, and purging (REORG PURGE + VACUUM).
- ☐ I can read a query profile: files pruned, join type, shuffle, spill, and name the fix.
- ☐ I know the UC privilege chain (USE CATALOG + USE SCHEMA + SELECT), inheritance, BROWSE, MANAGE, ownership, view semantics.
- ☐ I can write a row filter and a column mask; I know is_account_group_member vs is_member; I can classify hashing/tokenisation/generalisation/suppression.
- ☐ I know the purge sequence: DELETE → REORG/OPTIMIZE → VACUUM after retention → propagate → streams skipChangeCommits/CDF.
- ☐ I know the system tables for cost (billing.usage + list_prices), audit (access.audit), jobs (lakeflow.job_task_run_timeline), queries (query.history).
- ☐ I know the Jobs API calls for list-runs / get / get-output / repair / run-now and the CLI equivalents.
- ☐ I can describe bundle targets, development vs production mode, and the CI flow with a service principal.
- ☐ I know what does and does not require a new streaming checkpoint.
- ☐ I know the differences D2D vs open Delta Sharing, WITH HISTORY, and what a foreign catalog is.
- ☐ I can describe the Lakeflow Connect database connector components (connection → gateway → staging → ingestion pipeline) and why the gateway must keep running.
- ☐ I know when a Clean Room is the answer rather than OpenSharing, and when a managed Iceberg table / UniForm is the answer rather than Delta.
- ☐ I can store and query VARIANT data (parse_json, colon paths, variant_get vs try_variant_get) and explain VARIANT vs STRUCT vs STRING.
- ☐ I can use ai_query / ai_classify / ai_mask inside a streaming table with failOnError and an expectation.
- ☐ I can write an ABAC policy from a governed tag and explain EXCEPT, scope inheritance, column-tag non-inheritance and compute requirements.
- ☐ I know the Lakehouse alert signals (metric views, anomaly detection, cost, warehouse health, audit, agent quality) and the SQL alert task for job branching.
- ☐ I can explain Predictive Optimization, CLUSTER BY AUTO and the Delta cache, and pick the right technique for an access pattern.

- ☐ I can define a metric view in YAML and query it with MEASURE(); I know why MVs and metric views are used together.

CHAPTER 11

Final Mixed Mock Exam

20 QUESTIONS · 40 MIN

Questions are mixed across domains in roughly the exam's proportions. Time yourself: 40 minutes for 20 questions mirrors the real pace. Score 17+ before booking.

M1 **A Structured Streaming job with a 10-second trigger normally processes each batch in 3 seconds. At peak, batches take up to 30 seconds and records must still be processed within 10 seconds. Holding cluster size constant, which change is most likely to meet the requirement?**

- **A.** Decrease the trigger interval to 5 seconds so batches are smaller and less likely to spill or back up.
- **B.** Use `availableNow` and schedule the job every 8 seconds.
- **C.** Increase the trigger interval to 60 seconds.
- **D.** Change `spark.sql.shuffle.partitions`; the trigger cannot change without a new checkpoint.

M2 **A bronze streaming table ingests JSON with Auto Loader. Silver must contain one current row per `device_id`, applying updates and deletes from the feed, ordered by `event_ts`. Which SDP construct?**

- **A.** A materialized view with `ROW_NUMBER()` over `device_id`.
- **B.** AUTO CDC INTO a streaming table, `KEYS (device_id)`, `SEQUENCE BY event_ts`, `APPLY AS DELETE WHEN op = 'D'`, `STORED AS SCD TYPE 1`.
- **C.** A streaming table with `dropDuplicates`.
- **D.** A view with `DISTINCT`.

M3 **A team needs a 1-TB JSON dataset written as Parquet files of ~512 MB in one pass without an extra shuffle. Which setting achieves the target file size?**

- **A.** `spark.sql.shuffle.partitions = 2048`
- **B.** `spark.sql.files.maxPartitionBytes = 512 MB` so each input split (and output file after narrow transforms) is ~512 MB
- **C.** `repartition(2048)` then write
- **D.** `coalesce(2048)` then write

M4 **An external Delta table receives a nightly `OPTIMIZE` and weekly `VACUUM` from a maintenance job that frequently fails and delays other work. Leadership wants to eliminate this maintenance. What is the recommended change?**

- **A.** Increase the maintenance job's retries.
- **B.** Migrate to a Unity Catalog managed table so predictive optimisation handles OPTIMIZE/VACUUM/ANALYZE automatically.
- **C.** Convert the table to Parquet.
- **D.** Run VACUUM with `RETAIN 0 HOURS` to make it faster.

M5 **Analysts querying a 5 TB clustered table by `customer_id` observe a scan that reads 95% of files. The table was created with `CLUSTER BY (event_date)`. What is the lowest-effort fix?**

- **A.** `ALTER TABLE t CLUSTER BY (event_date, customer_id)` and run `OPTIMIZE t FULL` once.
- **B.** Recreate the table partitioned by `customer_id`.
- **C.** Add a broadcast hint to every query.
- **D.** Enable Photon.

M6 **A job runs three notebook tasks: A, then B and C in parallel depending on A. A and B succeed; C fails midway after writing some rows to a Delta table. What is the state afterwards?**

- **A.** A and B's writes are committed; C's partial work is whatever C committed before failing (there is no cross-task rollback).
- **B.** All writes from A, B and C are rolled back.
- **C.** A and B's writes are rolled back because C failed.
- **D.** No writes are committed until all tasks succeed.

M7 **Which grant set allows group `bi_users` to read all current and future tables in `prod.gold` and nothing else in `prod`?**

- **A.** `GRANT SELECT ON CATALOG prod TO bi_users`
- **B.** `GRANT USE CATALOG ON CATALOG prod TO bi_users; GRANT USE SCHEMA, SELECT ON SCHEMA prod.gold TO bi_users`
- **C.** `GRANT SELECT ON SCHEMA prod.gold TO bi_users` only
- **D.** `GRANT ALL PRIVILEGES ON SCHEMA prod.gold TO bi_users`

M8 **A hospital must guarantee that after a patient's erasure request, no copy of the identifiers can be recovered from the lakehouse within 48 hours. Which combination is incomplete?**

- **A.** DELETE rows, REORG PURGE, shorten retention, VACUUM, delete the token mapping row, refresh downstream tables.
- **B.** DELETE rows and rely on the default 7-day VACUUM retention.
- **C.** DELETE rows, set `delta.deletedFileRetentionDuration = '1 day'`, REORG PURGE, VACUUM after 24h, full-refresh dependent streaming tables.
- **D.** DELETE rows in bronze/silver/gold, purge via REORG + VACUUM, invalidate shares containing the data.

M9 **A bronze table stores raw JSON payloads of varying shape in a single STRING column. The team wants efficient storage and fast field extraction without pre-declaring a fixed schema. What should they do?**

- **A.** Parse the payloads into a VARIANT column with `parse_json` and extract fields with `variant_get` / colon-path access.
- **B.** Explode the JSON into one row per key with a fixed STRUCT schema.
- **C.** Keep the STRING and use `regexp_extract` per field at query time.
- **D.** Convert the JSON to Parquet with a strict schema before ingestion.

M10 **A data scientist's notebook reads a masked column and sees full SSNs although a column mask exists. Investigation shows the notebook runs on a cluster with a legacy (non-UC) access mode. What is the explanation?**

- **A.** Column masks apply only to SQL warehouses.
- **B.** The cluster is not Unity Catalog-enabled, so UC governance (masks, filters, grants) is not enforced; move to standard/dedicated access mode or serverless.
- **C.** The mask function must be owned by the scientist.
- **D.** Masks require Photon.

M11 **Which Auto Loader configuration ingests CSV files, infers types, keeps unexpected columns without failing, and records their content for later review?**

- **A.** `cloudFiles.schemaEvolutionMode = "failOnNewColumns"`
- **B.** `cloudFiles.inferColumnTypes = true` with `cloudFiles.schemaEvolutionMode = "rescue"` so new columns land in `_rescued_data`
- **C.** `cloudFiles.schemaEvolutionMode = "none"`
- **D.** `mode = "DROPMALFORMED"`

M12 **A bundle deploy to `prod` fails validation with a message that `run_as` must be specified. Why?**

- **A.** Production mode requires jobs to run as a specified identity (typically a service principal) rather than the deploying user.
- **B.** Development mode was selected.
- **C.** The wheel was not built.
- **D.** Bundles cannot deploy to production.

M13 **A Spark UI stage shows total shuffle write of 900 GB for a join between a 1 TB fact table and a 12 MB dimension. The physical plan shows SortMergeJoin. Which change eliminates most of the shuffle?**

- **A.** Force a broadcast of the dimension (hint or raise `autoBroadcastJoinThreshold` above 12 MB).
- **B.** Coalesce the fact table to 10 partitions.
- **C.** Cache the fact table.
- **D.** Use a Python UDF to join.

M14 **An organisation with hundreds of PII-bearing tables wants a single, centrally managed rule that masks any column tagged `pii` across all current and future tables without altering each table. What should be implemented?**

- **A.** A per-table `ALTER COLUMN SET MASK` on every existing table.
- **B.** A row filter function attached to each schema.
- **C.** Dynamic views duplicating every base table with masking logic.
- **D.** An ABAC policy that applies a column mask to any column carrying the governed `pii` tag.

M15 **A pipeline validates `member_id IS NOT NULL`. Missing ids indicate a broken upstream extract and continuing would corrupt gold joins. Which expectation behaviour is appropriate?**

- **A.** Warn only
- **B.** `ON VIOLATION DROP ROW`
- **C.** `ON VIOLATION FAIL UPDATE` so the pipeline stops and alerts
- **D.** Remove the constraint and fix in gold

M16 **Which Jobs API call reruns only the failed tasks of run 987 with a corrected parameter value?**

- **A.** `POST /api/2.2/jobs/run-now {"job_id": 42}`
- **B.** `POST /api/2.2/jobs/runs/repair {"run_id": 987, "rerun_all_failed_tasks": true, "job_parameters": {"date": "2026-09-21"}}`
- **C.** `POST /api/2.2/jobs/runs/cancel {"run_id": 987}`
- **D.** `GET /api/2.2/jobs/runs/get-output?run_id=987`

M17 **A materialized view in SDP joins three silver tables that receive updates and deletes. A colleague proposes replacing it with a streaming table "for lower cost". What is the correct assessment?**

- **A.** Correct; streaming tables are always cheaper.
- **B.** Incorrect; streaming tables cannot consume updates/deletes from their sources, so results would be wrong or the flow would fail. Keep the MV (serverless incremental refresh) or use CDF-driven flows deliberately.
- **C.** Correct if `skipChangeCommits` is enabled, which applies the changes.
- **D.** Incorrect; MVs cannot join tables.

M18 **Which practice violates least privilege?**

- **A.** Granting SELECT on a de-identified view to analysts instead of the base table
- **B.** Making the ETL service principal the owner of production jobs
- **C.** Granting `ALL PRIVILEGES ON CATALOG prod` to `account users` to reduce access requests
- **D.** Giving operators CAN MANAGE RUN on jobs

M19 **An engineer must continuously replicate insert, update and delete events from an operational PostgreSQL database into the lakehouse with minimal custom code. Which approach fits?**

- **A.** A nightly JDBC full-table read that overwrites the target.
- **B.** A Lakeflow Connect managed PostgreSQL connector ingesting CDC into the lakehouse.
- **C.** Export to CSV in cloud storage and ingest with Auto Loader.
- **D.** OpenSharing the PostgreSQL tables into the lakehouse.

M20 **A daily job must publish a gold table only when a freshness alert is not triggered, otherwise notify on-call — inside the job DAG. Which native design applies?**

- **A.** A SQL alert task followed by an If/else task that branches on the alert task's result value.
- **B.** A notebook polling the alert API in a loop.
- **C.** A job health rule on run duration.
- **D.** Two jobs on the same schedule.

Mock Exam — Answer Key

Q	Ans	Explanation
M1	**A**	Smaller, more frequent batches reduce per-batch volume and spill; the trigger can change without a new checkpoint (D false); scheduling a job every 8 s is not how streaming works.
M2	**B**	SCD Type 1 via AUTO CDC handles ordering and deletes; MV/DISTINCT approaches don't apply deletes correctly.
M3	**B**	Input split size drives output file size for narrow pipelines with no shuffle; repartition shuffles, coalesce can't increase partitions, shuffle partitions only affect wide ops.
M4	**B**	The objective: managed tables + predictive optimisation remove maintenance jobs.
M5	**A**	Change clustering keys in place; FULL reclusters existing data. Partitioning by a high-cardinality key is wrong.
M6	**A**	Delta transactions are per-table commit; there is no distributed transaction across tasks.
M7	**B**	USE CATALOG + USE SCHEMA + SELECT at the schema level, inherited by future tables. A grants the whole catalog; C lacks USE privileges; D over-grants.
M8	**B**	Default retention keeps deleted data for 7 days and ignores DVs, downstream copies and mappings.
M9	**A**	VARIANT stores semi-structured data efficiently and supports schema-flexible, fast path queries; the others are rigid or slow.
M10	**B**	UC enforcement requires UC-enabled compute; legacy access modes bypass masks and filters (and shouldn't reach UC tables).
M11	**B**	Rescue mode captures new/unexpected columns in `_rescued_data` without failing; failOnNewColumns fails; none ignores them.
M12	**A**	Production mode validation requires explicit run_as.
M13	**A**	A 12 MB dimension should broadcast; the fact table then needs no shuffle.
M14	**D**	An ABAC policy keyed on a governed tag applies wherever the tag is present, including tables created later; per-table masks and duplicated views don't scale.
M15	**C**	An invariant whose violation signals a systemic upstream problem warrants failing the update.
M16	**B**	runs/repair with rerun_all_failed_tasks and job_parameters.
M17	**B**	Streaming tables need append-only sources; skipChangeCommits skips (does not apply) changes.
M18	**C**	Blanket ALL PRIVILEGES to everyone is the opposite of least privilege; the other three are recommended patterns.
M19	**B**	Lakeflow Connect database connectors deliver CDC including deletes with minimal code; the alternatives are full reloads or don't apply to an RDBMS.
M20	**A**	The SQL alert task exposes its evaluation state as a task value; If/else branches on it (the "Lakeflow Job branching" objective).

SCORING GUIDE

18–20: book the exam. 15–17: re-read the chapters for the domains you missed, retake in a few days. Below 15: work through the Academy pathway labs hands-on before another attempt. Good luck, Mike.